

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Διαμαντή Αναστασία | Δήμου Δέσποινα

ΤΕΚΜΗΡΙΩΣΗ ΒΑΣΗΣ ΔΕΔΟΜΕΝΩΝ

Γενικό Νοσοκομείο «Υγειόπολης»

Σύστημα Διαχείρισης Νοσοκομείου — PostgreSQL

Μάθημα: Βάσεις Δεδομένων

Ακαδημαϊκό Έτος: 2025–2026

36 πίνακες · 15 views · 42 triggers · ~22.000 εγγραφές · 15 queries SQL

Περιεχόμενα

Περιεχόμενα	2
Κεφάλαιο A: Σχεδιασμός Βάσης Δεδομένων	6
A.1 Ανάλυση Απαιτήσεων	6
Βασικές σχεδιαστικές αποφάσεις που ελήφθησαν κατά την ανάλυση:	6
A.2 Διάγραμμα Οντοτήτων-Συσχετίσεων (ER)	7
A.2.1 Συνολικό ER Διάγραμμα	7
Οι κεντρικές οντότητες του ER διαγράμματος και οι σχέσεις τους:	8
A.3 Σχεσιακό Σχήμα (Relational Schema)	9
A.4 Στατιστικά Δεδομένων	11
A.4.1 Παρατηρήσεις	12
A.4.2 Διαδικασία Παραγωγής Δεδομένων	13
A.4.3: Παραδοχές (Assumptions)	14
Παραδοχές για το Πρόγραμμα Βαρδιών	14
Παραδοχές για τη Φαρμακευτική Αγωγή	14
Παραδοχές για τα Πολυμέσα (Εικόνες)	14
A.5 Περιβάλλον Υλοποίησης	14
Κεφάλαιο B: Ανάλυση Κώδικα (install.sql)	16
Ενότητα 1: REFERENCE DATA	17
Προδιαγραφές Απαιτήσεων Συστήματος	17
Πίνακας 1.1: ken_code	17
Πίνακας 1.2: icd10_code	19
Πίνακας 1.3: country	20
Πίνακας 1.4: insurer	21
Πίνακας 1.5: active_substance	23
Πίνακας 1.6: drug	24
Πίνακας 1.7: drug_substance	25
Ενότητα 2: DEPARTMENTS & BEDS	27
Προδιαγραφές Απαιτήσεων Συστήματος	27
Πίνακας 2.1: department	27
Πίνακας 2.2: department_specialty	29
Πίνακας 2.3: department_beds	30
Ενότητα 3: STAFF	32
Προδιαγραφές Απαιτήσεων Συστήματος	32
Πίνακας 3.1: staff	32
Πίνακας 3.2: doctor_specialty	34
Πίνακας 3.3: doctor	35
Πίνακας 3.4: doctor_department	37
Πίνακας 3.5: nurse	38
Πίνακας 3.6: admin_staff	39

Πίνακας 3.7: admin_role	40
Ενότητα 4: PATIENTS	41
Προδιαγραφές Απαιτήσεων Συστήματος	41
Πίνακας 4.1: patient	41
Πίνακας 4.2: emergency_contact	45
Πίνακας 4.3: patient_emergency_contact (junction table)	46
Πίνακας 4.4: patient_allergy (weak entity)	48
Ενότητα 5: Triage	50
Προδιαγραφές Απαιτήσεων Συστήματος	50
Πίνακας 5.1: triage	50
Ενότητα 6: HOSPITALIZATION	53
Προδιαγραφές Απαιτήσεων Συστήματος	53
Πίνακας 6.1: hospitalization	53
Πίνακας 6.3: hospitalization_staff	56
Ενότητα 7: PRESCRIPTIONS	58
Προδιαγραφές Απαιτήσεων Συστήματος	58
Πίνακας 7.1: prescription	58
Ενότητα 8: LAB & PROCEDURES	61
Προδιαγραφές Απαιτήσεων Συστήματος	61
Πίνακας 8.1: medical_procedure	61
Πίνακας 8.2: lab_exam	62
Πίνακας 8.3: operating_room	62
Πίνακας 8.4: hospitalization_procedure	63
Πίνακας 8.5: procedure_participant	64
Πίνακας 8.6: hospitalization_lab	65
Ενότητα 9: SHIFT SCHEDULING	68
Προδιαγραφές Απαιτήσεων Συστήματος	68
Πίνακας 9.1: shifts	68
Πίνακας 9.2: shift_staff	69
Ενότητα 10: REVIEWS	72
Προδιαγραφές Απαιτήσεων Συστήματος	72
Πίνακας 10.1: patient_hospitalization_review	72
Πίνακας 10.2: patient_doctor_review	74
Ενότητα 11: MEDIA	76
Προδιαγραφές Απαιτήσεων Συστήματος	76
Πίνακας 11.1: image	76
Κεφάλαιο Γ: Αναλυτική Τεκμηρίωση Views	80
Γ.1 Ενότητα 1 - Hospitalization Departments	80
v_department_full_profile	80
v_available_beds	81
Γ.2 Ενότητα 2 — Staff	82

v_staff_full_profile	82
v_department_directors	84
Γ.3 Ενότητα 4 — Patients	84
v_patient_full_profile	84
Γ.4 Ενότητα 5 — Triage	86
v_triage_queue	86
Γ.5 Ενότητα 6 — Hospitalization	86
v_active_hospitalizations	86
v_hospitalization_cost_breakdown	87
Γ.6 Ενότητα 8 — Prescriptions	89
v_patient_prescriptions	89
Γ.7 Ενότητα 9 — Shift Scheduling	90
v_triage_nurses	90
v_monthly_shift_summary	91
Γ.8 Ενότητα 10 — Reviews	92
v_hospitalization_review_avg	92
v_doctor_review_avg	92
Κεφάλαιο Δ: Triggers - Πλήρης Αναφορά	94
Δ.1 Ενότητα 1 — Hospital Departments	96
#01 trg_department_director_check	96
#02 trg_doctor_specialty_in_department	98
#03 trg_one_director_per_department	99
Δ.2 Ενότητα 2 — Staff ISA Integrity	100
#04 trg_staff_has_subtype (CONSTRAINT TRIGGER — DEFERRABLE)	100
#05/#06/#07 trg_doctor/nurse/admin_matches_type	100
#08 trg_doctor_supervisor_check	102
#09 trg_supervisor_same_department (CONSTRAINT — DEFERRABLE)	104
Δ.3 Ενότητα 4 — Patients	105
#10 trg_patient_updated_at	105
#11 trg_patient_discharge_check	105
#12 trg_patient_prescription_check	106
#13 trg_patient_amka_gr_check	106
Δ.4 Ενότητα 5 — Triage	107
#14 trg_triage_nurse_in_emergency	107
#15 trg_triage_single_active	108
#16 trg_triage_create_hospitalization	109
Δ.5 Ενότητα 6 — Hospitalization	110
#17 trg_hosp_bed_dept_match	110
#18 trg_hosp_bed_status	111
#19 trg_hosp_bed_overlap	112
#20 trg_hosp_discharge_diagnosis	113

#21 trg_hosp_total_cost	114
Δ.6 Ενότητα 7 — Lab & Procedures	115
#23 trg_procedure_during_hosp	115
#24 trg_lab_exam_during_hosp	116
Δ.7 Triggers Procedures	117
#25 trg_surgery_room_overlap	117
#26 trg_surgery_doctor_overlap	118
#27 trg_surgery_assistant_overlap	119
Δ.8 Ενότητα 8 — Prescriptions	120
#28 trg_allergy_check	120
#29 trg_prescription_doctor_guard	121
#30 trg_prescription_within_hosp	121
Δ.9 Ενότητα 9 — Shift Scheduling	122
#31 trg_shift_staff_belongs_to_department	122
#32 trg_shift_min_staff (DEFERRABLE)	124
#33 trg_shift_resident_needs_senior (DEFERRABLE)	125
#34 trg_shift_monthly_limit	127
#35 trg_shift_rest_8h	128
#36 trg_shift_max_3_night	130
Δ.10 Ενότητα 10 — Reviews	132
#37 trg_hr_after_discharge	132
#38 trg_dr_after_discharge	132
#39 trg_dr_prescribed_doctor	133
Δ.11 Ενότητα 11 — Media	134
#40 trg_image_entity_check	134
Κεφάλαιο E: Ευρετήρια (Indexes)	136
E.1 Ενότητα 0 — Reference Data	136
E.2 Ενότητα 1 — Hospital Departments	136
E.3 Ενότητα 2 — Staff	137
E.4 Ενότητα 3 — Pharmacy	138
E.5 Ενότητα 4 — Patients	138
E.6 Ενότητα 5 — Triage	139
E.7 Ενότητα 6 — Hospitalization	140
E.8 Ενότητα 7 — Lab & Procedures	141
E.9 Ενότητα 8 — Prescriptions	142
E.10 Ενότητα 9 — Shifts	142
E.11 Ενότητα 10–11 — Reviews & Media	143
E.12 Συνοπτικός Πίνακας Αντιστοίχισης Index ↔ Query	143
Κεφάλαιο E: Ερωτήματα SQL	145
Παραδοχές για τα Ερωτήματα	145
Q01: Έσοδα ανά Τμήμα, Έτος και KEN Κωδικός	145

Q02: Ιατροί ανά Ειδικότητα με Εφημερίες και Επεμβάσεις	147
Q03: Ασθενείς με Πολλαπλές Νοσηλείες στο Ίδιο Τμήμα	148
Q04: Ερώτημα με EXPLAIN/EXPLAIN ANALYZE	149
Q05: Νέοι Ιατροί (< 35 ετών) με τις Περισσότερες Χειρουργικές Επεμβάσεις	151
Q06: Ερώτημα με EXPLAIN/EXPLAIN ANALYZE - Δεύτερο	152
Q07: Δραστικές Ουσίες: Αλλεργίες Ασθενών και Φάρμακα	154
Q08: Προσωπικό χωρίς Εφημερία σε Συγκεκριμένη Ημερομηνία και Τμήμα	155
Q09: Ασθενείς με Ίδιο Αριθμό Νοσηλευτικών Ημερών	155
Q10: Top-3 Ζεύγη Δραστικών Ουσιών που Συνταγογραφήθηκαν Μαζί	157
Q11: Ιατροί με Τουλάχιστον 5 Λιγότερες Επεμβάσεις από τον Πρώτο	158
Q12: Κατανομή Προσωπικού ανά Τμήμα, Βάρδια και Υποκατηγορία	159
Q13: Ιεραρχία Εποπτείας Ιατρών (Αναδρομικό CTE)	160
Q14: Κατηγορίες ICD-10 με Ίδιο Αριθμό Εισαγωγών σε Συνεχόμενα Έτη	161
Q15: Κατανομή Triage ανά Επίπεδο Επείγοντος	162
Κεφάλαιο Z: Διεπαφή Χρήστη (Hospital Management UI)	165
Z.1 Αρχιτεκτονική Εφαρμογής	165
Z.2 Dashboard — Κεντρική Οθόνη	166
Z.3 Μητρώο Ασθενών	167
Z.4 Τριάζ / Τμήμα Επειγόντων Περιστατικών	169
Z.5 Νοσηλείες	170
Z.6 Διαχείριση Προσωπικού	171
Z.6.1 Ιατρικό Προσωπικό	171
Z.6.2 Νοσηλευτικό & Διοικητικό Προσωπικό	172
Z.7 Συνταγογράφηση Φαρμάκων	173
Z.8 Εφημερίες & Προγραμματισμός Βαρδιών	174
Z.9 Χειρουργεία & Επεμβάσεις	176
Z.10 Τεχνικά Χαρακτηριστικά	177
Κεφάλαιο H: Χρήση AI για το Project	178
H.1 Φάση 1 - Σχεδιασμός & Αρχιτεκτονική	178
H.1 Φάση 2 - Ανάπτυξη & Δεδομένα	178
H.1 Φάση 3 - Εφαρμογή & UI	178
H.1 Φάση 4 - Documentation	178

Κεφάλαιο Α: Σχεδιασμός Βάσης Δεδομένων

Το κεφάλαιο αυτό παρουσιάζει τη διαδικασία σχεδιασμού της βάσης δεδομένων του Γενικού Νοσοκομείου «Υγειόπολης», από την ανάλυση απαιτήσεων και το διάγραμμα οντοτήτων-συσχετίσεων (ER) έως τη μετάφρασή του σε σχεσιακό σχήμα (Relational Schema) και την τελική υλοποίηση σε PostgreSQL. Παρουσιάζεται επίσης η στατιστική εικόνα των δεδομένων που φορτώθηκαν στη βάση.

A.1 Ανάλυση Απαιτήσεων

Οι κεντρικές λειτουργικές απαιτήσεις που καθόρισαν τη δομή της βάσης ήταν:

1. Διαχείριση προσωπικού τριών κατηγοριών (ιατροί, νοσηλευτές, διοικητικοί) με ISA ιεραρχία.
2. Οργάνωση τμημάτων νοσοκομείου με κλίνες, ειδικότητες και διευθυντές.
3. Διαδικασία διαλογής (triage) για κάθε ασθενή που προσέρχεται στα Επείγοντα Περιστατικά.
4. Πλήρης διαχείριση νοσηλείας: εισαγωγή, κλίνη, διάγνωση ICD-10, κωδικός KEN, κόστος.
5. Συνταγογράφηση φαρμάκων με αυτόματο έλεγχο αλλεργιών.
6. Ιατρικές πράξεις και εργαστηριακές εξετάσεις ανά νοσηλεία.
7. Προγραμματισμός βαρδίων με αυτόματη επιβολή ορίων (μηνιαίο όριο, ανάπαυση 8ω, 3 νυχτερινές).
8. Αξιολόγηση νοσηλείας και ιατρών από ασθενείς μετά την έξοδο.

Βασικές σχεδιαστικές αποφάσεις που ελήφθησαν κατά την ανάλυση:

ISA ιεραρχία προσωπικού

Αντί για τρεις ανεξάρτητους πίνακες (doctor, nurse, admin), επιλέχθηκε supertype/subtype αρχιτεκτονική με κεντρικό πίνακα staff. Αυτό εξαλείφει την επανάληψη κοινών πεδίων και επιτρέπει ενιαία αναφορά από πίνακες όπως shifts και hospitalization_staff.

Φυσικά κλειδιά για lookup tables

Για πίνακες αναφοράς με εξωτερικά ορισμένα, σταθερά και μοναδικά αναγνωριστικά (department_name, icd10_code, ken_code, drug_code, active_substance) χρησιμοποιήθηκαν φυσικά PK αντί για surrogate keys (SERIAL), για αναγνωσιμότητα και αποφυγή περιττών JOINS.

Υποστήριξη emergency περιστατικών

Ο πίνακας patient ορίζει σχεδόν όλα τα δημογραφικά πεδία ως NULL, ώστε να μπορεί να δημιουργηθεί εγγραφή ασθενή άμεσα κατά την άφιξη. Η πληρότητα στοιχείων επιβάλλεται μόνο πριν την έξοδο, μέσω trigger.

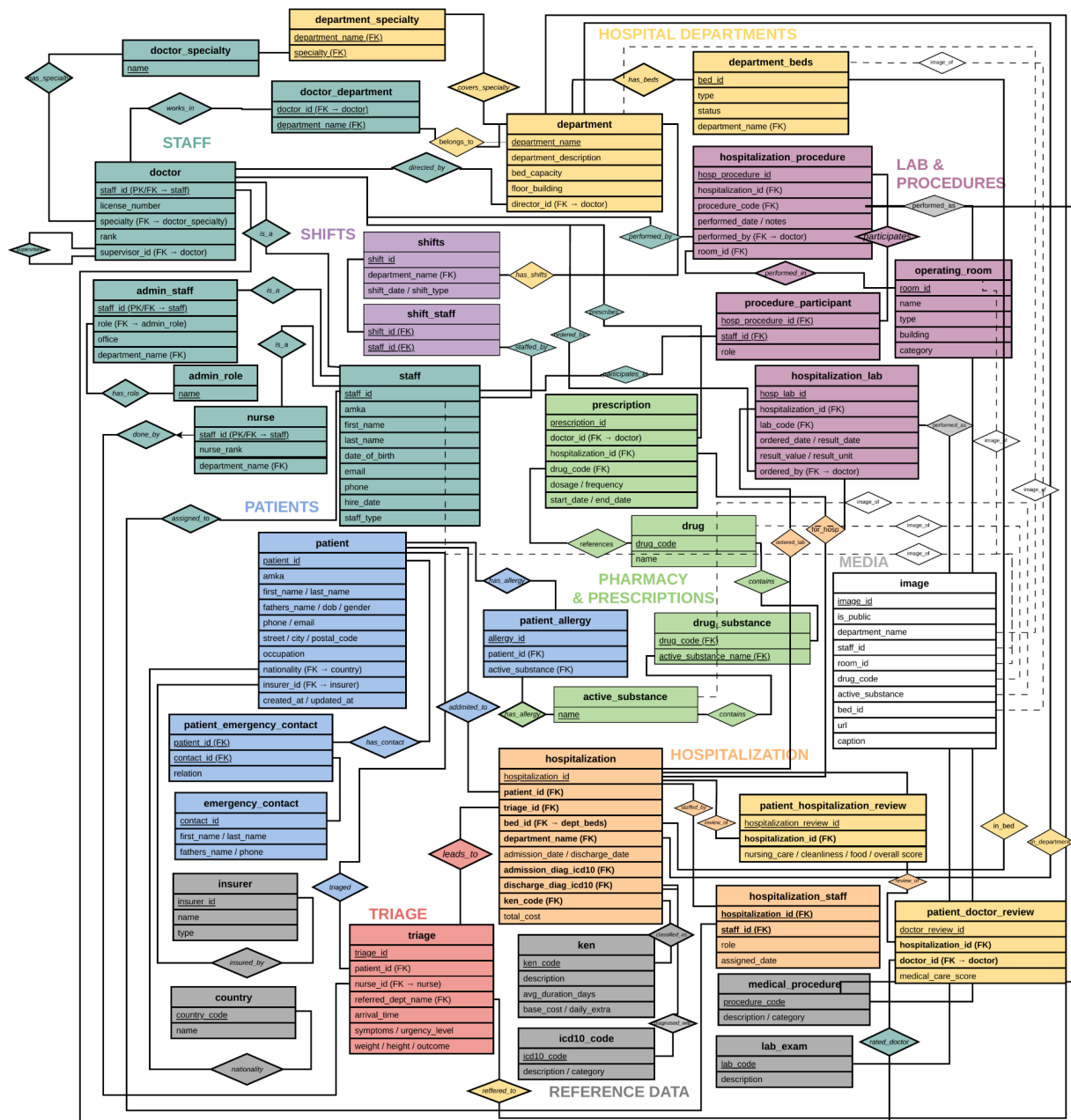
Αυτόματος υπολογισμός κόστους νοσηλείας

Το total_cost υπολογίζεται αυτόματα από trigger (trg_hosp_total_cost) κατά τη συμπλήρωση της ημερομηνίας εξόδου, βάσει του κωδικού KEN και των ημερών παραμονής, χωρίς χειροκίνητη παρέμβαση.

A.2 Διάγραμμα Οντοτήτων-Συσχετίσεων (ER)

Το ER διάγραμμα αποτυπώνει τις οντότητες του συστήματος, τις μεταξύ τους συσχετίσεις και τους περιορισμούς πολλαπλότητας. Σχεδιάστηκε σε δύο επίπεδα: ένα συνοπτικό διάγραμμα ανά ενότητα και ένα συνολικό διάγραμμα ολόκληρης της βάσης.

A.2.1 Συνολικό ER Διάγραμμα



Οι κεντρικές οντότητες του ER διαγράμματος και οι σχέσεις τους:**STAFF (ISA) → DOCTOR / NURSE / ADMIN_STAFF**

Η ISA ιεραρχία εκφράζεται με τον supertype STAFF και τρεις subtypes. Κάθε μέλος προσωπικού ανήκει υποχρεωτικά σε ακριβώς έναν subtype (ολική, αποκλειστική συμμετοχή).

PATIENT — TRIAGE — HOSPITALIZATION

Κάθε ασθενής περνά από τη διαλογή (TRIAGE) πριν τη νοσηλεία (HOSPITALIZATION). Η σχέση είναι 1:1 μεταξύ TRIAGE και HOSPITALIZATION — κάθε νοσηλεία προκύπτει από ακριβώς μία διαλογή.

HOSPITALIZATION — DEPARTMENT_BEDS — DEPARTMENT

Κάθε νοσηλεία ανατίθεται σε μία κλίνη (DEPARTMENT_BEDS) συγκεκριμένου τμήματος (DEPARTMENT). Το σύνθετο FK (bed_id, department_name) εξασφαλίζει συνοχή.

HOSPITALIZATION — PRESCRIPTION — DRUG

Κατά τη νοσηλεία, ο ιατρός μπορεί να συνταγογραφήσει φάρμακα. Το trigger trg_allergy_check ελέγχει αυτόματα αλλεργίες πριν κάθε συνταγογράφηση.

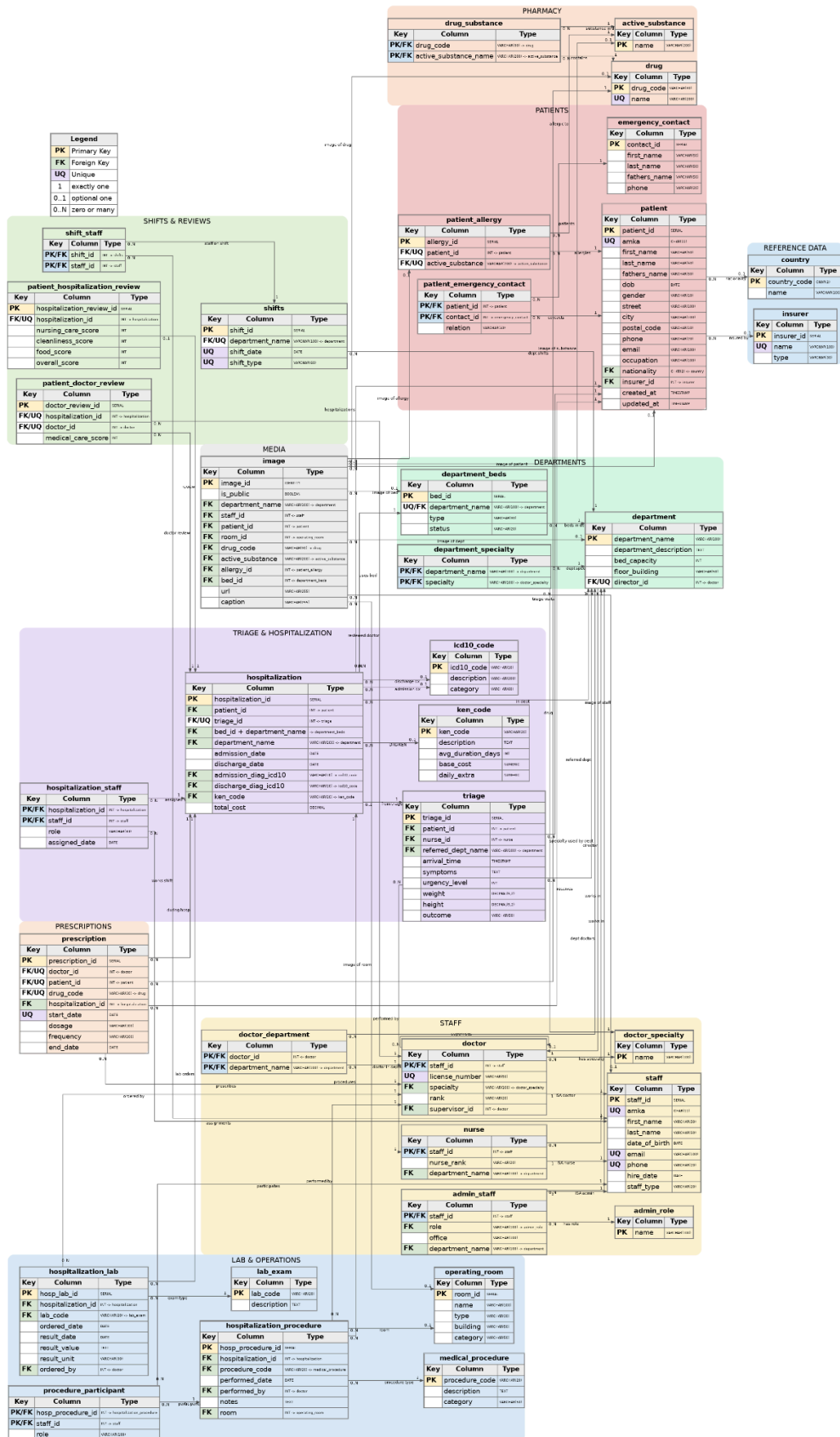
SHIFTS — SHIFT_STAFF — STAFF

Η M:N σχέση βαρδιών-προσωπικού υλοποιείται μέσω του junction table SHIFT_STAFF, με έξι triggers που επιβάλλουν τους κανόνες εφημεριών.

A.3 Σχεσιακό Σχήμα (Relational Schema)

Το σχεσιακό σχήμα προέκυψε από τη μετάφραση του ER διαγράμματος σε σχεσιακούς πίνακες, ακολουθώντας τους κανόνες κανονικοποίησης (3NF). Το τελικό σχήμα αποτελείται από 30 πίνακες, 15 views και 42 triggers.

Hospital Database - Detailed Grouped Relational Diagram (PK/FK/UQ + Cardinalities)



Α.4 Στατιστικά Δεδομένων

Ο παρακάτω πίνακας παρουσιάζει τον αριθμό εγγγραφών που φορτώθηκαν στη βάση δεδομένων ανά πίνακα. Τα δεδομένα δημιουργήθηκαν συνθετικά (synthetic data generation) με βάση τις απαιτήσεις της εκφώνησης, εξασφαλίζοντας αναφορική ακεραιότητα και ρεαλιστική κατανομή τιμών.\

Σύνολο εγγγραφών: ~22.000

Πίνακας	Πλήθος Εγγγραφών	Σχόλιο
country	30	Χώρες (υποσύνολο ISO 3166-1 alpha-2)
insurer	13	Ασφαλιστικοί φορείς (δημόσιοι, ιδιωτικοί, ΕΟΧ, ανασφάλιστοι)
ken_code	760	Κωδικοί KEN (DRG) για κοστολόγηση νοσηλείας
icd10_code	5.670	Κωδικοί ICD-10 για διαγνώσεις εισαγωγής/εξόδου
drug	40	Φάρμακα από τον κατάλογο EMA Article 57
active_substance	30	Δραστικές ουσίες από το EMA Article 57
drug_substance	40	Συνδέσεις φαρμάκου ↔ δραστικής ουσίας
department	15	Τμήματα νοσοκομείου
department_beds	406	Κλίνες ανά τμήμα
department_specialty	31	Αντιστοιχίσεις τμημάτων με ειδικότητες (M:N)
doctor_specialty	25	Αναγνωρισμένες ιατρικές ειδικότητες
staff	291	Σύνολο προσωπικού (ιατροί + νοσηλευτές + διοικητικοί)
doctor	91	Ιατροί: Διευθυντές, Επιμελητές Α/Β, Ειδικευόμενοι
doctor_department	111	Αναθέσεις ιατρών σε τμήματα (M:N)
nurse	150	Νοσηλευτές ανά τμήμα
admin_role	7	Ρόλοι διοικητικού προσωπικού
admin_staff	50	Διοικητικό προσωπικό
patient	310	Ασθενείς — δημογραφικά στοιχεία
patient_phone	238	Τηλέφωνα ασθενών
patient_email	200	Email ασθενών
emergency_contact	40	Πρόσωπα επαφής έκτακτης ανάγκης
patient_emergency_contact	44	Συνδέσεις ασθενών με οικείους (M:N)
patient_allergy	44	Αλλεργίες ασθενών σε δραστικές ουσίες
triage	864	Διαλογές στα Επείγοντα Περιστατικά
hospitalization	565	Νοσηλείες ασθενών
hospitalization_staff	1.207	Αναθέσεις προσωπικού ανά νοσηλεία

Πίνακας	Πλήθος Εγγραφών	Σχόλιο
medical_procedure	5.394	Κωδικοί ιατρικών πράξεων/επεμβάσεων
lab_exam	71	Κωδικοί εργαστηριακών εξετάσεων
operating_room	10	Χειρουργεία και αίθουσες επεμβάσεων
hospitalization_procedure	334	Εκτελεσμένες ιατρικές πράξεις ανά νοσηλεία
procedure_participant	647	Συμμετέχοντες σε πράξεις
hospitalization_lab	324	Παραγγελθείσες εργαστηριακές εξετάσεις
prescription	450	Συνταγογραφήσεις φαρμάκων κατά νοσηλεία
shifts	900	Βάρδιες ανά τμήμα/ημερομηνία/τύπο
shift_staff	3.003	Αναθέσεις προσωπικού ανά βάρδια
patient_hospitalization_review	100	Αξιολογήσεις νοσηλείας από ασθενείς
patient_doctor_review	100	Αξιολογήσεις ιατρών από ασθενείς
image	0	Εικόνες (δεν φορτώθηκαν)
ΣΥΝΟΛΟ	~21.958	Συνολικές εγγραφές στη βάση δεδομένων

A.4.1 Παρατηρήσεις

Αναφορικά δεδομένα (Reference Data)

Οι 5.670 κωδικοί ICD-10 φορτώθηκαν από την επίσημη κατάταξη του Παγκόσμιου Οργανισμού Υγείας (ΠΟΥ). Οι 760 κωδικοί KEN προέρχονται από τον επίσημο κατάλογο Κλειστών Ενοποιημένων Νοσηλίων του Υπουργείου Υγείας. Τα 40 φάρμακα και οι 30 δραστικές ουσίες αποτελούν υποσύνολο του καταλόγου EMA Article 57 (εγκεκριμένα φάρμακα EOX). Οι 5.394 ιατρικές πράξεις αντλήθηκαν από τον επίσημο κατάλογο ιατρικών πράξεων. Οι χώρες (30 εγγραφές) αποτελούν επιλεγμένο υποσύνολο της λίστας ISO 3166-1 alpha-2 που καλύπτει τις συχνότερα εμφανιζόμενες υπηκοότητες στο ελληνικό νοσοκομειακό σύστημα.

Προσωπικό

Το νοσοκομείο απασχολεί 291 μέλη προσωπικού: 91 ιατρούς κατανεμημένους στις 4 βαθμίδες (Διευθυντής, Επιμελητής Α, Επιμελητής Β, Ειδικεύομενος) και 25 ειδικότητες, 150 νοσηλευτές στα 15 τμήματα, και 50 διοικητικούς σε 7 ρόλους. Κάθε τμήμα έχει έναν Διευθυντή βαθμίδας 'Διευθυντής' ορισμένο ως `director_id`. Οι 111 εγγραφές στον `doctor_department` αντικατοπτρίζουν το γεγονός ότι ορισμένοι ιατροί ανήκουν σε περισσότερα από ένα τμήματα (Μ:Ν σχέση).

Ασθενείς & Νοσηλείες

Οι 565 νοσηλείες αντιστοιχούν σε 310 ασθενείς, κατά μέσο όρο 1,8 νοσηλείες ανά ασθενή, με αρκετούς ασθενείς να έχουν πολλαπλές εισαγωγές, γεγονός ρεαλιστικό για νοσοκομειακό σύστημα. Οι 864 εγγραφές στον `triage` σημαίνουν ότι υπάρχουν και διαλογές που κατέληξαν σε 'αποχώρηση με οδηγίες' (χωρίς νοσηλεία). Οι 1.207 εγγραφές στον `hospitalization_staff` (κατά μέσο όρο ~2,1 ανά νοσηλεία) αντικατοπτρίζουν την ανάθεση θεράποντος ιατρού, συμβούλου και υπεύθυνου νοσηλευτή ανά νοσηλεία.

Βάρδιες

Οι 3.003 εγγραφές στον `shift_staff` αντικατοπτρίζουν τη λειτουργία 15 τμημάτων × 3 βαρδιές/ημέρα × τουλάχιστον 11 μέλη ανά βάρδια (3 ιατροί + 6 νοσηλευτές + 2 διοικητικοί), για χρονικό ορίζοντα

αρκετών μηνών. Οι 900 εγγραφές στον shifts αντιστοιχούν σε 300 ημέρες $\times$ 3 τύπους βάρδιας (Πρωινή/Απογευματινή/Νυχτερινή).

Εργαστηριακές Εξετάσεις & Ιατρικές Πράξεις

Οι 324 εργαστηριακές εξετάσεις (hospitalization_lab) και οι 334 ιατρικές πράξεις (hospitalization_procedure) κατανέμονται στις 565 νοσηλείες, δηλαδή κατά μέσο όρο περίπου 0,6 εξετάσεις και 0,6 πράξεις ανά νοσηλεία, ρεαλιστικό ποσοστό αφού δεν κάθε νοσηλεία συνοδεύεται από επέμβαση ή εξέταση.

Αξιολογήσεις

Οι 100 αξιολογήσεις νοσηλείας και οι 100 αξιολογήσεις ιατρών αντιστοιχούν σε ποσοστό ~18% των ολοκληρωμένων νοσηλείων, γεγονός που είναι ρεαλιστικό για εθελοντικό σύστημα αξιολόγησης.

A.4.2 Διαδικασία Παραγωγής Δεδομένων

Τα δεδομένα της βάσης παράχθηκαν συνθετικά (synthetic data generation) μέσω custom Python scripts, με στόχο να προσομοιώνουν ρεαλιστικό νοσοκομειακό περιβάλλον τηρώντας ταυτόχρονα πλήρη αναφορική ακεραιότητα.

Εργαλεία

Χρησιμοποιήθηκε η βιβλιοθήκη Faker (Python) για την παραγωγή ψευδο-τυχαίων ονομάτων, διευθύνσεων, AMKA, email και τηλεφώνων. Για τιμές που έπρεπε να ανήκουν σε συγκεκριμένο σύνολο (ειδικότητες, βαθμίδες, τμήματα, ασφαλιστικοί φορείς) χρησιμοποιήθηκε η συνάρτηση random.choice() της βιβλιοθήκης random. Για ημερομηνίες (εισαγωγής, εξόδου, βαρδιών, γέννησης) χρησιμοποιήθηκαν τυχαίες ημερομηνίες εντός λογικών εύρων με τη βιβλιοθήκη datetime.

Σειρά παραγωγής

Τα δεδομένα παράχθηκαν σε αυστηρά καθορισμένη σειρά που σέβεται τις εξαρτήσεις FK: πρώτα τα αναφορικά δεδομένα (country, insurer, ken_code, icd10_code, drug, active_substance), μετά τα τμήματα και το προσωπικό (με ειδική μέριμνα για την αυτοαναφορά supervisor_id και τη διαδοχή Διευθυντής $\rightarrow$ Επιμελητής A $\rightarrow$ Επιμελητής B $\rightarrow$ Ειδικευόμενος), έπειτα οι ασθενείς, οι διαλογές, οι νοσηλείες και τέλος οι συνταγές, εξετάσεις, βάρδιες και αξιολογήσεις.

Διαχείριση περιορισμών

Ιδιαίτερη προσοχή δόθηκε στην τήρηση των επιχειρησιακών κανόνων κατά την παραγωγή: η performed_date κάθε πράξης και η ordered_date κάθε εξέτασης παράχθηκαν πάντα εντός του χρονικού πλαισίου της αντίστοιχης νοσηλείας ($\text{admission_date} \leq \text{date} \leq \text{discharge_date}$). Τα start_date και end_date συνταγών παράχθηκαν επίσης εντός της νοσηλείας. Το discharge_date παράχθηκε πάντα μεταγενέστερα του admission_date. Τα supervisor_id των Ειδικευόμενων παραπέμπουν αποκλειστικά σε Επιμελητές A ή Διευθυντές.

Αλλεργίες & Συνταγές

Κατά την παραγωγή συνταγών, ελέγχθηκε ότι καμία δραστική ουσία του συνταγογραφούμενου φαρμάκου δεν εμφανίζεται στις αλλεργίες του ασθενή — αναπαράγοντας έτσι σε επίπεδο δεδομένων τη λογική του trigger trg_allergy_check. Αυτό εξασφάλισε ότι τα δεδομένα είναι συμβατά με τους κανόνες του συστήματος και δεν προκαλούν σφάλματα κατά τη φόρτωση.

Φόρτωση στη βάση

Τα Python scripts παρήγαγαν αρχεία SQL (INSERT statements) που φορτώθηκαν στη βάση PostgreSQL μέσω pgAdmin 4, ακολουθώντας την αυστηρή σειρά φόρτωσης `install.sql (schema)` → `reference data` → `departments` → `staff` → `patients` → `triage` → `hospitalizations` → `shifts` → `prescriptions` → `lab/procedures` → `reviews`.

A.4.3: Παραδοχές (Assumptions)

Παραδοχές για το Πρόγραμμα Βαρδιών

- Οι βάρδιες φορτώθηκαν για **7 ημέρες × 3 βάρδιες × 15 τμήματα = 315 βάρδιες**, με 11 άτομα ανά βάρδια (3 ιατροί + 6 νοσηλευτές + 2 διοικητικοί), σύμφωνα με την εκφώνηση.
- Χρησιμοποιήθηκε **κυκλική εναλλαγή (rotation)** των ίδιων υπαλλήλων ανά τμήμα ώστε να τηρούνται αυτόματα οι περιορισμοί: ≥ 8 ώρες ανάπαυσης μεταξύ διαδοχικών βαρδιών, ≤ 3 συνεχόμενες νυχτερινές βάρδιες, μέγιστα όρια βαρδιών/μήνα ανά κατηγορία (15/20/25).
- Σε κάθε βάρδια με ειδικευόμενο εξασφαλίζεται η παρουσία τουλάχιστον ενός Επιμελητή Α΄ ή Διευθυντή.

Παραδοχές για τη Φαρμακευτική Αγωγή

- Το **drug_substance** υλοποιεί τη M:N σχέση φαρμάκου–δραστικής ουσίας (ένα φάρμακο μπορεί να περιέχει πολλές ουσίες).
- Ο έλεγχος αλλεργιών γίνεται μέσω trigger **trg_allergy_check** πριν από κάθε **INSERT** στο **prescription**, που απαγορεύει συνταγογράφηση φαρμάκου του οποίου οποιαδήποτε ουσία βρίσκεται στις αλλεργίες του ασθενούς (**RAISE EXCEPTION**).
- Οι δραστικές ουσίες είναι **απλοποιημένες** (subset 29 ουσιών αντί όλου του EMA Article 57), όπως επιτρέπει η εκφώνηση.

Παραδοχές για τα Πολυμέσα (Εικόνες)

- Όλες οι κύριες οντότητες (**department**, **doctor**, **medical_procedure**, κλπ.) διαθέτουν προαιρετικά πεδία **image_url VARCHAR(1000)** και **image_description TEXT** για μελλοντική σύνδεση με την ιστοσελίδα του νοσοκομείου.
- Στα δεδομένα δοκιμής δεν φορτώθηκαν εικόνες — μόνο placeholder URLs όπου ήταν απαραίτητο.

A.5 Περιβάλλον Υλοποίησης

Η βάση δεδομένων υλοποιήθηκε σε PostgreSQL 15+. Η επιλογή της PostgreSQL έναντι άλλων RDBMS αιτιολογείται από τα εξής:

DEFERRABLE INITIALLY DEFERRED constraints

Απαραίτητα για τη διαχείριση της κυκλικής εξάρτησης `department` → `doctor` (ο διευθυντής πρέπει να είναι ιατρός, αλλά ο ιατρός ανήκει σε τμήμα). Η PostgreSQL υποστηρίζει πλήρως αναβαλλόμενα constraints.

CONSTRAINT TRIGGERS (DEFERRABLE)

Triggers όπως `trg_shift_min_staff` και `trg_shift_resident_needs_senior` ορίστηκαν ως DEFERRABLE INITIALLY DEFERRED, ώστε να εκτελούνται μετά την ολοκλήρωση όλων των εισαγωγών μιας βάρδιας, και όχι με κάθε μεμονωμένη εισαγωγή.

GENERATED ALWAYS AS IDENTITY

Σύγχρονο πρότυπο SQL για αυτόματη παραγωγή κλειδιού, χρησιμοποιείται στον πίνακα `duty_assignment` και `image` αντί για το παλαιότερο SERIAL.

Window Functions & CTEs

Χρησιμοποιούνται εκτεταμένα στα views (π.χ. `COUNT()` OVER PARTITION BY στο `v_available_beds`, CTEs στο `v_staff_availability`) για αποδοτικά ερωτήματα χωρίς υποερωτήματα.

Το schema αναπτύχθηκε στο αρχείο `install.sql` και τα δεδομένα φορτώθηκαν μέσω αρχείων SQL. Η εφαρμογή διαχείρισης υλοποιήθηκε σε Flask (Python) με REST API και HTML/JavaScript frontend.

Κεφάλαιο Β: Ανάλυση Κώδικα (install.sql)

Το κεφάλαιο αυτό αποτελεί την πλήρη τεκμηρίωση του αρχείου `install.sql`, τον κώδικα του `schema` της βάσης δεδομένων του Γενικού Νοσοκομείου «Υγειόπολης». Για κάθε πίνακα, `view` και `trigger` αναλύεται και αιτιολογείται κάθε σχεδιαστική απόφαση: η επιλογή των πεδίων, οι τύποι δεδομένων, τα `constraints`, οι σχέσεις μεταξύ πινάκων και η επιχειρησιακή λογική που τα υπαγορεύει. Στόχος είναι κάθε γραμμή κώδικα να συνοδεύεται από σαφή αιτιολόγηση που να απαντά στο ερώτημα «γιατί είναι έτσι» και όχι απλώς «τι κάνει».

Ενότητα 1: REFERENCE DATA

Η ενότητα αυτή περιέχει τους πίνακες αναφοράς του συστήματος. Αποτελείται από δύο πίνακες που αποθηκεύουν σταθερά δεδομένα τα οποία δεν αλλάζουν συχνά και χρησιμοποιούνται ως αναφορά από άλλες ενότητες του συστήματος.

Προδιαγραφές Απαιτήσεων Συστήματος

Το σύστημα χρησιμοποιεί δύο τυποποιημένες κωδικοποιήσεις: τους **κωδικούς KEN** (Κλειστά Ενοποιημένα Νοσήλια) για τον υπολογισμό κόστους νοσηλείας και τους **κωδικούς ICD-10** για την κωδικοποίηση διαγνώσεων εισαγωγής και εξόδου.

Πίνακας 1.1: ken_code

Σκοπός

Ο πίνακας **ken_code** αποθηκεύει τους κωδικούς KEN (Κλειστά Ενοποιημένα Νοσήλια), το σύστημα DRG που χρησιμοποιεί το νοσοκομείο για τον **υπολογισμό του κόστους νοσηλείας**. Κάθε κωδικός ορίζει ένα βασικό κόστος και μια μέση διάρκεια παραμονής. Αν η παραμονή υπερβεί τη μέση διάρκεια, εφαρμόζεται επιπλέον χρέωση ανά ημέρα.

Κώδικας SQL

```
CREATE TABLE ken_code (
    ken_code VARCHAR(10) NOT NULL,
    description VARCHAR(255) NOT NULL,
    base_cost DECIMAL(10, 2) NOT NULL CHECK (base_cost >= 0),
    avg_duration_days DECIMAL(5, 2) NOT NULL CHECK (avg_duration_days > 0),
    daily_extra DECIMAL(10, 2) NOT NULL DEFAULT 0 CHECK (daily_extra >= 0),
    CONSTRAINT pk_ken_code PRIMARY KEY (ken_code)
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
ken_code	VARCHAR (10)	NOT NULL	PK	Κωδικός KEN. Χρησιμοποιείται απευθείας ως PK γιατί είναι από μόνος του μοναδικός και σταθερός, δεν χρειάζεται surrogate key. VARCHAR γιατί μπορεί να περιέχει γράμματα και αριθμούς.
description	VARCHAR (255)	NOT NULL		Περιγραφή του κωδικού KEN. NOT NULL γιατί κάθε κωδικός πρέπει να έχει περιγραφή για αναγνώριση.
base_cost	DECIMAL (10,2)	NOT NULL	CHECK ≥ 0	Βασικό κόστος νοσηλείας. NOT NULL γιατί κάθε κωδικός KEN ορίζει υποχρεωτικά βασικό κόστος.

Στήλη	Τύπος		Constraint	Αιτιολόγηση
avg_duration_days	DECIMAL (5,2)	NOT NULL	CHECK > 0	Μέση διάρκεια παραμονής (Mean Duration of Stay). NOT NULL γιατί είναι απαραίτητη για τον υπολογισμό κόστους.
daily_extra	DECIMAL (10,2)	NOT NULL	DEFAULT 0, CHECK ≥ 0	Επιπλέον χρέωση ανά ημέρα όταν η παραμονή υπερβαίνει το avg_duration_days. DEFAULT 0 γιατί αν δεν υπάρχει επιπλέον χρέωση, η τιμή είναι 0, δεν χρειάζεται NULL.

Explanations & Constrains

ken_code

Ο κωδικός KEN χρησιμοποιείται απευθείας ως PK αντί για surrogate key (SERIAL) γιατί είναι εξωτερικά ορισμένος, μοναδικός και σταθερός.

daily_extra NOT NULL με DEFAULT 0

Το **daily_extra** είναι NOT NULL γιατί η τιμή 0 εκφράζει ξεκάθαρα ότι δεν υπάρχει επιπλέον χρέωση (δεν υπάρχει λόγος να είναι NULL). Με το **DEFAULT 0** καλύπτονται αυτόματα οι κωδικοί KEN που δεν έχουν επιπλέον χρέωση.

CHECK constraints

base_cost CHECK ≥ 0

Το βασικό κόστος δεν μπορεί να είναι αρνητικό. Μηδενικό κόστος επιτρέπεται για ειδικές περιπτώσεις.

avg_duration_days CHECK > 0

Η μέση διάρκεια παραμονής πρέπει να είναι αυστηρά θετική. Μηδενική διάρκεια δεν έχει νόημα επιχειρησιακά.

daily_extra CHECK ≥ 0

Η επιπλέον χρέωση δεν μπορεί να είναι αρνητική. Μηδενική τιμή εκφράζει απουσία επιπλέον χρέωσης

Σχετικά Triggers

Ο πίνακας **ken_code** δεν έχει δικά του triggers, είναι lookup table που χρησιμοποιείται παθητικά από άλλους πίνακες. Ωστόσο, συμμετέχει ενεργά στο trigger **trg_hosp_total_cost** του πίνακα **hospitalization**:

Ενότητα Triggers → trg_hosp_total_cost

Κατά τη συμπλήρωση του **discharge_date**, το trigger διαβάζει από τον **ken_code** τα πεδία **base_cost**, **avg_duration_days** και **daily_extra** για να υπολογίσει αυτόματα το **total_cost** της νοσηλείας. Αν η παραμονή υπερβαίνει το **avg_duration_days**, προστίθεται **daily_extra** για κάθε επιπλέον ημέρα.

Πίνακας 1.2: icd10_code

Σκοπός

Ο πίνακας **icd10_code** είναι **lookup table** που **αποθηκεύει τους κωδικούς ICD-10** (International Classification of Diseases, 10th Revision), το διεθνές σύστημα κωδικοποίησης νόσων που χρησιμοποιεί το νοσοκομείο για την καταγραφή διαγνώσεων εισαγωγής και εξόδου κάθε νοσηλείας.

Κώδικας SQL

```
CREATE TABLE icd10_code (
  icd10_code VARCHAR(10) NOT NULL,
  description VARCHAR(200) NOT NULL,
  category VARCHAR(60) NOT NULL,
  PRIMARY KEY (icd10_code)
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
icd10_code	VARCHAR(10)	NOT NULL	PK	Κωδικός ICD-10. Χρησιμοποιείται απευθείας ως PK γιατί είναι εξωτερικά ορισμένος, μοναδικός και σταθερός. VARCHAR γιατί περιέχει γράμματα και αριθμούς.
description	VARCHAR(200)	NOT NULL		Περιγραφή του κωδικού ICD-10. NOT NULL γιατί κάθε κωδικός πρέπει να έχει περιγραφή για αναγνώριση.
category	VARCHAR(60)	NOT NULL		Κατηγορία του κωδικού ICD-10. NOT NULL γιατί κάθε κωδικός ανήκει υποχρεωτικά σε μια κατηγορία.

Explanations

icd10_code

Ο **κωδικός ICD-10** χρησιμοποιείται απευθείας ως PK αντί για surrogate key (SERIAL) γιατί είναι εξωτερικά ορισμένος από τον Παγκόσμιο Οργανισμό Υγείας, **μοναδικός και σταθερός**.

Constraints

Ο πίνακας **icd10_code** δεν έχει constraints πέρα από το PK. Τα constraints ορίζονται στον **hospitalization** που τον αναφέρει μέσω δύο FK: ένα για τη διάγνωση εισαγωγής (**admission_diag_icd10**) και ένα για τη διάγνωση εξόδου (**discharge_diag_icd10**). Και οι δύο ορίζονται με **ON DELETE SET NULL** γιατί αν ένας κωδικός ICD-10 διαγραφεί ή αναθεωρηθεί διεθνώς, το ιστορικό της νοσηλείας δεν πρέπει να χαθεί, μόνο η σύνδεση με τον κωδικό γίνεται NULL.

Σχετικά Triggers

Ο πίνακας `icd10_code` δεν έχει δικά του triggers. Συμμετέχει ωστόσο στο trigger `trg_hosp_discharge_diagnosis` του πίνακα `hospitalization`, κατά τη συμπλήρωση του `discharge_date`, το trigger ελέγχει ότι έχει συμπληρωθεί έγκυρος κωδικός ICD-10 εξόδου (`discharge_diag_icd10`) που υπάρχει στον `icd10_code`. **Ενότητα Triggers** → `trg_hosp_discharge_diagnosis`.

Assumption

Το πεδίο `category` αποθηκεύεται ως ελεύθερο κείμενο αντί για FK σε ξεχωριστό πίνακα `icd10_category`. Οι κατηγορίες ICD-10 είναι εξωτερικά ορισμένες από τον ΠΟΥ και δεν αλλάζουν, η περαιτέρω κανονικοποίηση θα πρόσθετε πολυπλοκότητα χωρίς ουσιαστικό όφελος για τις ανάγκες του project.

Πίνακας 1.3: country

Σκοπός

Ο πίνακας `country` είναι **lookup table** που αποθηκεύει τις χώρες βάσει του διεθνούς προτύπου **ISO 3166-1 alpha-2**. Χρησιμοποιείται για την αποθήκευση της υπηκοότητας των ασθενών με τυποποιημένο και συνεπή τρόπο, αποτρέποντας ασυνέπειες ελεύθερου κειμένου (π.χ. "Ελληνική", "Greek", "GR"). Τα δεδομένα φορτώνονται από την επίσημη λίστα ISO στο `load.sql`.

Κώδικας SQL

```
CREATE TABLE country (
    country_code CHAR(2) NOT NULL,
    name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_country PRIMARY KEY (country_code)
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
<code>country_code</code>	CHAR(2)	NOT NULL	PK	Κωδικός χώρας ISO 3166-1 alpha-2 (π.χ. 'GR', 'DE', 'US'). CHAR(2) γιατί είναι πάντα ακριβώς 2 χαρακτήρες. Χρησιμοποιείται ως PK γιατί είναι εξωτερικά ορισμένος, μοναδικός και σταθερός.
<code>name</code>	VARCHAR(100)	NOT NULL		Πλήρες όνομα χώρας στα αγγλικά βάσει της επίσημης λίστας ISO 3166-1 (π.χ. 'Greece', 'Germany'). NOT NULL γιατί κάθε κωδικός πρέπει να έχει αντίστοιχο όνομα.

Explanations

country_code

Ο κωδικός ISO 3166-1 χρησιμοποιείται απευθείας ως PK γιατί είναι εξωτερικά ορισμένος, μοναδικός και σταθερός.

CHAR(2)

Ο κωδικός είναι πάντα ακριβώς 2 χαρακτήρες. Άρα CHAR(2) είναι πιο κατάλληλο από VARCHAR(2) γιατί δεν υπάρχει μεταβλητό μήκος.

Σχέση με τον patient

Ο πίνακας country χρησιμοποιείται από τον patient μέσω FK στο πεδίο nationality. Εξασφαλίζει ότι η υπηκοότητα κάθε ασθενή είναι έγκυρος κωδικός ISO, δεν μπορεί να καταχωρηθεί άγνωστη ή λανθασμένη χώρα.

Με τη χρήση του country_code ως FK αντί για ελεύθερο κείμενο, αποτρέπονται ασυνέπειες όπως "Ελληνική", "Greek", "ΕΛΛΗΝΙΚΗ" που θα έκαναν αδύνατο τον αξιόπιστο έλεγχο υπηκοότητας.

Πίνακας 1.4: insurer

Σκοπός

Ο πίνακας insurer αποθηκεύει τους ασφαλιστικούς φορείς που αναγνωρίζει το νοσοκομείο. Αντιμετωπίζεται ως ανεξάρτητη οντότητα, καταχωρείται μία φορά και συνδέεται με τους ασθενείς μέσω FK. Εξασφαλίζει συνέπεια και αποτρέπει ασυνέπειες ελεύθερου κειμένου που θα έκαναν αδύνατη την αξιόπιστη κατανομή νοσηλειών ανά ασφαλιστικό φορέα. Τα δεδομένα φορτώνονται από το load.sql.

Κώδικας SQL

```
CREATE TABLE insurer (  
    insurer_id SERIAL NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    type VARCHAR(30) NOT NULL CHECK (  
        type IN (  
            'δημόσια',  
            'στρατιωτική',  
            'επαγγελματική',  
            'ιδιωτική',  
            'ΕΟΧ',  
            'διμερής',  
            'διεθνής',  
            'ανασφάλιστος'  
        )  
    )
```

```
),
CONSTRAINT pk_insurer PRIMARY KEY (insurer_id),
CONSTRAINT uq_insurer_name UNIQUE (name)
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
insurer_id	SERIAL	NOT NULL	PK	ID κάθε ασφαλιστικού φορέα. SERIAL γιατί ο πίνακας περιέχει προκαθορισμένες εγγραφές που φορτώνονται από το <code>load.sql</code> .
name	VARCHAR(100)	NOT NULL	UNIQUE	Όνομα ασφαλιστικού φορέα. NOT NULL γιατί κάθε εγγραφή πρέπει να έχει όνομα. UNIQUE γιατί δεν μπορούν να υπάρχουν δύο φορείς με ίδιο όνομα.
type	VARCHAR(30)	NOT NULL	CHECK	Κατηγορία ασφαλιστικού φορέα. NOT NULL γιατί κάθε φορέας ανήκει υποχρεωτικά σε μια κατηγορία. CHECK περιορίζει τις αποδεκτές τιμές.

Explanations

Οι αποδεκτές κατηγορίες είναι:

- **'δημόσια'** — δημόσιοι φορείς (e-ΕΦΚΑ, ΕΟΠΥΥ)
- **'στρατιωτική'** — ταμεία υγείας ενόπλων δυνάμεων και σωμάτων ασφαλείας
- **'επαγγελματική'** — επαγγελματικά ταμεία συγκεκριμένων κλάδων (ΕΔΟΕΑΠ, ΤΥΠΕΤ)
- **'ιδιωτική'** — ιδιωτικές ασφαλιστικές εταιρείες
- **'ΕΟΧ'** — Ευρωπαϊκή Κάρτα Ασφάλισης Ασθένειας για αλλοδαπούς ΕΟΧ
- **'διμερής'** — χώρες με διμερείς συμφωνίες κοινωνικής ασφάλισης με την Ελλάδα
- **'διεθνής'** — διεθνείς ιδιωτικές ασφαλιστικές εταιρείες για αλλοδαπούς εκτός ΕΟΧ
- **'ανασφάλιστος'** — ασθενής χωρίς ασφαλιστική κάλυψη

'ανασφάλιστος'

Ο ανασφάλιστος ασθενής αντιμετωπίζεται ως ξεχωριστή εγγραφή στον **insurer** αντί για NULL στον **patient**. Εξασφαλίζει ότι η ασφαλιστική κατάσταση κάθε ασθενή είναι πάντα καταγεγραμμένη και αναγνωρίσιμη, υποχρεωτική πληροφορία για τη χρέωση και τη στατιστική ανάλυση.

Σχέση με τον patient

Ο πίνακας **insurer** χρησιμοποιείται από τον **patient** μέσω FK στο πεδίο **insurer_id**. Εξασφαλίζει ότι ο ασφαλιστικός φορέας κάθε ασθενή είναι έγκυρος και αναγνωρισμένος από το νοσοκομείο. Η χρήση FK αντί για ελεύθερο κείμενο εξασφαλίζει αξιόπιστο **GROUP BY** για την κατανομή νοσηλείων ανά ασφαλιστικό φορέα.

Πίνακας 1.5: active_substance

Σκοπός

Ο πίνακας **active_substance** είναι **lookup table** που αποθηκεύει τις δραστικές ουσίες των φαρμάκων. Χρησιμοποιείται από δύο ενότητες: από την ενότητα **PATIENTS** μέσω του **patient_allergy** για την καταγραφή αλλεργιών ασθενών, και από την ενότητα **PHARMACY** μέσω του **drug_substance** για τη σύνδεση φαρμάκων με τις δραστικές ουσίες που περιέχουν. Τα δεδομένα φορτώνονται από το αρχείο **EMA Article 57** στο load.sql.

Κώδικας SQL

```
CREATE TABLE active_substance (
    name VARCHAR(200) NOT NULL,
    CONSTRAINT pk_active_substance PRIMARY KEY (name)
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
name	VARCHAR(200)	NOT NULL	PK	Όνομα δραστικής ουσίας (INN — International Nonproprietary Name). Χρησιμοποιείται απευθείας ως PK γιατί είναι εξωτερικά ορισμένο από τον ΠΟΥ, μοναδικό και σταθερό. VARCHAR(200) γιατί ορισμένα INN ονόματα είναι αρκετά μακριά.

Explanations

name

Το INN όνομα χρησιμοποιείται απευθείας ως PK, γιατί είναι εξωτερικά ορισμένο από τον ΠΟΥ, μοναδικό και σταθερό. Δεν χρειάζεται surrogate key.

Πηγή δεδομένων

Τα ονόματα των δραστικών ουσιών προέρχονται από το αρχείο EMA Article 57 που περιλαμβάνει όλα τα εγκεκριμένα φάρμακα στον ΕΟΧ. Στο αρχείο, πολλαπλές δραστικές ουσίες ανά φάρμακο διαχωρίζονται με |, κάθε ουσία εισάγεται ως ξεχωριστή εγγραφή στον **active_substance**.

Σχέση με patient_allergy και drug_substance

Ο πίνακας χρησιμοποιείται από δύο FKs:

- **patient_allergy** → το πεδίο **active_substance** δείχνει στο **name** του **active_substance**, καταγράφει αλλεργία ασθενή σε συγκεκριμένη ουσία.
- **drug_substance** → το πεδίο **active_substance** δείχνει στο **name** του **active_substance**, συνδέει κάθε φάρμακο με τις δραστικές ουσίες που περιέχει.

Και οι δύο ορίζονται με **ON DELETE RESTRICT**, αν προσπαθήσεις να διαγράψεις μια δραστική ουσία ενώ υπάρχουν αλλεργίες ή φάρμακα που την αναφέρουν, η διαγραφή απαγορεύεται.

Σχετικά Triggers

Ο πίνακας **active_substance** δεν έχει δικά του triggers. Συμμετέχει ωστόσο στο trigger **trg_allergy_check** — κατά τη συνταγογράφηση, ελέγχεται αν κάποια δραστική ουσία του φαρμάκου υπάρχει στις αλλεργίες του ασθενή. **Ενότητα Triggers** → **trg_allergy_check**.

Πίνακας 1.6: drug

Σκοπός

Ο πίνακας **drug** είναι **lookup table** που αποθηκεύει τα εγκεκριμένα φάρμακα βάσει της βάσης δεδομένων EMA Article 57, τον επίσημο κατάλογο όλων των φαρμάκων που έχουν εγκριθεί στον Ευρωπαϊκό Οικονομικό Χώρο (EOX). Το νοσοκομείο υποχρεούται να συνταγογραφεί αποκλειστικά φάρμακα από αυτόν τον κατάλογο. Τα δεδομένα φορτώνονται από το αρχείο EMA Article 57 στο load.sql.

Κώδικας SQL

```
CREATE TABLE drug (
  drug_code VARCHAR(30) NOT NULL,
  name VARCHAR(200) NOT NULL,
  image_url VARCHAR(255),
  CONSTRAINT pk_drug PRIMARY KEY (drug_code)
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
drug_code	VARCHAR(30)	NOT NULL	PK	Κωδικός φαρμάκου από το EMA Article 57. Χρησιμοποιείται ως φυσικό PK γιατί είναι εξωτερικά ορισμένος, μοναδικός και σταθερός. VARCHAR(30) γιατί οι κωδικοί EMA μπορεί να περιέχουν γράμματα και αριθμούς.
name	VARCHAR(200)	NOT NULL	UNIQUE	Πλήρες όνομα φαρμάκου (εμπορικό ή γενόσημο). NOT NULL γιατί κάθε φάρμακο πρέπει να έχει όνομα. VARCHAR(200) γιατί τα ονόματα φαρμάκων μπορεί να είναι αρκετά μακριά.
image_url	VARCHAR(255)	NULL	CHECK	URL φωτογραφίας φαρμάκου για την ιστοσελίδα του νοσοκομείου. Προαιρετικό.

Explanations

drug_code

Ο κωδικός EMA χρησιμοποιείται απευθείας ως PK, γιατί είναι εξωτερικά ορισμένος, μοναδικός και σταθερός.

Πηγή δεδομένων

Τα δεδομένα προέρχονται από το αρχείο EMA Article 57 που περιλαμβάνει όλα τα εγκεκριμένα φάρμακα στον ΕΟΧ.

Σχέση με drug_substance και prescription

Ο πίνακας χρησιμοποιείται από δύο FKs:

- **drug_substance** → συνδέει κάθε φάρμακο με τις δραστικές ουσίες που περιέχει
- **prescription** → καταγράφει τη συνταγογράφηση συγκεκριμένου φαρμάκου σε ασθενή

Σχετικά Triggers

Ο πίνακας **drug** δεν έχει δικά του triggers. Συμμετέχει στο trigger **trg_allergy_check**, κατά τη συνταγογράφηση, ελέγχονται οι δραστικές ουσίες του φαρμάκου μέσω του **drug_substance** έναντι των αλλεργιών του ασθενή. Βλέπε **Ενότητα Triggers** → **trg_allergy_check**.

Πίνακας 1.7: drug_substance

Σκοπός

Ο πίνακας **drug_substance** είναι το **junction table** που υλοποιεί τη σχέση **many-to-many** μεταξύ **φαρμάκων και δραστικών ουσιών**. Ένα φάρμακο μπορεί να περιέχει πολλές δραστικές ουσίες και μια δραστική ουσία μπορεί να βρίσκεται σε πολλά φάρμακα. Τα δεδομένα φορτώνονται από το αρχείο EMA Article 57 στο load.sql, στο αρχείο, πολλαπλές δραστικές ουσίες ανά φάρμακο διαχωρίζονται με | και κάθε ουσία εισάγεται ως ξεχωριστή εγγραφή.

Κώδικας SQL

```
CREATE TABLE drug_substance (  
    drug_code VARCHAR(30) NOT NULL,  
    active_substance_name VARCHAR(200) NOT NULL,  
    CONSTRAINT pk_drug_substance PRIMARY KEY (drug_code,  
active_substance_name),  
    CONSTRAINT fk_ds_drug FOREIGN KEY (drug_code) REFERENCES drug (drug_code)  
ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT fk_ds_substance FOREIGN KEY (active_substance_name) REFERENCES  
active_substance (name) ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
drug_code	VARCHAR(30)	NOT NULL	PK, FK → drug	Σύνδεση με το φάρμακο. NOT NULL γιατί κάθε εγγραφή πρέπει να αντιστοιχεί σε γνωστό φάρμακο.
active_substance_name	VARCHAR(200)	NOT NULL	PK, FK → active_substance	Σύνδεση με τη δραστική ουσία. NOT NULL γιατί κάθε εγγραφή πρέπει να αντιστοιχεί σε γνωστή ουσία.

Explanations

CONSTRAINT pk_drug_substance PRIMARY KEY (drug_code, active_substance_name)

Σύνθετο πρωτεύον κλειδί. Εξασφαλίζει ότι ο ίδιος συνδυασμός φαρμάκου-ουσίας δεν μπορεί να καταχωρηθεί δύο φορές. Δεν απαιτείται surrogate key γιατί ο συνδυασμός είναι από μόνος του μοναδικός και σημαντικός.

CONSTRAINT fk_ds_drug ... ON DELETE CASCADE

Αν διαγραφεί ένα φάρμακο από τον **drug**, διαγράφονται αυτόματα και όλες οι συνδέσεις του με δραστικές ουσίες. Η σύνδεση δεν έχει νόημα χωρίς το φάρμακο.

CONSTRAINT fk_ds_substance ... ON DELETE RESTRICT

Αν προσπαθήσεις να διαγράψεις μια δραστική ουσία ενώ κάποιο φάρμακο την περιέχει, η διαγραφή απαγορεύεται. Δεν μπορεί να υπάρξει φάρμακο που αναφέρεται σε άγνωστη ουσία.

Σχετικά Triggers

Ο πίνακας **drug_substance** δεν έχει δικά του triggers. Συμμετέχει στο trigger **trg_allergy_check** του πίνακα **prescription** ως πηγή δεδομένων, κατά τη συνταγογράφηση, το trigger διαβάζει από τον **drug_substance** τις δραστικές ουσίες του φαρμάκου και τις συγκρίνει με τις αλλεργίες του ασθενή στον **patient_allergy**. Αν υπάρχει κοινή ουσία, η συνταγογράφηση απαγορεύεται αυτόματα.

Ενότητα Triggers → **trg_allergy_check**

Ενότητα 2: DEPARTMENTS & BEDS

Προδιαγραφές Απαιτήσεων Συστήματος

Το νοσοκομείο διαθέτει τμήματα (π.χ. Καρδιολογία, Χειρουργική, ΜΕΘ, Επείγοντα). Κάθε τμήμα έχει όνομα, περιγραφή, αριθμό κλινών, όροφο/κτίριο και έναν διευθυντή τμήματος (ιατρό). Κάθε τμήμα μπορεί να εξυπηρετεί πολλές ειδικότητες, και κάθε τμήμα διαθέτει κλίνες με μοναδικό αριθμό, τύπο (π.χ. ΜΕΘ, μονόκλινο, πολύκλινο) και κατάσταση (π.χ. διαθέσιμη, κατειλημμένη, υπό συντήρηση).

Πίνακας 2.1: department

Σκοπός

Ο πίνακας **department** αποθηκεύει τα τμήματα του νοσοκομείου. Κάθε τμήμα έχει μοναδικό όνομα, το οποίο χρησιμοποιείται ως φυσικό PK και διαδίδεται ως FK σε όλους τους πίνακες που αναφέρονται σε τμήμα (nurse, admin_staff, shifts, hospitalization, triage κ.ά.). Ο διευθυντής κάθε τμήματος είναι ιατρός του νοσοκομείου και αποθηκεύεται ως αυτοαναφορικό FK προς τον doctor.

Κώδικας SQL

```
CREATE TABLE department (  
    department_name VARCHAR(100) NOT NULL,  
    department_description TEXT NOT NULL,  
    bed_capacity INT NOT NULL CHECK (bed_capacity >= 0),  
    floor_building VARCHAR(50) NOT NULL,  
    director_id INT, -- FK προστίθεται με ALTER TABLE παρακάτω  
    CONSTRAINT pk_department PRIMARY KEY (department_name),  
    CONSTRAINT uq_dept_director UNIQUE (director_id)  
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
department_name	VARCHAR(100)		NOT NULL PK	Φυσικό PK. Το όνομα τμήματος είναι εξ ορισμού μοναδικό και σταθερό, δεν χρειάζεται surrogate key. Διαδίδεται ως FK σε 8+ πίνακες, οπότε είναι αναγνώσιμο και ελέγχσιμο.
description	VARCHAR(255)		NULL	Προαιρετική περιγραφή τμήματος. NULL γιατί δεν είναι απαραίτητο για τη λειτουργία.

Στήλη	Τύπος		Constraint	Αιτιολόγηση
num_beds	INT		NOT NULL CHECK > 0	Αριθμός κλινών του τμήματος. NOT NULL γιατί κάθε τμήμα έχει υποχρεωτικά κλίνες. CHECK > 0 γιατί τμήμα με μηδενικές κλίνες δεν έχει νόημα.
floor_building	VARCHAR (50)		NULL	Όροφος/κτίριο. NULL γιατί είναι προαιρετικό.
director_id	INT		NULL FK → doctor	Ο διευθυντής του τμήματος. NULL αρχικά λόγω κυκλικής εξάρτησης (οι ιατροί δεν υπάρχουν ακόμα κατά τη δημιουργία τμημάτων). ON DELETE SET NULL: αν φύγει ο διευθυντής, το τμήμα συνεχίζει να υπάρχει.

Explanations & Constraints

department_name

Αντί για surrogate key (**department_id SERIAL**), το όνομα τμήματος χρησιμοποιείται απευθείας ως PK. Αυτό κάνει τα queries και τα FK στους εξαρτημένους πίνακες άμεσα αναγνώσιμα χωρίς JOIN. Το κόστος είναι μεγαλύτερος χώρος αποθήκευσης, αλλά για 15–20 τμήματα αυτό είναι αμελητέο.

DEFERRABLE INITIALLY DEFERRED

Υπάρχει κυκλική εξάρτηση: **department** → **doctor** → **staff** → (χρειάζεται να υπάρχει το **department** για ανάθεση τμήματος). Για να σπάσει ο κύκλος κατά τη φόρτωση, τα τμήματα εισάγονται αρχικά με **director_id = NULL** και ενημερώνονται με **UPDATE** αφού εισαχθούν οι ιατροί. Το **DEFERRABLE INITIALLY DEFERRED** επιτρέπει αυτή τη ροή.

ON DELETE SET NULL

Αν ο διευθυντής-ιατρός διαγραφεί ή αποχωρήσει, το τμήμα δεν πρέπει να διαγραφεί — συνεχίζει να λειτουργεί με κενή θέση διευθυντή.

Σχετικά Triggers

Ο πίνακας **department** δεν έχει δικά του triggers. Συμμετέχει ωστόσο σε πολλά triggers ως πηγή δεδομένων (π.χ. έλεγχος τμήματος κατά δημιουργία βάρδιας, ανάθεση κλίνης στη νοσηλεία).

Πίνακας 2.2: department_specialty

Σκοπός

Ο πίνακας **department_specialty** είναι το junction table που υλοποιεί τη σχέση many-to-many μεταξύ τμημάτων και ιατρικών ειδικοτήτων. Ένα τμήμα μπορεί να εξυπηρετεί πολλές ειδικότητες (π.χ. το Χειρουργικό τμήμα μπορεί να καλύπτει και Ορθοπεδική) και μια ειδικότητα μπορεί να εξυπηρετείται από πολλά τμήματα.

Κώδικας SQL

```
CREATE TABLE department_beds (
    bed_id SERIAL NOT NULL,
    type VARCHAR(30) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'διαθέσιμη',
    department_name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_bed PRIMARY KEY (bed_id),
    CONSTRAINT uq_bed_dept UNIQUE (bed_id, department_name),
    CONSTRAINT fk_bed_dept FOREIGN KEY (department_name) REFERENCES department
(department_name) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_bed_type CHECK (type IN ('ΜΕΘ', 'μονόκλινο', 'πολύκλινο',
'δίκλινο', 'απομόνωσης', 'ημερήσιας νοσηλείας', 'MENN')),
    CONSTRAINT chk_bed_status CHECK (status IN ('διαθέσιμη', 'κατειλημμένη', 'υπό
συντήρηση'))
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
department_name	VARCHAR(100)	NOT NULL	FK → department	Σύνδεση με το τμήμα. ON DELETE CASCADE: αν διαγραφεί τμήμα, αφαιρούνται αυτόματα οι αντιστοιχίσεις ειδικοτήτων.
specialty_name	VARCHAR(100)	NOT NULL	FK → doctor_specialty	Σύνδεση με την ειδικότητα. ON DELETE CASCADE: αν διαγραφεί ειδικότητα, αφαιρείται η αντιστοίχιση.

Explanations

Σύνθετο φυσικό PK (**department_name**, **specialty_name**): εξασφαλίζει ότι ο ίδιος συνδυασμός δεν μπορεί να καταχωρηθεί δύο φορές. Δεν απαιτείται surrogate key.

ON DELETE CASCADE

Η αντιστοίχιση δεν έχει νόημα αν ένα από τα δύο διαγραφεί.

Πίνακας 2.3: department_beds

Σκοπός

Ο πίνακας **department_beds** αποθηκεύει τις κλίνες κάθε τμήματος. Κάθε κλίνη έχει τύπο (ΜΕΘ, μονόκλινο, δίκλινο, πολύκλινο) και κατάσταση (διαθέσιμη, κατειλημμένη, υπό συντήρηση). Το PK είναι σύνθετο (**bed_id**, **department_name**) γιατί ο αριθμός κλίνης είναι μοναδικός μόνο εντός τμήματος. Ο πίνακας αυτός χρησιμοποιείται άμεσα από τον **hospitalization** για την ανάθεση κλίνης σε κάθε νοσηλεία.

Κώδικας SQL

```
CREATE TABLE department_beds (
    bed_id SERIAL NOT NULL,
    type VARCHAR(30) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'διαθέσιμη',
    department_name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_bed PRIMARY KEY (bed_id),
    CONSTRAINT uq_bed_dept UNIQUE (bed_id, department_name),
    CONSTRAINT fk_bed_dept FOREIGN KEY (department_name) REFERENCES department
(department_name) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_bed_type CHECK (type IN ('ΜΕΘ', 'μονόκλινο', 'πολύκλινο',
'dίκλινο', 'απομόνωσης', 'ημερήσιας νοσηλείας', 'ΜΕΝΝ')),
    CONSTRAINT chk_bed_status CHECK (status IN ('διαθέσιμη', 'κατειλημμένη', 'υπό
συντήρηση'))
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
bed_id	INT	NOT NULL	PK	Αριθμός κλίνης εντός τμήματος. Δεν είναι SERIAL γιατί οι κλίνες αριθμούνται ανά τμήμα (π.χ. κλίνη 1 υπάρχει σε κάθε τμήμα).
department_name	VARCHAR (100)	NOT NULL	PK, FK → department	Σύνδεση με το τμήμα. Μέρος του PK γιατί ο bed_id είναι μοναδικός μόνο εντός τμήματος. ON DELETE RESTRICT: δεν μπορεί να διαγραφεί τμήμα με κλίνες.
type	VARCHAR (20)	NOT NULL	CHECK	Τύπος κλίνης. CHECK περιορίζει σε: 'ΜΕΘ', 'μονόκλινο', 'δίκλινο', 'πολύκλινο'. NOT NULL γιατί κάθε κλίνη έχει υποχρεωτικά τύπο.

Στήλη	Τύπος		Constraint	Αιτιολόγηση
status	VARCHAR (20)	NOT NULL	DEFAULT 'διαθέσιμη' CHECK 'διαθέσιμη'	Κατάσταση κλίνης. DEFAULT 'διαθέσιμη' γιατί κάθε νέα κλίνη είναι αρχικά διαθέσιμη. CHECK περιορίζει σε: 'διαθέσιμη', 'κατειλημμένη', 'υπό συντήρηση'.

Explanations & Constraints

PK (bed_id, department_name)

Ο αριθμός κλίνης δεν είναι globally μοναδικός — κλίνη 1 υπάρχει στην Καρδιολογία και στη Χειρουργική. Ο συνδυασμός (bed_id, department_name) είναι μοναδικός.

Αυτό σημαίνει ότι και το FK στον **hospitalization** είναι σύνθετο:

FOREIGN KEY (bed_id, department_name) REFERENCES department_beds (bed_id, department_name).

status DEFAULT 'διαθέσιμη'

Κάθε νέα κλίνη καταχωρείται αυτόματα ως διαθέσιμη. Η κατάσταση αλλάζει αυτόματα σε 'κατειλημμένη' κατά τη δημιουργία νοσηλείας και επιστρέφει σε 'διαθέσιμη' κατά την έξοδο, μέσω trigger.

ON DELETE RESTRICT

Αν ένα τμήμα προσπαθήσει να διαγραφεί ενώ έχει κλίνες, η διαγραφή απαγορεύεται. Το ιστορικό κλινών δεν πρέπει να χαθεί.

Σχετικά Triggers

Ο πίνακας **department_beds** δεν έχει δικά του triggers. Συμμετέχει στο trigger **trg_hosp_bed_status**: κατά την εισαγωγή νοσηλείας, ελέγχεται ότι η κλίνη έχει status 'διαθέσιμη' και αλλάζει αυτόματα σε 'κατειλημμένη'. Κατά τη συμπλήρωση του **discharge_date**, το status επιστρέφει αυτόματα σε 'διαθέσιμη'. Βλέπε Ενότητα Triggers → **trg_hosp_bed_status**.

Ενότητα 3: STAFF

Η ενότητα αυτή διαχειρίζεται όλο το προσωπικό του νοσοκομείου. Υλοποιεί ιεραρχία ISA (supertype / subtype) με τον πίνακα **staff** ως κεντρική οντότητα και τους πίνακες **doctor**, **nurse** και **admin_staff** ως εξειδικευμένους υποτύπους. Κάθε μέλος προσωπικού ανήκει υποχρεωτικά σε ακριβώς έναν από τους τρεις υποτύπους.

Προδιαγραφές Απαιτήσεων Συστήματος

Το νοσοκομείο απασχολεί τρεις κατηγορίες προσωπικού: **ιατροί**, **νοσηλευτές** και **διοικητικό προσωπικό**. Κάθε γιατρός έχει αριθμό άδειας ασκήσεως επαγγέλματος, ειδικότητα και βαθμίδα (Ειδικευόμενος, Επιμελητής Β, Επιμελητής Α, Διευθυντής). Οι Ειδικευόμενοι υποχρεωτικά έχουν επόπτη, ενώ οι Διευθυντές δεν μπορούν να έχουν. Κάθε ιατρός μπορεί να ανήκει σε ένα ή περισσότερα τμήματα. Κάθε νοσηλευτής έχει βαθμίδα και ανήκει σε ένα τμήμα. Το σύστημα επιβάλλει μέγιστα όρια βαρδιών ανά μήνα: Ιατροί 15, Νοσηλευτές 20, Διοικητικό Προσωπικό 25.

Πίνακας 3.1: staff

Σκοπός

Ο πίνακας **staff** λειτουργεί ως supertype στην ISA ιεραρχία του προσωπικού. Αποθηκεύει τα κοινά στοιχεία όλων των κατηγοριών προσωπικού, ταυτοποίηση, ονοματεπώνυμο, στοιχεία επικοινωνίας, ημερομηνία πρόσληψης και τύπο. Κάθε εγγραφή στον staff αντιστοιχεί υποχρεωτικά σε μία εγγραφή σε έναν από τους τρεις υποτύπους (doctor, nurse, admin_staff) μέσω κοινού staff_id. Η επιλογή ISA αντί για ξεχωριστούς ανεξάρτητους πίνακες εξαλείφει την επανάληψη κοινών πεδίων και επιτρέπει ενιαία αναφορά σε οποιοδήποτε μέλος προσωπικού (π.χ. στα shifts, surgery_participant).

Κώδικας SQL

```
CREATE TABLE staff (
    staff_id SERIAL NOT NULL,
    amka CHAR(11) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    hire_date DATE NOT NULL,
    staff_type VARCHAR(20) NOT NULL,
    CONSTRAINT pk_staff PRIMARY KEY (staff_id),
    CONSTRAINT uq_staff_amka UNIQUE (amka),
```

```

CONSTRAINT uq_staff_email UNIQUE (email),
CONSTRAINT uq_staff_phone UNIQUE (phone),
CONSTRAINT chk_staff_dob_min CHECK (date_of_birth >= CURRENT_DATE -
INTERVAL '105 years'),
CONSTRAINT chk_staff_dob_max CHECK (date_of_birth <= (CURRENT_DATE -
INTERVAL '18 years')),
CONSTRAINT chk_hire_not_future CHECK (hire_date <= CURRENT_DATE),
CONSTRAINT chk_hire_min CHECK (hire_date >=
CURRENT_SETTING('app.hospital_founded')::DATE),
CONSTRAINT chk_hire_after_dob CHECK (hire_date >= (date_of_birth + INTERVAL
'18 years')),
CONSTRAINT chk_staff_type CHECK (staff_type IN ('Ιατρός', 'Νοσηλεύτης',
'Διοικητικό Προσωπικό'))
);

```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
staff_id	SERIAL	NOT NULL	PK	ID μέλους προσωπικού. SERIAL για αυτόματη αύξηση. Χρησιμοποιείται ως FK σε όλους τους υποτύπους (doctor, nurse, admin_staff).
amka	CHAR(11)	NOT NULL	UNIQUE	ΑΜΚΑ μέλους. CHAR(11) γιατί είναι πάντα ακριβώς 11 ψηφία. UNIQUE γιατί κανένα δύο μέλη δεν μπορούν να έχουν ίδιο ΑΜΚΑ.
first_name	VARCHAR(50)	NOT NULL		Όνομα. NOT NULL γιατί κάθε μέλος πρέπει να έχει όνομα κατά την πρόσληψη.
last_name	VARCHAR(50)	NOT NULL		Επίθετο. NOT NULL για τον ίδιο λόγο.
date_of_birth	DATE	NOT NULL	CHECK	Ημερομηνία γέννησης. NOT NULL γιατί χρησιμοποιείται για επαλήθευση ηλικίας. CHECK εξασφαλίζει ότι το μέλος είναι τουλάχιστον 18 ετών και το πολύ 105 ετών (CURRENT_DATE).
email	VARCHAR(100)	NOT NULL	UNIQUE	Email επικοινωνίας. UNIQUE γιατί κάθε μέλος έχει το δικό του εταιρικό email.
phone	VARCHAR(20)	NOT NULL	UNIQUE	Τηλέφωνο. VARCHAR γιατί μπορεί να περιέχει +, κενά, παρενθέσεις. UNIQUE γιατί κάθε μέλος έχει μοναδικό αριθμό.
hire_date	DATE	NOT NULL	CHECK	Ημερομηνία πρόσληψης. CHECK εξασφαλίζει ότι δεν είναι στο μέλλον και ότι το μέλος ήταν τουλάχιστον 18 ετών κατά την πρόσληψη.
staff_type	VARCHAR(20)	NOT NULL	CHECK	Discriminator ISA ιεραρχίας. Καθορίζει σε ποιον υπότυπο ανήκει το μέλος. CHECK περιορίζει σε: 'Ιατρός', 'Νοσηλεύτης', 'Διοικητικό Προσωπικό'.

ISA ιεραρχία (γιατί ένας κοινός supertype)

Ιατροί, νοσηλευτές και διοικητικό προσωπικό μοιράζονται τα ίδια βασικά στοιχεία (ταυτοποίηση, επικοινωνία, πρόσληψη). Αντί να επαναλαμβάνονται αυτά τα πεδία σε τρεις ανεξάρτητους πίνακες, τοποθετούνται μία φορά στον `staff`. Κάθε υπότυπος αποθηκεύει μόνο τα επιπλέον πεδία που τον αφορούν. Επιπλέον, πίνακες όπως `shifts`, `surgery_participant` και `hospitalization_staff` μπορούν να αναφέρονται σε οποιοδήποτε μέλος με ένα μόνο `FK → staff_id`, χωρίς να χρειάζεται να γνωρίζουν τον υπότυπο.

CONSTRAINT `chk_staff_dob_min / chk_staff_dob_max`

Εξασφαλίζει ότι το μέλος δεν μπορεί να είναι νεότερο των 18 ετών (κατώτατο νόμιμο όριο εργασίας) και όχι μεγαλύτερο των 105 ετών (λογικό ανώτατο όριο για ενεργό εγγραφή). Και τα δύο χρησιμοποιούν `CURRENT_DATE` για αυτόματη ενημέρωση.

CONSTRAINT `chk_hire_after_dob`

Εξασφαλίζει ότι το μέλος ήταν τουλάχιστον 18 ετών κατά την πρόσληψη. Αποτρέπει εσφαλμένες καταχωρήσεις όπου η `hire_date` είναι πριν από την `date_of_birth + 18` χρόνια.

CONSTRAINT `chk_staff_type`

Ο discriminator `staff_type` καθορίζει τον υπότυπο ISA. CHECK περιορίζει τις αποδεκτές τιμές ακριβώς στους τρεις υποτύπους που υπάρχουν στη βάση. Αποτρέπεται η εισαγωγή μέλους χωρίς αντίστοιχο subtype.

Σχετικά Triggers

Ενότητα Triggers → `trg_staff_subtype_check`

Ελέγχει ότι κάθε εγγραφή στον `staff` έχει αντίστοιχη εγγραφή στον σωστό υπότυπο (`doctor`, `nurse` ή `admin_staff`) βάσει του `staff_type`. Αποτρέπει ορφανές εγγραφές στον supertype χωρίς υπότυπο.

Πίνακας 3.2: `doctor_specialty`

Σκοπός

Ο πίνακας `doctor_specialty` είναι **lookup table** που αποθηκεύει τις αναγνωρισμένες ιατρικές ειδικότητες του νοσοκομείου. Χρησιμοποιείται ως FK στον `doctor` για να εξασφαλιστεί ότι κάθε ιατρός έχει έγκυρη ειδικότητα και αποτρέπονται ασυνέπειες ελεύθερου κειμένου.

Κώδικας SQL

```
CREATE TABLE doctor_specialty (name VARCHAR(100) PRIMARY KEY);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
name	VARCHAR(100)	NOT NULL	PK	Όνομα ειδικότητας. Φυσικό PK, μοναδικό και σταθερό. Δεν χρειάζεται surrogate key. Επιτρέπει αναγνώσιμο FK στον doctor.

Explanations

Φυσικό PK αντί για SERIAL

Το όνομα της ειδικότητας χρησιμοποιείται απευθείας ως PK γιατί είναι το ίδιο το αναγνωριστικό — δεν χρειάζεται surrogate key. Αυτό επιτρέπει στον doctor να αναφέρεται στην ειδικότητα με ανθρώπινα αναγνώσιμο FK αντί για αριθμό.

Πίνακας 3.3: doctor

Σκοπός

Ο πίνακας **doctor** είναι ο subtype της ISA ιεραρχίας για τους ιατρούς. Αποθηκεύει τα επιπλέον στοιχεία που αφορούν αποκλειστικά ιατρούς: αριθμό άδειας, ειδικότητα, βαθμίδα και επόπτη. Κάθε εγγραφή στον doctor αντιστοιχεί 1:1 σε εγγραφή στον staff με staff_type = 'Ιατρός'.

Κώδικας SQL

```
CREATE TABLE doctor (
    staff_id INT NOT NULL,
    license_number VARCHAR(50) NOT NULL,
    specialty VARCHAR(100) NOT NULL,
    rank VARCHAR(20) NOT NULL,
    supervisor_id INT,
    CONSTRAINT pk_doctor PRIMARY KEY (staff_id),
    CONSTRAINT uq_doctor_license UNIQUE (license_number),
    CONSTRAINT chk_doctor_rank CHECK (rank IN ('Ειδικευόμενος', 'Επιμελητής Β',
'Επιμελητής Α', 'Διευθυντής')),
    CONSTRAINT chk_director_no_supervisor CHECK (rank <> 'Διευθυντής' OR
supervisor_id IS NULL),
    CONSTRAINT chk_resident_has_supervisor CHECK (rank <> 'Ειδικευόμενος' OR
supervisor_id IS NOT NULL),
    CONSTRAINT chk_doctor_no_self_supervisor CHECK (supervisor_id <> staff_id),
    CONSTRAINT fk_doctor_staff FOREIGN KEY (staff_id) REFERENCES staff
(staff_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_doctor_supervisor FOREIGN KEY (supervisor_id) REFERENCES
doctor (staff_id) ON DELETE SET NULL ON UPDATE CASCADE,
```

```
CONSTRAINT fk_doctor_specialty FOREIGN KEY (specialty) REFERENCES
doctor_specialty (name) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
staff_id	INT		NOT NULL PK FK → staff	ID ιατρού = ID μέλους προσωπικού. Κοινό κλειδί ISA – δεν χρειάζεται ξεχωριστό PK. ON DELETE CASCADE: αν διαγραφεί το μέλος, διαγράφεται και η εγγραφή doctor.
license_number	VARCHAR (50)		NOT NULL UNIQUE	Αριθμός άδειας ασκήσεως επαγγέλματος. UNIQUE γιατί κανένας δύο ιατροί δεν μπορούν να έχουν ίδιο αριθμό. VARCHAR γιατί μπορεί να περιέχει γράμματα και αριθμούς.
specialty	VARCHAR (100)		NOT NULL FK → doctor_specialty	Ιατρική ειδικότητα. NOT NULL γιατί κάθε ιατρός έχει υποχρεωτικά ειδικότητα. FK εξασφαλίζει ότι είναι έγκυρη. ON DELETE RESTRICT.
rank	VARCHAR (20)		NOT NULL CHECK	Βαθμίδα ιατρού. CHECK περιορίζει σε: 'Ειδικευόμενος', 'Επιμελητής Β', 'Επιμελητής Α', 'Διευθυντής'.
supervisor_id	INT		NULL FK → doctor	Επόπτης ιατρός. NULL για Επιμελητές και Διευθυντές. Υποχρεωτικό για Ειδικευόμενους (CHECK). Αυτοαναφορά στον doctor. ON DELETE SET NULL.

Explanations & Constraints

CONSTRAINT chk_director_no_supervisor

Ο Διευθυντής δεν μπορεί να έχει επόπτη — είναι η ανώτατη βαθμίδα. Αν supervisor_id IS NOT NULL και rank = 'Διευθυντής', η εγγραφή απορρίπτεται.

CONSTRAINT chk_resident_has_supervisor

Κάθε Ειδικευόμενος πρέπει υποχρεωτικά να έχει επόπτη. Αποτρέπεται η εισαγωγή Ειδικευόμενου χωρίς supervisor_id.

CONSTRAINT chk_doctor_no_self_supervisor

Ένας ιατρός δεν μπορεί να είναι ο ίδιος επόπτης του εαυτού του. Απλός αλλά κρίσιμος έλεγχος εγκυρότητας.

CONSTRAINT fk_doctor_supervisor ... ON DELETE SET NULL

Αν ο επόπτης ιατρός διαγραφεί, το supervisor_id γίνεται NULL αντί να διαγραφεί ο Ειδικευόμενος. Το ιατρικό προσωπικό διατηρείται στο σύστημα ανεξάρτητα από τον επόπτη.

CONSTRAINT fk_doctor_specialty ... ON DELETE RESTRICT

Αν μια ειδικότητα προσπαθήσει να διαγραφεί ενώ υπάρχουν ιατροί με αυτή την ειδικότητα, η διαγραφή απαγορεύεται. Δεν μπορεί να υπάρξει ιατρός με άγνωστη ειδικότητα.

Σχετικά Triggers**Ενότητα Triggers → trg_shift_supervisor_presence**

Κατά τη δημιουργία βάρδιας, ελέγχεται ότι αν συμμετέχει Ειδικευόμενος, υπάρχει τουλάχιστον ένας ιατρός βαθμίδας Επιμελητής Α ή Διευθυντής στην ίδια βάρδια. Απαιτεί έλεγχο στον doctor.

Πίνακας 3.4: doctor_department**Σκοπός**

Ο πίνακας **doctor_department** είναι το junction table που υλοποιεί τη σχέση many-to-many μεταξύ ιατρών και τμημάτων. Ένας ιατρός μπορεί να ανήκει σε ένα ή περισσότερα τμήματα και ένα τμήμα μπορεί να έχει πολλούς ιατρούς.

Κώδικας SQL

```
CREATE TABLE doctor_department (
  doctor_id INT NOT NULL,
  department_name VARCHAR(100) NOT NULL,
  CONSTRAINT pk_doctor_department PRIMARY KEY (doctor_id, department_name),
  CONSTRAINT fk_dd_doctor FOREIGN KEY (doctor_id) REFERENCES doctor
(staff_id) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_dd_dept FOREIGN KEY (department_name) REFERENCES department
(department_name) ON DELETE CASCADE ON UPDATE CASCADE
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
doctor_id	INT	NOT NULL	PK, FK → doctor	Σύνδεση με τον ιατρό. ON DELETE CASCADE: αν διαγραφεί ο ιατρός, αφαιρείται αυτόματα από όλα τα τμήματα.
department_name	VARCHAR (100)	NOT NULL	PK, FK → department	Σύνδεση με το τμήμα. ON DELETE CASCADE: αν διαγραφεί τμήμα, αφαιρούνται αυτόματα οι αναθέσεις ιατρών.

Explanations

ON DELETE CASCADE και στα δύο FKs

Η ανάθεση ιατρού-τμήματος δεν έχει νόημα αν ένα από τα δύο διαγραφεί. Και στις δύο περιπτώσεις, η αυτόματη διαγραφή της σχέσης είναι η σωστή συμπεριφορά — δεν χάνονται δεδομένα, απλώς αφαιρείται ο σύνδεσμος.

Πίνακας 3.5: nurse**Σκοπός**

Ο πίνακας **nurse** είναι ο subtype της ISA ιεραρχίας για τους νοσηλευτές. Αποθηκεύει τα επιπλέον στοιχεία που αφορούν αποκλειστικά νοσηλευτές: βαθμίδα και τμήμα ανάθεσης. Κάθε εγγραφή αντιστοιχεί 1:1 σε εγγραφή στον staff με staff_type = 'Νοσηλευτής'.

Κώδικας SQL

```
CREATE TABLE nurse (
    staff_id INT NOT NULL,
    nurse_rank VARCHAR(20) NOT NULL CHECK (nurse_rank IN ('Βοηθός Νοσηλευτή',
'Νοσηλευτής', 'Προϊστάμενος')),
    department_name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_nurse PRIMARY KEY (staff_id),
    CONSTRAINT fk_nurse_staff FOREIGN KEY (staff_id) REFERENCES staff
(staff_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_nurse_dept FOREIGN KEY (department_name) REFERENCES
department (department_name) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
staff_id	INT		NOT NULL PK FK → staff	ID νοσηλευτή = ID μέλους προσωπικού. Κοινό κλειδί ISA. ON DELETE CASCADE: αν διαγραφεί το μέλος, διαγράφεται και η εγγραφή nurse.
nurse_rank	VARCHAR (20)		NOT NULL CHECK	Βαθμίδα νοσηλευτή. CHECK περιορίζει σε: 'Βοηθός Νοσηλευτή', 'Νοσηλευτής', 'Προϊστάμενος'. NOT NULL γιατί κάθε νοσηλευτής έχει υποχρεωτικά βαθμίδα.
department_name	VARCHAR (100)		NOT NULL FK → department	Τμήμα ανάθεσης νοσηλευτή. NOT NULL γιατί κάθε νοσηλευτής ανήκει υποχρεωτικά σε ένα τμήμα. ON DELETE RESTRICT: το τμήμα δεν μπορεί να διαγραφεί αν έχει νοσηλευτές.

Explanations & Constraints

Ένα τμήμα ανά νοσηλευτή (αντί για M:N)

Σε αντίθεση με τους ιατρούς που μπορούν να ανήκουν σε πολλά τμήματα (M:N μέσω `doctor_department`), κάθε νοσηλευτής ανήκει σε ένα μόνο τμήμα. Αυτό αντικατοπτρίζει την επιχειρησιακή πραγματικότητα: ο νοσηλευτής έχει σταθερή θέση σε τμήμα, σε αντίθεση με τον ιατρό που μπορεί να εξετάζει σε πολλά.

CONSTRAINT `fk_nurse_dept ... ON DELETE RESTRICT`

Αν ένα τμήμα προσπαθήσει να διαγραφεί ενώ έχει νοσηλευτές, η διαγραφή απαγορεύεται. Το προσωπικό δεν μπορεί να μείνει χωρίς τμήμα.

Πίνακας 3.6: `admin_staff`

Σκοπός

Ο πίνακας `admin_staff` είναι ο subtype της ISA ιεραρχίας για το διοικητικό προσωπικό. Αποθηκεύει τον ρόλο, το γραφείο και το τμήμα ανάθεσης κάθε διοικητικού υπαλλήλου. Κάθε εγγραφή αντιστοιχεί 1:1 σε εγγραφή στον `staff` με `staff_type = 'Διοικητικό Προσωπικό'`.

Κώδικας SQL

```
CREATE TABLE admin_staff (
    staff_id INT NOT NULL,
    role VARCHAR(100) NOT NULL,
    office VARCHAR(100),
    department_name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_admin_staff PRIMARY KEY (staff_id),
    CONSTRAINT fk_admin_role FOREIGN KEY (role) REFERENCES admin_role (name) ON
DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_admin_staff_s FOREIGN KEY (staff_id) REFERENCES staff
(staff_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_admin_staff_dept FOREIGN KEY (department_name) REFERENCES
department (department_name) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
<code>staff_id</code>	INT	NOT NULL	PK, FK → <code>staff</code>	ID διοικητικού = ID μέλους προσωπικού. Κοινό κλειδί ISA. ON DELETE CASCADE: αν

Στήλη	Τύπος		Constraint	Αιτιολόγηση
				διαγραφεί το μέλος, διαγράφεται και η εγγραφή admin_staff.
role	VARCHAR(100)	NOT NULL	FK → admin_role	Ρόλος διοικητικού προσωπικού. FK εξασφαλίζει έγκυρο ρόλο. NOT NULL γιατί κάθε υπάλληλος έχει υποχρεωτικά ρόλο. ON DELETE RESTRICT.
office	VARCHAR(100)	NULL	NULL	Γραφείο εργασίας. NULL γιατί είναι προαιρετικό, μπορεί να μην έχει σταθερό γραφείο.
department_name	VARCHAR(100)	NOT NULL	FK → department	Τμήμα ανάθεσης. NOT NULL γιατί κάθε διοικητικός ανήκει υποχρεωτικά σε τμήμα. ON DELETE RESTRICT: το τμήμα δεν μπορεί να διαγραφεί αν έχει διοικητικό προσωπικό.

Explanations

Ίδια λογική με τον doctor_specialty: το όνομα ρόλου χρησιμοποιείται απευθείας ως PK αντί για surrogate key, γιατί είναι αναγνώσιμο, μοναδικό και σταθερό. Αποτρέπονται ασυνέπειες ελεύθερου κειμένου στον admin_staff.

Πίνακας 3.7: admin_role

Σκοπός

Ο πίνακας **admin_role** είναι lookup table που αποθηκεύει τους αναγνωρισμένους ρόλους διοικητικού προσωπικού. Χρησιμοποιείται ως FK στον admin_staff για να εξασφαλιστεί ότι κάθε διοικητικός υπάλληλος έχει έγκυρο ρόλο.

Κώδικας SQL

```
CREATE TABLE admin_role (name VARCHAR(100) PRIMARY KEY);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
name	VARCHAR(100)	NOT NULL	PK	Όνομα ρόλου (π.χ. 'Γραμματέας', 'Λογιστής', 'Τεχνικός', 'Προσωπικό IT'). Φυσικό PK, μοναδικό και σταθερό. Αποτρέπει ασυνέπειες ελεύθερου κειμένου στον admin_staff.

Explanations

Γιατί ξεχωριστός υπότυπος για το διοικητικό προσωπικό

Το διοικητικό προσωπικό συμμετέχει στις βάρδιες (shifts / duty_assignment) όπως και ιατροί και νοσηλεύτες, άρα χρειάζεται εγγραφή στον staff. Ωστόσο δεν έχει ειδικότητα, βαθμίδα ή αριθμό άδειας — τα μοναδικά του χαρακτηριστικά είναι ο ρόλος και το γραφείο, που αποθηκεύονται εδώ.

Ενότητα 4: PATIENTS

Η ενότητα αυτή διαχειρίζεται όλα τα δημογραφικά και προσωπικά στοιχεία των ασθενών του νοσοκομείου. Αποτελείται από τρεις πίνακες που μαζί καλύπτουν το πλήρες φάσμα πληροφοριών που απαιτεί η επιχειρησιακή απαίτηση.

Προδιαγραφές Απαιτήσεων Συστήματος

Οι ασθενείς διαθέτουν ΑΜΚΑ, όνομα, επώνυμο, πατρώνυμο, ηλικία, φύλο, βάρος, ύψος, διεύθυνση, τηλέφωνο, email, επάγγελμα, υπηκοότητα, στοιχεία οικείου(-ων) ατόμου(-ων) σε περίπτωση ανάγκης, και ασφαλιστικό φορέα (πχ ΕΦΚΑ, ιδιωτική ασφάλεια, ανασφάλιστος). Κάθε ασθενής μπορεί να έχει πολλαπλές νοσηλείες στο ιστορικό του.

Πίνακας 4.1: patient

Σκοπός

Ο πίνακας **patient** λειτουργεί ως το **Κεντρικό Ευρετήριο Ασθενών (Master Patient Index)** του νοσοκομείου. Κάθε ασθενής εγγράφεται μία φορά και αποκτά έναν μοναδικό αναγνωριστικό (**patient_id**) που παραμένει αμετάβλητος για πάντα. Ο πίνακας περιέχει αποκλειστικά δημογραφικά και διοικητικά στοιχεία κάθε ασθενή. Τα ιατρικά στοιχεία (βάρος, ύψος, αλλεργίες, συμπτώματα κλπ.) δεν συμπεριλαμβάνονται σε αυτόν τον πίνακα, γιατί αποθηκεύονται σε ξεχωριστούς πίνακες από εξουσιοδοτημένο ιατρικό προσωπικό.

Δομή πεδίων

Ομάδα	Πεδία
PK	patient_id
Ταυτοποίηση	amka
Ονοματεπώνυμο	first_name, last_name, fathers_name
Δημογραφικά	dob, gender
Επικοινωνία	street, city, postal_code
Διοικητικά	occupation, nationality, insurer
Σύστημα	image_url, created_at, updated_at

Κώδικας SQL

```
CREATE TABLE patient (
  patient_id SERIAL NOT NULL,
  amka VARCHAR(11),
  first_name VARCHAR(50),
```

```

last_name VARCHAR(50),
fathers_name VARCHAR(50),
dob DATE CHECK (
    dob <= CURRENT_DATE
    AND dob >= CURRENT_DATE - INTERVAL '120 years'
),
gender VARCHAR(10) CHECK (gender IN ('Ανδρας', 'Γυναίκα', 'Άλλο')),
street VARCHAR(100),
city VARCHAR(100),
postal_code VARCHAR(10),
occupation VARCHAR(100),
nationality CHAR(2),
insurer VARCHAR(100) CHECK (
    insurer IN (
        'e-ΕΦΚΑ / ΕΟΠΥΥ',
        'Στρατιωτικά Ταμεία',
        'ΕΚΑΑ',
        'Ιδιωτική Ασφάλιση',
        'Ανασφάλιστος'
    )
),
image_url VARCHAR(255),
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP,
CONSTRAINT pk_patient PRIMARY KEY (patient_id),
CONSTRAINT uq_patient_amka UNIQUE (amka),
CONSTRAINT fk_patient_country FOREIGN KEY (nationality) REFERENCES country
(country_code) ON DELETE RESTRICT ON UPDATE CASCADE
);

```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
patient_id	SERIAL	NOT NULL	PK	ID ασθενή. SERIAL γιατί σε emergency περιστατικά το amka μπορεί να μην είναι γνωστό, άρα δεν μπορεί να εγγυηθεί μοναδικότητα ως PK.
amka	VARCHAR(11)	NULL	UNIQUE, CHECK (LENGTH(amka) = 11 AND amka ~ '^\d{11}\$')	AMKA ασθενή. 11-ψήφιος κωδικός. VARCHAR γιατί τα μπροστινά μηδενικά πρέπει να διατηρηθούν. UNIQUE γιατί κανένας δύο ασθενείς δεν μπορεί να έχουν ίδιο AMKA.

Στήλη	Τύπος		Constraint	Αιτιολόγηση
first_name	VARCHAR(50)	NULL	–	Όνομα ασθενή. VARCHAR(50): μεταβλητό κείμενο με μήκος έως 50 χαρακτήρες. NULL για υποστήριξη emergency περιστατικών.
last_name	VARCHAR(50)	NULL	–	Επίθετο ασθενή. VARCHAR(50): μεταβλητό κείμενο με μήκος έως 50 χαρακτήρες. NULL για υποστήριξη emergency περιστατικών.
fathers_name	VARCHAR(50)	NULL	–	Πατρώνυμο ασθενή. VARCHAR(50): μεταβλητό κείμενο με μήκος έως 50 χαρακτήρες. NULL: προαιρετικό για κάθε ασθενή (δεν είναι πάντα διαθέσιμο ή εφαρμόσιμο)
dob	DATE	NULL	CHECK (dob <= CURRENT_DATE AND dob >= CURRENT_DATE - INTERVAL '120 years')	Ημερομηνία γέννησης ασθενή (date of birth). DATE: αποθηκεύει μόνο την ημερομηνία. NOT NULL: υποχρεωτικό γιατί κάθε ασθενής πρέπει να έχει ημερομηνία γέννησης. CHECK (...): εξασφαλίζει ότι η ημερομηνία γέννησης είναι έγκυρη. CURRENT_DATE: αυτόματη ενημέρωση.
gender	VARCHAR(10)	NULL	CHECK (gender IN ('Ανδρας', 'Γυναίκα', 'Άλλο'))	Φύλο ασθενή. VARCHAR(10): μεταβλητό κείμενο με μήκος έως 10 χαρακτήρες (αρκετό για τις αποδεκτές τιμές). NULL για emergency περιστατικά. CHECK (...): περιορίζει σε: 'Ανδρας', 'Γυναίκα', 'Άλλο'.
street	VARCHAR(100)	NULL	–	Οδός και αριθμός κατοικίας.
city	VARCHAR(100)	NULL		Πόλη κατοικίας.
postal_code	VARCHAR(10)			Ταχυδρομικός κώδικας. VARCHAR γιατί μπορεί να περιέχει γράμματα (π.χ. βρετανικοί TK).
occupation	VARCHAR(100)	NULL	–	Επάγγελμα ασθενή. VARCHAR(100): ελεύθερο κείμενο γιατί τα επαγγέλματα είναι αναρίθμητα. NULL: προαιρετικό (δεν είναι απαραίτητο για την εγγραφή του ασθενή και μπορεί να μην είναι γνωστό).
nationality	CHAR(2)	NULL	FK → country	Υπηκοότητα ασθενή βάσει ISO 3166-1 alpha-2. NULL για emergency περιστατικά. FK εξασφαλίζει έγκυρο κωδικό χώρας.
insurer_id	INT	NULL	FK → insurer	Ασφαλιστικός φορέας του ασθενή. NULL για emergency περιστατικά όπου δεν είναι ακόμα γνωστός. Υποχρεωτικό πριν την έξοδο μέσω trigger. FK εξασφαλίζει ότι ο φορέας είναι έγκυρος και αναγνωρισμένος.
image_url	VARCHAR(255)	NULL		URL φωτογραφίας ασθενή. Προαιρετικό.
created_at	TIMESTAMP	NOT NULL	DEFAULT CURRENT_TIMESTAMP	Χρονική σήμανση δημιουργίας της εγγραφής. NOT NULL: υποχρεωτικό (κάθε εγγραφή πρέπει να έχει χρόνο δημιουργίας). DEFAULT CURRENT_TIMESTAMP: συμπληρώνεται αυτόματα από το σύστημα χωρίς παρέμβαση.
updated_at	TIMESTAMP	NULL		Χρονική σήμανση τελευταίας ενημέρωσης της εγγραφής. NULL: προαιρετικό (είναι κενό όταν η εγγραφή δεν έχει ενημερωθεί ακόμα από την αρχική δημιουργία της).

Assumption

Γιατί όλα τα δημογραφικά είναι NULL

Τα δημογραφικά πεδία ορίζονται ως NULL για να υποστηρίζονται emergency περιστατικά, ο ασθενής μπορεί να φτάσει αναίσθητος χωρίς κανένα γνωστό στοιχείο. Η εγγραφή δημιουργείται αμέσως και τα στοιχεία συμπληρώνονται αργότερα. **Η ταυτοποίηση επιβάλλεται υποχρεωτικά πριν την έξοδο μέσω trigger.**

address

Το πεδίο address της εκφώνησης αναλύεται σε τρία ξεχωριστά πεδία (street, city, postal_code) για πλήρη κανονικοποίηση και αποφυγή ασυνεπειών ελεύθερου κειμένου.

age

Το age δεν αποθηκεύεται, υπολογίζεται δυναμικά από το dob. Είναι **derived attribute** που προκύπτει δυναμικά από την **dob** και την τρέχουσα ημερομηνία. Αποθήκευση της **age** θα απαιτούσε ενημέρωση κάθε χρόνο με κίνδυνο ασυνέπειας.

Explanations

amka VARCHAR(11) CHECK (LENGTH(amka) = 11)

Το **amka** είναι VARCHAR και όχι INT γιατί μπορεί να αρχίζει από μηδέν — αν το ορίζαμε ως INT, το μπροστινό μηδέν θα χανόταν. Το **CHECK (LENGTH(amka) = 11)** εξασφαλίζει ακριβώς 11 χαρακτήρες. **UNIQUE** γιατί κανένας δύο ασθενείς δεν μπορεί να μοιράζονται το ίδιο AMKA. **NULL** γιατί αλλοδαποί ασθενείς δεν έχουν AMKA, το **UNIQUE** δεν παραβιάζεται από πολλαπλές NULL τιμές στην PostgreSQL. Υποχρεωτικό πριν την έξοδο μόνο για Έλληνες (**nationality = 'GR'**) μέσω trigger.

dob DATE NULL CHECK (dob <= CURRENT_DATE AND dob >= CURRENT_DATE - INTERVAL '120 years')

Για το **CHECK** χρησιμοποιείται **CURRENT_DATE** αντί για σταθερή χρονολογία ώστε ο περιορισμός να **ενημερώνεται αυτόματα** κάθε μέρα χωρίς καμία παρέμβαση, και εξασφαλίζω ότι η ημερομηνία γέννησης δεν είναι στο μέλλον (δεν ξεπερνά την σημερινή και δεν υπερβαίνει τα 120 χρόνια από σήμερα).

nationality ως FK → country

Η υπηκοότητα αποθηκεύεται ως κωδικός ISO 3166-1 alpha-2 (π.χ. 'GR', 'DE', 'FR') και όχι ως ελεύθερο κείμενο. Αυτό εξασφαλίζει: Συνέπεια (αποτρέπονται ασυνέπειες όπως "Ελληνική", "Greek", "ΕΛΛΗΝΙΚΗ" που θα έκαναν αδύνατο τον αξιόπιστο έλεγχο), ότι δεν μπορεί να καταχωρηθεί άγνωστη ή λανθασμένη χώρα και ότι το trigger **trg_patient_discharge_check** ελέγχει **nationality = 'GR'** για να καθορίσει αν το **amka** είναι υποχρεωτικό πριν την έξοδο

ON DELETE RESTRICT: αν μια χώρα προσπαθήσει να διαγραφεί από τον country ενώ υπάρχουν ασθενείς με αυτή την υπηκοότητα, η διαγραφή απαγορεύεται.

ON UPDATE CASCADE: αν αλλάξει ο κωδικός μιας χώρας στον country, η αλλαγή μεταδίδεται αυτόματα στον patient.

insurer_id ως FK → insurer

Ο ασφαλιστικός φορέας αποθηκεύεται ως FK που δείχνει στον πίνακα **insurer** αντί για ελεύθερο κείμενο. Αυτό εξασφαλίζει: Συνέπεια (αποτρέπονται ασυνέπειες όπως "ΕΦΚΑ", "εφκα", "Ε.Φ.Κ.Α." που θα έκαναν αδύνατο τον αξιόπιστο έλεγχο), ότι δεν μπορεί να καταχωρηθεί άγνωστος φορέας και ότι η κατανομή νοσηλείων ανά ασφαλιστικό φορέα είναι πάντα σωστή.

ON DELETE RESTRICT: αν ένας φορέας προσπαθήσει να διαγραφεί ενώ υπάρχουν ασθενείς που τον αναφέρουν, η διαγραφή απαγορεύεται.

ON UPDATE CASCADE: αν αλλάξει το `insurer_id`, η αλλαγή μεταδίδεται αυτόματα στον `patient`.

updated_at TIMESTAMP NULL

Ενημερώνεται αυτόματα μέσω trigger `trg_patient_updated_at` κάθε φορά που τροποποιείται η εγγραφή (δεν απαιτείται χειροκίνητη ενημέρωση από την εφαρμογή).

Για περισσότερα: Βλέπε ενότητα **Triggers** → `trg_patient_updated_at`

Σχετικά Triggers

Ενότητα Triggers → `trg_patient_discharge_check`

Ενεργοποιείται κατά την ενημέρωση του `discharge_date` στον `hospitalization`. Ο σκοπός του είναι να εξασφαλίσει ότι κανένας ασθενής δεν εξέρχεται από το νοσοκομείο χωρίς πλήρη ταυτοποίηση. Ελέγχει τα εξής:

- `amka` → υποχρεωτικό μόνο αν `nationality = 'GR'` — αλλοδαποί ασθενείς δεν έχουν AMKA
- `first_name`, `last_name`, `dob`, `gender`, `nationality`, `street`, `city`, `postal_code` → υποχρεωτικά για όλους τους ασθενείς
- `insurer` → υποχρεωτικό για όλους τους ασθενείς, η ασφαλιστική κατάσταση πρέπει να είναι γνωστή για τη χρέωση

Ενότητα Triggers → `trg_patient_updated_at`

Ενεργοποιείται αυτόματα κατά κάθε **UPDATE** της εγγραφής και ενημερώνει το `updated_at` με την τρέχουσα χρονική σήμανση. Εξασφαλίζει ότι κάθε αλλαγή στα στοιχεία του ασθενή καταγράφεται αυτόματα χωρίς χειροκίνητη παρέμβαση.

Πίνακας 4.2: `emergency_contact`

Σκοπός

Ο πίνακας `emergency_contact` αποθηκεύει τα στοιχεία των προσώπων επαφής σε περίπτωση ανάγκης. Κάθε πρόσωπο επαφής αντιμετωπίζεται ως ανεξάρτητη οντότητα, καταχωρείται μία φορά στη βάση και μπορεί να συνδεθεί με έναν ή περισσότερους ασθενείς (π.χ. ένας γονέας που είναι οικείος δύο παιδιών που νοσηλεύονται). Η σχέση μεταξύ ασθενών και οικείων υλοποιείται μέσω του junction table `patient_emergency_contact`.

Κώδικας SQL

```
CREATE TABLE emergency_contact (
    contact_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    fathers_name VARCHAR(50),
    phone VARCHAR(20) NOT NULL,
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
contact_id	SERIAL	NOT NULL	PK	ID οικείου προσώπου. SERIAL γιατί ο πίνακας πλέον αποθηκεύει ανεξάρτητες οντότητες και κάθε μία χρειάζεται δικό της σταθερό ID.
first_name	VARCHAR(50)	NOT NULL	—	Όνομα οικείου. NOT NULL γιατί όταν καταχωρείται οικείος, το όνομά του είναι γνωστό.
last_name	VARCHAR(50)	NOT NULL	—	Επώνυμο οικείου. NOT NULL για τον ίδιο λόγο.
fathers_name	VARCHAR(50)	NULL	—	Πατρώνυμο οικείου. Προαιρετικό, γιατί δεν είναι πάντα διαθέσιμο ή αναγκαίο.
phone	VARCHAR(20)	NOT NULL		Τηλέφωνο οικείου. NOT NULL γιατί χωρίς τηλέφωνο δεν έχει νόημα η καταχώρηση οικείου. VARCHAR γιατί μπορεί να περιέχει +, κενά, παρενθέσεις.

Explanations

Γιατί ο contact δεν έχει patient_id ως στήλη;

Το πρόσωπο επαφής αντιμετωπίζεται ως ανεξάρτητη οντότητα, καταχωρείται μία φορά και συνδέεται με ασθενείς μέσω του junction table **patient_emergency_contact**. Έτσι αποτρέπεται η αποθήκευση διπλότυπων εγγραφών για το ίδιο πρόσωπο (π.χ. ένας γονέας που είναι οικείος δύο ασθενών καταχωρείται μία φορά, όχι δύο).

Γιατί δεν υπάρχει relation στον emergency_contact;

Η σχέση δεν είναι ιδιότητα του ίδιου του προσώπου, είναι **ιδιότητα της σύνδεσης μεταξύ ασθενή και οικείου**. Επομένως ανήκει στον πίνακα **patient_emergency_contact**, όχι εδώ. Αυτό είναι και η σωστή κανονικοποιημένη τοποθέτησή της.

Πίνακας 4.3: patient_emergency_contact (junction table)

Σκοπός

Ο πίνακας **patient_emergency_contact** είναι το **junction table** που υλοποιεί τη σχέση **many-to-many** μεταξύ ασθενών και οικείων. Κάθε εγγραφή εκφράζει ότι ένα συγκεκριμένο πρόσωπο επαφής είναι δηλωμένος οικείος ενός συγκεκριμένου ασθενή, με τη σχέση τους (π.χ. πατέρας, σύζυγος) ως επιπλέον χαρακτηριστικό της σύνδεσης.

Κώδικας SQL

```
CREATE TABLE patient_emergency_contact (
    patient_id INT NOT NULL,
    contact_id INT NOT NULL,
    relation VARCHAR(50),
```

```
PRIMARY KEY (patient_id, contact_id),
CONSTRAINT fk_pec_patient FOREIGN KEY (patient_id) REFERENCES patient
(patient_id) ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_pec_contact FOREIGN KEY (contact_id) REFERENCES
emergency_contact (contact_id) ON DELETE CASCADE ON UPDATE CASCADE
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
patient_id	INT	NOT NULL	PK, FK → patient	Σύνδεση με τον ασθενή. NOT NULL γιατί κάθε εγγραφή πρέπει να αντιστοιχεί σε γνωστό ασθενή.
contact_id	INT	NOT NULL	PK, FK → emergency_contact	Σύνδεση με το πρόσωπο επαφής. NOT NULL γιατί κάθε εγγραφή πρέπει να αντιστοιχεί σε γνωστό οικείο.
relation	VARCHAR(50)	NULL		Σχέση με τον ασθενή (πχ πατέρας, σύζυγος). Ελεύθερο κείμενο γιατί οι σχέσεις είναι πολυάριθμες. Προαιρετικό.

Explanations & Constrains

CONSTRAINT pk_pec PRIMARY KEY (patient_id, contact_id)

Σύνθετο πρωτεύον κλειδί. Εξασφαλίζει ότι ο ίδιος οικείος δεν μπορεί να καταχωρηθεί δύο φορές για τον ίδιο ασθενή. Δεν απαιτείται surrogate key γιατί ο συνδυασμός (**patient_id**, **contact_id**) είναι από μόνος του μοναδικός και σημαντικός.

CONSTRAINT fk_pec_patient ... ON DELETE CASCADE ON UPDATE CASCADE

Αν διαγραφεί ο ασθενής, διαγράφονται αυτόματα και όλες οι συνδέσεις του με οικείους — αποφεύγουμε ορφανές εγγραφές. Αν αλλάξει το **patient_id**, η αλλαγή μεταδίδεται αυτόματα.

ON DELETE CASCADE ON UPDATE CASCADE

Αν διαγραφεί ο οικείος από τον **emergency_contact**, διαγράφονται αυτόματα και όλες οι συνδέσεις του με ασθενείς. Αν αλλάξει το **contact_id**, η αλλαγή μεταδίδεται αυτόματα.

relation ως ιδιότητα του junction

Η **relation** τοποθετείται εδώ και όχι στον **emergency_contact** γιατί είναι ιδιότητα της σχέσης μεταξύ ασθενή και οικείου, όχι του ίδιου του οικείου. Ο ίδιος άνθρωπος μπορεί να είναι "πατέρας" για τον έναν ασθενή και "παππούς" για τον άλλον.

Assumptions

Δεν τίθεται CHECK constraint στο **relation** γιατί οι σχέσεις είναι αναρίθμητες και πολιτισμικά ποικίλες. Αν η εφαρμογή απαιτεί κλειστό σύνολο τιμών στο μέλλον, μπορεί να προστεθεί **CHECK (relation IN (...))** χωρίς δομικές αλλαγές στον πίνακα.

Πίνακας 4.4: patient_allergy (weak entity)

Σκοπός

Ο πίνακας **patient_allergy** αποθηκεύει τις αλλεργίες κάθε ασθενή σε δραστικές ουσίες. Ξεχωριστός πίνακας γιατί ένας ασθενής μπορεί να έχει πολλαπλές αλλεργίες. Κάθε αλλεργία συνδέεται με μια καταχωρημένη δραστική ουσία από τον πίνακα **active_substance**, εξασφαλίζοντας ότι δεν μπορεί να καταχωρηθεί αλλεργία σε άγνωστη ουσία. Ο πίνακας αυτός χρησιμοποιείται από το **trigger** **ελέγχου αλλεργιών** κατά την έκδοση συνταγής (βλ. Triggers → **trg_allergy_check**).

Κώδικας SQL

```
CREATE TABLE patient_allergy (
    allergy_id SERIAL NOT NULL,
    patient_id INT NOT NULL,
    active_substance VARCHAR(200) NOT NULL,
    CONSTRAINT pk_patient_allergy PRIMARY KEY (allergy_id),
    CONSTRAINT fk_pa_patient FOREIGN KEY (patient_id) REFERENCES patient
(patient_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_pa_substance FOREIGN KEY (active_substance) REFERENCES
active_substance (name) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT uq_pa_patient_subst UNIQUE (patient_id, active_substance)
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
allergy_id	SERIAL	NOT NULL	PK	ID κάθε εγγραφής αλλεργίας. SERIAL γιατί ένας ασθενής μπορεί να έχει πολλές αλλεργίες και κάθε μία χρειάζεται δικό της ID.
patient_id	INT	NOT NULL	FK → patient	Σύνδεση με τον ασθενή . NOT NULL γιατί κάθε αλλεργία ανήκει υποχρεωτικά σε έναν ασθενή.
active_substance	VARCHAR(200)	NOT NULL	FK → active_substance	Η δραστική ουσία στην οποία ο ασθενής έχει αλλεργία . VARCHAR(200) γιατί τα INN ονόματα μπορεί να είναι αρκετά μακριά. NOT NULL γιατί δεν έχει νόημα εγγραφή αλλεργίας χωρίς ουσία. FK γιατί η ουσία πρέπει να υπάρχει ήδη στο active_substance .

Explanations & Constraints

CONSTRAINT fk_pa_substance ... ON DELETE RESTRICT

Αν προσπαθήσεις να διαγράψεις μια ουσία από τον **active_substance** ενώ κάποιος ασθενής έχει αλλεργία σε αυτή, η διαγραφή απαγορεύεται. Εξασφαλίζεται έτσι ότι το ιατρικό ιστορικό αλλεργιών δεν χάνεται ποτέ αθόρυβα.

CONSTRAINT fk_pa_substance ... ON UPDATE CASCADE

Αν αλλάξει το όνομα μιας ουσίας στον **active_substance**, η αλλαγή μεταδίδεται αυτόματα και στον **patient_allergy**. Δεν απαιτείται χειροκίνητη ενημέρωση.

CONSTRAINT uq_pa_patient_subst UNIQUE (patient_id, active_substance)

Εξασφαλίζει ότι ο ίδιος ασθενής δεν μπορεί να έχει καταχωρηθεί δύο φορές η ίδια αλλεργία. Ο συνδυασμός (**patient_id**, **active_substance**) είναι μοναδικός.

TRIGGER ελέγχου αλλεργιών

Ο πίνακας αυτός είναι κεντρικός για την ασφάλεια των συνταγογραφήσεων. Πριν από κάθε έκδοση συνταγής, το σύστημα ελέγχει αν κάποια δραστική ουσία του φαρμάκου υπάρχει στις αλλεργίες του ασθενή. Αν ναι, η συνταγή απαγορεύεται αυτόματα.

Ενότητα 5: Triage

Η ενότητα αυτή διαχειρίζεται τη **διαδικασία διαλογής** των ασθενών κατά την άφιξή τους στο **Τμήμα Επειγόντων Περιστατικών**. Ο πίνακας **triage** είναι το **σημείο εισόδου** κάθε ασθενή στο σύστημα νοσηλείας. Κάθε εγγραφή καταγράφει την άφιξη, την αξιολόγηση επείγοντος και την τελική έκβαση, αποτελώντας τη **γέφυρα** μεταξύ της άφιξης του ασθενή και της απόφασης για νοσηλεία.

Προδιαγραφές Απαιτήσεων Συστήματος

Κάθε ασθενής που καταφθάνει στο νοσοκομείο, οδηγείται στο Τμήμα Επειγόντων Περιστατικών και περνά από τον νοσηλευτή διαλογής (triage), ο οποίος καταγράφει τα συμπτώματα και ορίζει το επίπεδο επείγοντος (1=άμεσο, 2=επείγον, 3=επιτακτικό, 4=λιγότερο επείγον, 5=μη επείγον). Οι ασθενείς εξυπηρετούνται με βάση το επίπεδο επείγοντος και, για το ίδιο επίπεδο, ακολουθείται αυστηρή σειρά προτεραιότητας βάσει χρόνου άφιξης (FIFO)¹. Ο ασθενής είτε παίρνει οδηγίες και αποχωρεί, είτε παραπέμπεται για νοσηλεία.

Πίνακας 5.1: triage

Σκοπός

Ο πίνακας **triage** καταγράφει τη **διαδικασία διαλογής** κάθε ασθενή κατά την άφιξή του στο **Τμήμα Επειγόντων Περιστατικών**. Ο αρμόδιος νοσηλευτής καταγράφει τα συμπτώματα και ορίζει το επίπεδο επείγοντος (1=άμεσο έως 5=μη επείγον). Οι ασθενείς εξυπηρετούνται με βάση το επίπεδο επείγοντος και, για το ίδιο επίπεδο, ακολουθείται αυστηρή σειρά προτεραιότητας **βάσει χρόνου άφιξης** (FIFO). Στο τέλος της διαλογής, ο ασθενής είτε αποχωρεί με οδηγίες είτε παραπέμπεται για νοσηλεία σε συγκεκριμένο τμήμα.

Κώδικας SQL

```
CREATE TABLE triage (
    triage_id SERIAL NOT NULL,
    patient_id INT NOT NULL,
    nurse_id INT NOT NULL,
    referred_dept_id INT,
    arrival_time TIMESTAMP NOT NULL,
    symptoms TEXT,
    urgency_level INT NOT NULL CHECK (urgency_level BETWEEN 1 AND 5),
    outcome VARCHAR(50) CHECK (outcome IN ('αποχώρησε με οδηγίες', 'παραπομπή νοσηλεία')),
    CONSTRAINT pk_triage PRIMARY KEY (triage_id),
    CONSTRAINT fk_tr_patient FOREIGN KEY (patient_id) REFERENCES patient
(patient_id) ON DELETE RESTRICT ON UPDATE CASCADE,
```

```

    CONSTRAINT fk_tr_nurse FOREIGN KEY (nurse_id) REFERENCES nurse (staff_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_tr_dept FOREIGN KEY (referred_dept_id) REFERENCES department
    (department_id) ON DELETE SET NULL ON UPDATE CASCADE
);

```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
triage_id	SERIAL	NOT NULL	PK	ID κάθε διαλογής. SERIAL γιατί ένας ασθενής μπορεί να έχει πολλαπλές επισκέψεις στα Επείγοντα.
patient_id	INT	NOT NULL	FK → patient	Σύνδεση με τον ασθενή. NOT NULL γιατί κάθε διαλογή αφορά υποχρεωτικά έναν ασθενή.
nurse_id	INT	NOT NULL	FK → nurse	Ο νοσηλευτής που διενήργησε τη διαλογή. NOT NULL γιατί κάθε διαλογή πρέπει να έχει αρμόδιο νοσηλευτή.
referred_dept_id	INT	NULL	FK → department	Το τμήμα στο οποίο παραπέμπεται ο ασθενής. NULL αν αποχωρεί με οδηγίες. Η λογική συνέπεια με το outcome εξασφαλίζεται μέσω trigger.
arrival_time	TIMESTAMP	NOT NULL		Χρόνος άφιξης του ασθενή. NOT NULL γιατί χρησιμοποιείται για την εφαρμογή της σειράς FIFO εντός του ίδιου επιπέδου επείγοντος.
symptoms	TEXT	NULL		Συμπτώματα που καταγράφει ο νοσηλευτής. TEXT γιατί η περιγραφή συμπτωμάτων είναι ελεύθερο κείμενο χωρίς προκαθορισμένο μήκος. NULL γιατί σε επείγοντα περιστατικά μπορεί να μην είναι άμεσα διαθέσιμα.
urgency_level	INT	NOT NULL	CHECK (1-5)	Επίπεδο επείγοντος (1=άμεσο, 2=επείγον, 3=επιτακτικό, 4=λιγότερο επείγον, 5=μη επείγον). NOT NULL γιατί η διαλογή δεν έχει νόημα χωρίς αυτό.
outcome	VARCHAR(50)	NULL	CHECK	Έκβαση της διαλογής. NULL αν δεν έχει ακόμα οριστεί. CHECK περιορίζει τις αποδεκτές τιμές σε 'αποχώρησε με οδηγίες' ή 'παραπομπή για νοσηλεία'.

Explanations & Constraints

outcome NULL

Το outcome παραμένει NULL όσο δεν έχει ακόμα οριστεί η έκβαση της διαλογής — ο νοσηλευτής μπορεί να καταγράψει πρώτα τα συμπτώματα και το επίπεδο επείγοντος και να ορίσει το outcome αργότερα. Η λογική συνέπεια μεταξύ outcome και referred_dept_id εξασφαλίζεται μέσω trigger.

urgency_level CHECK (1-5)

Το επίπεδο επείγοντος ορίζεται σε κλίμακα 1-5 με συγκεκριμένη σημασία για κάθε τιμή:

Τιμή	Χαρακτηρισμός	Περιγραφή
1	Άμεσο	Απαιτεί άμεση παρέμβαση.
2	Επείγον	Απαιτεί γρήγορη παρέμβαση.
3	Επιτακτικό	Χρειάζεται αντιμετώπιση αλλά δεν απειλεί άμεσα τη ζωή.
4	Λιγότερο επείγον	Μπορεί να περιμένει.
5	Μη επείγον	Δεν υπάρχει κίνδυνος.

arrival_time TIMESTAMP

Η διαλογή απαιτεί ακριβή χρόνο άφιξης (ημερομηνία + ώρα) γιατί η σειρά FIFO εντός του ίδιου επιπέδου επείγοντος καθορίζεται από τη χρονική στιγμή άφιξης.

CONSTRAINT fk_tr_patient ... ON DELETE RESTRICT

Αν ένας ασθενής έχει ιστορικό διαλογής, δεν μπορεί να διαγραφεί από το σύστημα. **Το ιατρικό ιστορικό πρέπει να διατηρείται.**

CONSTRAINT fk_tr_nurse ... ON DELETE RESTRICT

Αν ένας νοσηλευτής έχει διενεργήσει διαλογές, δεν μπορεί να διαγραφεί από το σύστημα. **Διατηρείται η ιχνηλασιμότητα των ιατρικών πράξεων.**

CONSTRAINT fk_tr_dept ... ON DELETE SET NULL

Αν ένα τμήμα διαγραφεί, το **referred_dept_id** γίνεται NULL αντί να διαγραφεί η εγγραφή διαλογής. **Το ιστορικό της διαλογής διατηρείται ακόμα και αν το τμήμα δεν υπάρχει πλέον.**

Σχετικά Triggers

Ελέγχει τη λογική συνέπεια μεταξύ των πεδίων outcome και **referred_dept_id** πριν από κάθε INSERT ή UPDATE:

- Αν **outcome** = 'παραπομπή για νοσηλεία' τότε **referred_dept_id** πρέπει να είναι NOT NULL, δεν επιτρέπεται παραπομπή χωρίς προορισμό.
- Αν **outcome** = 'αποχώρησε με οδηγίες' τότε **referred_dept_id** πρέπει να είναι NULL, δεν επιτρέπεται προορισμός αν ο ασθενής αποχωρεί.

Ενότητα Triggers → trg_triage_referral_check

Ενότητα 6: HOSPITALIZATION

Η ενότητα αυτή διαχειρίζεται όλες τις νοσηλείες των ασθενών του νοσοκομείου. Αποτελείται από έναν πίνακα που καλύπτει το πλήρες φάσμα πληροφοριών που απαιτεί η επιχειρησιακή απαίτηση.

Προδιαγραφές Απαιτήσεων Συστήματος

Κάθε **νοσηλεία** αφορά έναν ασθενή, μια κλίνη και ένα τμήμα, έχει ημερομηνία εισαγωγής και εξόδου, διάγνωση εισαγωγής και εξόδου (κωδικός ICD-10 και περιγραφή), κωδικό KEN για τον υπολογισμό κόστους και συνολικό κόστος νοσηλείας. Κάθε νοσηλεία προκύπτει υποχρεωτικά από διαλογή και **κάθε ασθενής μπορεί να έχει πολλαπλές νοσηλείες στο ιστορικό του**.

Πίνακας 6.1: hospitalization

Σκοπός

Ο πίνακας **hospitalization** καταγράφει κάθε νοσηλεία ασθενή στο νοσοκομείο. Κάθε νοσηλεία συνδέεται υποχρεωτικά με μια διαλογή (**triage**), έναν ασθενή, μια κλίνη και ένα τμήμα. Περιλαμβάνει **διάγνωση εισαγωγής και εξόδου** (κωδικός ICD-10 και περιγραφή), τον **κωδικό KEN** για τον υπολογισμό **κόστους νοσηλείας**, καθώς και το συνολικό κόστος που υπολογίζεται αυτόματα από το σύστημα.

Κώδικας SQL

```
CREATE TABLE hospitalization (  
    hospitalization_id SERIAL NOT NULL,  
    patient_id INT NOT NULL,  
    triage_id INT NOT NULL,  
    bed_id INT NOT NULL,  
    department_id INT NOT NULL,  
    admission_date DATE NOT NULL,  
    discharge_date DATE,  
    admission_diag_icd10 VARCHAR(10),  
    admission_diag_desc VARCHAR(255),  
    discharge_diag_icd10 VARCHAR(10),  
    discharge_diag_desc VARCHAR(255),  
    ken_code VARCHAR(10),  
    total_cost DECIMAL(10, 2) CHECK (total_cost >= 0),  
    CONSTRAINT pk_hospitalization PRIMARY KEY (hospitalization_id),  
    CONSTRAINT fk_hosp_patient FOREIGN KEY (patient_id) REFERENCES patient  
(patient_id) ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_hosp_triage FOREIGN KEY (triage_id) REFERENCES triage
```

```
(triage_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_bed FOREIGN KEY (bed_id) REFERENCES bed (bed_id) ON
DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_dept FOREIGN KEY (department_id) REFERENCES department
(department_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_ken FOREIGN KEY (ken_code) REFERENCES ken_code
(ken_code) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_icd10_in FOREIGN KEY (admission_diag_icd10) REFERENCES
icd10_code (icd10_code) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_icd10_out FOREIGN KEY (discharge_diag_icd10) REFERENCES
icd10_code (icd10_code) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_hosp_dates CHECK (
        discharge_date IS NULL
        OR discharge_date >= admission_date
    )
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
hospitalization_id	SERIAL	NOT NULL	PK	ID κάθε νοσηλείας. SERIAL γιατί ένας ασθενής μπορεί να έχει πολλαπλές νοσηλείες στο ιστορικό του.
patient_id	INT	NOT NULL	FK → patient	Σύνδεση με τον ασθενή. NOT NULL γιατί κάθε νοσηλεία αφορά υποχρεωτικά έναν ασθενή.
triage_id	INT	NOT NULL	FK → triage	Σύνδεση με τη διαλογή από την οποία προέκυψε η νοσηλεία. NOT NULL γιατί κάθε ασθενής περνά υποχρεωτικά από διαλογή πριν τη νοσηλεία.
bed_id	INT	NOT NULL	FK → bed	Η κλίνη που ανατίθεται στον ασθενή. NOT NULL γιατί κάθε νοσηλεία απαιτεί ανάθεση κλίνης κατά την εισαγωγή.
department_id	INT	NOT NULL	FK → department	Το τμήμα νοσηλείας. NOT NULL γιατί κάθε νοσηλεία γίνεται υποχρεωτικά σε συγκεκριμένο τμήμα.
admission_date	DATE	NOT NULL		Ημερομηνία εισαγωγής. NOT NULL γιατί κάθε νοσηλεία πρέπει να έχει ημερομηνία έναρξης.
discharge_date	DATE	NULL		Ημερομηνία εξόδου. NULL όσο ο ασθενής νοσηλεύεται ακόμα. Η συμπλήρωσή του σηματοδοτεί την ολοκλήρωση της νοσηλείας και απαιτεί υποχρεωτικά συμπληρωμένη διάγνωση εξόδου.
admission_diag_icd10	VARCHAR(10)	NULL	FK → icd10_code	Κωδικός ICD-10 διάγνωσης εισαγωγής. NULL γιατί μπορεί να μην είναι γνωστός κατά την εισαγωγή.
admission_diag_desc	VARCHAR(255)	NULL		Περιγραφή διάγνωσης εισαγωγής. Ελεύθερο κείμενο που συμπληρώνει τον κωδικό ICD-10.

Στήλη	Τύπος		Constraint	Αιτιολόγηση
discharge_diag_icd10	VARCHAR(10)	NULL	FK → icd10_code	Κωδικός ICD-10 διάγνωσης εξόδου. NULL όσο ο ασθενής νοσηλεύεται ακόμα.
discharge_diag_desc	VARCHAR(255)	NULL		Περιγραφή διάγνωσης εξόδου. NULL όσο ο ασθενής νοσηλεύεται ακόμα.
ken_code	VARCHAR(10)	NULL	FK → ken_code	Κωδικός KEN (Κλειστά Ενοποιημένα Νοσήλια) για τον υπολογισμό κόστους. NULL γιατί μπορεί να οριστεί μετά την εισαγωγή.
total_cost	DECIMAL(10,2)	NULL	CHECK ≥ 0	Συνολικό κόστος νοσηλείας. Υπολογίζεται αυτόματα από trigger βάσει του ken_code και των ημερών παραμονής. CHECK εξασφαλίζει ότι δεν μπορεί να είναι αρνητικό.

Explanations & Constraints

CONSTRAINT fk_hosp_triage ... ON DELETE RESTRICT

Αν μια εγγραφή διαλογής προσπαθήσει να διαγραφεί ενώ υπάρχει νοσηλεία που την αναφέρει, η διαγραφή απαγορεύεται. Η ιχνηλασιμότητα από τη διαλογή στη νοσηλεία πρέπει να διατηρείται.

CONSTRAINT fk_hosp_bed ... ON DELETE RESTRICT

Αν μια κλίνη προσπαθήσει να διαγραφεί ενώ υπάρχει νοσηλεία που την αναφέρει, η διαγραφή απαγορεύεται. Το ιστορικό νοσηλείων δεν πρέπει να χάνεται.

CONSTRAINT fk_hosp_patient ... ON DELETE RESTRICT

Αν ένας ασθενής προσπαθήσει να διαγραφεί ενώ υπάρχει νοσηλεία που τον αναφέρει, η διαγραφή απαγορεύεται. Το ιατρικό ιστορικό νοσηλείων κάθε ασθενή πρέπει να διατηρείται.

CONSTRAINT fk_hosp_ken ... ON DELETE SET NULL

Αν ένας κωδικός KEN διαγραφεί, το ken_code γίνεται NULL αντί να διαγραφεί η νοσηλεία. Το ιστορικό της νοσηλείας διατηρείται ακόμα και αν ο κωδικός δεν υπάρχει πλέον.

CONSTRAINT fk_hosp_icd10_in / fk_hosp_icd10_out ... ON DELETE SET NULL

Αν ένας κωδικός ICD-10 διαγραφεί, τα αντίστοιχα πεδία διάγνωσης γίνονται NULL. Το ιστορικό της νοσηλείας δεν διαγράφεται, μόνο η σύνδεση με τον κωδικό χάνεται.

CONSTRAINT chk_hosp_dates CHECK (discharge_date IS NULL OR discharge_date >= admission_date)

Εξασφαλίζει ότι η ημερομηνία εξόδου δεν μπορεί να είναι πριν από την ημερομηνία εισαγωγής. NULL επιτρέπεται γιατί ο ασθενής μπορεί να νοσηλεύεται ακόμα.

Σχετικά Triggers

Ο πίνακας hospitalization σχετίζεται με τρία triggers που δεν μπορούν να εκφραστούν ως CHECK constraints γιατί απαιτούν έλεγχο σε άλλους πίνακες.

Κατά την εισαγωγή νοσηλείας ελέγχει ότι η κλίνη έχει status **'διαθέσιμη'** και την αλλάζει αυτόματα σε **'κατειλημμένη'**. Όταν συμπληρωθεί το **discharge_date**, το status επιστρέφει αυτόματα σε **'διαθέσιμη'**. **Απαιτεί έλεγχο και ενημέρωση στον πίνακα bed.**

Ενότητα Triggers → **trg_hosp_bed_status**.

Υπολογίζει αυτόματα το **total_cost** κατά τη συμπλήρωση του **discharge_date**, βάσει του **ken_code.base_cost** και των ημερών παραμονής. Αν η παραμονή υπερβαίνει το **avg_duration_days**, προστίθεται **daily_extra** για κάθε επιπλέον ημέρα. **Απαιτεί έλεγχο στον πίνακα ken_code.**

Ενότητα Triggers → **trg_hosp_total_cost**.

Ελέγχει ότι κατά τη συμπλήρωση του **discharge_date** έχουν συμπληρωθεί τα πεδία **discharge_diag_icd10** και **discharge_diag_desc**. Δεν επιτρέπεται έξοδος χωρίς διάγνωση εξόδου.

Ενότητα Triggers → **trg_hosp_discharge_diagnosis**

Ελέγχει ότι το **outcome** του αντίστοιχου **triage** είναι **'παραπομπή για νοσηλεία'** πριν δημιουργηθεί η νοσηλεία. Δεν επιτρέπεται δημιουργία νοσηλείας αν **outcome = 'αποχώρησε με οδηγίες'**. **Απαιτεί έλεγχο στον πίνακα triage.**

Ενότητα Triggers → **trg_hosp_triage_outcome**

Ελέγχει ότι η κλίνη δεν είναι ήδη κατειλημμένη για την ίδια χρονική περίοδο. Αποτρέπει επικάλυψη νοσηλείων στην ίδια κλίνη. **Απαιτεί έλεγχο στον πίνακα hospitalization.**

Ενότητα Triggers → **trg_hosp_bed_overlap**.

Πίνακας 6.3: hospitalization_staff

Στήλη	Τύπος		Constraint	Αιτιολόγηση
hospitalization_id	INT		NOT NULL PK (σύνθετο) FK → hospitalization	Σύνδεση με τη νοσηλεία. NOT NULL γιατί κάθε ανάθεση αφορά υποχρεωτικά μια νοσηλεία. ON DELETE CASCADE: αν διαγραφεί η νοσηλεία, διαγράφονται αυτόματα και οι αναθέσεις προσωπικού.
staff_id	INT		NOT NULL PK (σύνθετο) FK → staff	Σύνδεση με το μέλος προσωπικού. NOT NULL γιατί κάθε ανάθεση αφορά υποχρεωτικά ένα μέλος. ON DELETE RESTRICT: μέλος με αναθέσεις σε νοσηλείες δεν μπορεί να διαγραφεί.
role	VARCHAR (30)		NOT NULL CHECK	Ρόλος στη νοσηλεία. CHECK περιορίζει σε: 'attending_doctor' (κύριος θεράπων), 'consulting_doctor' (σύμβουλος), 'responsible_nurse' (υπεύθυνος νοσηλευτής).

Στήλη	Τύπος		Constraint	Αιτιολόγηση
assigned_date	DATE		NOT NULL DEFAULT CURRENT_DATE	Ημερομηνία ανάθεσης. DEFAULT CURRENT_DATE γιατί στη συντριπτική πλειονότητα η ανάθεση γίνεται την ημέρα εισαγωγής. Αποθηκεύεται για ιστορικό.

Explanations & Constraints

CONSTRAINT pk_hosp_staff PRIMARY KEY (hospitalization_id, staff_id)

Σύνθετο φυσικό κλειδί. Ένα μέλος προσωπικού μπορεί να αναληφθεί σε πολλές νοσηλείες αλλά δεν μπορεί να εμφανιστεί δύο φορές στην ίδια νοσηλεία. Δεν χρειάζεται surrogate key.

CONSTRAINT fk_hs_hospitalization ... ON DELETE CASCADE ON UPDATE CASCADE

Αν διαγραφεί μια νοσηλεία, διαγράφονται αυτόματα και οι αναθέσεις προσωπικού. Η ανάθεση δεν έχει νόημα χωρίς τη νοσηλεία στην οποία αναφέρεται.

CONSTRAINT fk_hs_staff ... ON DELETE RESTRICT ON UPDATE CASCADE

Ένα μέλος προσωπικού δεν μπορεί να διαγραφεί αν υπάρχουν αναθέσεις σε νοσηλείες. Το ιστορικό φροντίδας πρέπει να διατηρείται.

Επιλογή FK → staff αντί για doctor / nurse

Η σύνδεση στον πίνακα staff (supertype) αντί για doctor ή nurse ξεχωριστά επιτρέπει ένα ενιαίο constraint για όλους τους ρόλους. Η επιχειρησιακή επικύρωση ανήκει στο application layer.

Σχετικά Triggers

Ενότητα Triggers → trg_prescription_doctor_guard

Ο πίνακας hospitalization_staff είναι η βάση για το trigger ελέγχου συνταγογράφησης:

Ενότητα 7: PRESCRIPTIONS

Η ενότητα αυτή διαχειρίζεται τη φαρμακευτική αγωγή των ασθενών κατά τη νοσηλεία τους. Ο πίνακας **prescription** αποτελεί το κεντρικό σημείο καταγραφής κάθε συνταγογράφησης, συνδέοντας ιατρό, ασθενή, φάρμακο και νοσηλεία σε μία ενιαία εγγραφή

Προδιαγραφές Απαιτήσεων Συστήματος

Κάθε συνταγογράφηση αφορά έναν γιατρό, έναν ασθενή και ένα φάρμακο, γίνεται αποκλειστικά κατά τη νοσηλεία και περιλαμβάνει δοσολογία, συχνότητα χορήγησης και ημερομηνία έναρξης και λήξης. Κάθε συνδυασμός (ιατρός, ασθενής, φάρμακο, ημερομηνία έναρξης) είναι μοναδικός. Το νοσοκομείο υποχρεούται να συνταγογραφεί αποκλειστικά φάρμακα από τη βάση δεδομένων EMA Article 57. Απαγορεύεται η συνταγογράφηση φαρμάκου αν κάποια από τις δραστικές ουσίες του βρίσκεται στις καταγεγραμμένες αλλεργίες του ασθενή.

Πίνακας 7.1: prescription

Σκοπός

Ο πίνακας **prescription** καταγράφει κάθε συνταγογράφηση φαρμάκου κατά τη νοσηλεία ασθενή. Κάθε συνταγογράφηση αφορά έναν γιατρό, έναν ασθενή, ένα φάρμακο και μια νοσηλεία. Πριν από κάθε συνταγογράφηση ελέγχεται αυτόματα αν κάποια δραστική ουσία του φαρμάκου υπάρχει στις αλλεργίες του ασθενή, αν ναι, η συνταγογράφηση απαγορεύεται.

Κώδικας SQL

```
CREATE TABLE prescription (
    prescription_id SERIAL NOT NULL,
    doctor_id INT NOT NULL,
    patient_id INT NOT NULL,
    drug_code VARCHAR(30) NOT NULL,
    hospitalization_id INT NOT NULL,
    dosage VARCHAR(100) NOT NULL,
    frequency VARCHAR(100) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    CONSTRAINT pk_prescription PRIMARY KEY (prescription_id),
    CONSTRAINT uq_prescription UNIQUE (doctor_id, patient_id, drug_code,
start_date),
    CONSTRAINT chk_pr_dates CHECK (end_date >= start_date),
    CONSTRAINT fk_pr_doctor FOREIGN KEY (doctor_id)
        REFERENCES doctor (staff_id) ON DELETE RESTRICT ON UPDATE CASCADE,
```

```

CONSTRAINT fk_pr_patient FOREIGN KEY (patient_id)
    REFERENCES patient (patient_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_pr_drug FOREIGN KEY (drug_code)
    REFERENCES drug (drug_code) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_pr_hosp FOREIGN KEY (hospitalization_id)
    REFERENCES hospitalization (hospitalization_id) ON DELETE RESTRICT ON
UPDATE CASCADE
);

```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
prescription_id	SERIAL	NOT NULL	PK	ID κάθε συνταγογράφησης. SERIAL γιατί ένας ασθενής μπορεί να έχει πολλαπλές συνταγογραφήσεις.
doctor_id	INT	NOT NULL	FK → doctor	Ο ιατρός που συνταγογραφεί. NOT NULL γιατί κάθε συνταγογράφιση απαιτεί υποχρεωτικά ιατρό.
patient_id	INT	NOT NULL	FK → patient_id	Ο ασθενής που λαμβάνει το φάρμακο. NOT NULL γιατί κάθε συνταγογράφιση αφορά υποχρεωτικά έναν ασθενή.
drug_code	VARCHAR(30)	NOT NULL	FK → drug	Το φάρμακο που συνταγογραφείται. NOT NULL γιατί δεν έχει νόημα συνταγογράφιση χωρίς φάρμακο.
hospitalization_id	INT	NOT NULL	FK → hospitalization	Η νοσηλεία κατά την οποία γίνεται η συνταγογράφιση. NOT NULL γιατί η εκφώνηση ορίζει ότι η συνταγογράφιση γίνεται αποκλειστικά κατά τη νοσηλεία.
dosage	VARCHAR(100)	NOT NULL	FK → hospitalization	Δοσολογία. NOT NULL γιατί δεν έχει νόημα συνταγογράφιση χωρίς δοσολογία.
frequency	VARCHAR(100)	NOT NULL		Συχνότητα χορήγησης. NOT NULL γιατί δεν έχει νόημα συνταγογράφιση χωρίς συχνότητα.
start_date	VARCHAR(10)	NOT NULL		Ημερομηνία έναρξης αγωγής. NOT NULL γιατί κάθε συνταγογράφιση πρέπει να έχει ημερομηνία έναρξης.
end_date	VARCHAR(255)	NOT NULL	CHECK ≥ start_date	Ημερομηνία λήξης αγωγής. NOT NULL γιατί κάθε νοσοκομειακή αγωγή έχει συγκεκριμένη διάρκεια. Δεν υπάρχει ανοιχτή συνταγογράφιση.

Explanations & Constraints

CONSTRAINT uq_prescription UNIQUE (doctor_id, patient_id, drug_code, start_date)

Εξασφαλίζει ότι κάθε συνδυασμός (ιατρός, ασθενής, φάρμακο, ημερομηνία έναρξης) είναι μοναδικός. Ο ίδιος γιατρός μπορεί να συνταγογραφήσει το ίδιο φάρμακο στον ίδιο ασθενή σε διαφορετική ημερομηνία.

CONSTRAINT chk_pr_dates CHECK (end_date >= start_date)

Εξασφαλίζει ότι η ημερομηνία λήξης δεν μπορεί να είναι πριν από την ημερομηνία έναρξης. Και τα δύο πεδία είναι NOT NULL, κάθε αγωγή έχει υποχρεωτικά ημερομηνία έναρξης και λήξης γιατί η συνταγογράφηση γίνεται αποκλειστικά στο πλαίσιο νοσηλείας με συγκεκριμένη διάρκεια.

Όλα τα FKs με ON DELETE RESTRICT

Κανένα στοιχείο που αναφέρεται σε συνταγογράφηση δεν μπορεί να διαγραφεί, ούτε ο ιατρός, ούτε ο ασθενής, ούτε το φάρμακο, ούτε η νοσηλεία. Το ιστορικό συνταγογραφήσεων πρέπει να διατηρείται πάντα.

hospitalization_id NOT NULL

Η εκφώνηση ορίζει ρητά ότι η φαρμακευτική αγωγή δίνεται κατά τη νοσηλεία, δεν υπάρχει εξωτερική συνταγογράφηση στο σύστημα αυτό.

Σχετικά Triggers

Ο πίνακας **prescription** σχετίζεται με ένα trigger που δεν μπορεί να εκφραστεί ως CHECK constraint γιατί απαιτεί έλεγχο σε άλλους πίνακες:

Ενεργοποιείται πριν από κάθε INSERT. Διαβάζει από τον **drug_substance** τις δραστικές ουσίες του φαρμάκου και τις συγκρίνει με τις αλλεργίες του ασθενή στον **patient_allergy**. Αν υπάρχει κοινή ουσία, η συνταγογράφηση απαγορεύεται αυτόματα με **RAISE EXCEPTION**.

Ενότητα Triggers → **trg_allergy_check**

Ενότητα 8: LAB & PROCEDURES

Η ενότητα αυτή διαχειρίζεται τις ιατρικές πράξεις/επεμβάσεις και τις εργαστηριακές εξετάσεις που πραγματοποιούνται στο πλαίσιο νοσηλείας. Αποτελείται από έξι πίνακες που μαζί καλύπτουν το πλήρες φάσμα της χειρουργικής και εργαστηριακής δραστηριότητας του νοσοκομείου.

Προδιαγραφές Απαιτήσεων Συστήματος

Κατά τη νοσηλεία, ένας ασθενής μπορεί να υποβληθεί σε ιατρικές πράξεις/επεμβάσεις (χειρουργεία ή άλλες επεμβατικές πράξεις) και σε εργαστηριακές εξετάσεις. Κάθε πράξη εκτελείται από ένα κύριο ιατρό και μπορεί να συμμετέχουν και άλλα μέλη προσωπικού ως βοηθοί. Κάθε εργαστηριακή εξέταση παραγγέλλεται από ιατρό και έχει ημερομηνία παραγγελίας, ημερομηνία αποτελέσματος και αριθμητική ή κειμενική τιμή αποτελέσματος. Τόσο οι πράξεις όσο και οι εξετάσεις πρέπει να γίνονται εντός του χρονικού πλαισίου της νοσηλείας (μεταξύ `admission_date` και `discharge_date`).

Πίνακας 8.1: `medical_procedure`

Σκοπός

Ο πίνακας `medical_procedure` είναι **lookup table** που αποθηκεύει τους κωδικούς και τις περιγραφές των ιατρικών πράξεων/επεμβάσεων που μπορούν να διενεργηθούν στο νοσοκομείο. Χρησιμοποιείται ως FK στον `hospitalization_procedure` για να εξασφαλιστεί ότι κάθε εκτελεσμένη πράξη αντιστοιχεί σε αναγνωρισμένη και καταχωρημένη ιατρική πράξη.

Κώδικας SQL

```
CREATE TABLE medical_procedure (
  procedure_code VARCHAR(20) NOT NULL,
  description     TEXT          NOT NULL,
  category        VARCHAR(50),
  duration_minutes INT,
  CONSTRAINT pk_procedure PRIMARY KEY (procedure_code)
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
<code>procedure_code</code>	VARCHAR(20)		NOT NULL PK	Κωδικός ιατρικής πράξης. Φυσικό PK — εξωτερικά ορισμένος, μοναδικός και σταθερός.
<code>description</code>	TEXT		NOT NULL	Πλήρης περιγραφή της πράξης. TEXT γιατί μπορεί να είναι αρκετά μακρά. NOT NULL γιατί κάθε πράξη πρέπει να έχει περιγραφή.
<code>category</code>	VARCHAR(50)		NULL	Κατηγορία πράξης (π.χ. 'Χειρουργική', 'Ενδοσκοπική'). NULL γιατί είναι προαιρετική ταξινόμηση.

Explanations

procedure_code ως φυσικό PK

Ο κωδικός πράξης χρησιμοποιείται απευθείας ως PK αντί για surrogate key (SERIAL) γιατί είναι εξωτερικά ορισμένος, μοναδικός και σταθερός. Επιτρέπει αναγνώσιμο FK στον hospitalization_procedure.

Πίνακας 8.2: lab_exam**Σκοπός**

Ο πίνακας lab_exam είναι lookup table που αποθηκεύει τους κωδικούς και τις περιγραφές των εργαστηριακών εξετάσεων που μπορούν να παραγγελθούν κατά τη νοσηλεία. Χρησιμοποιείται ως FK στον hospitalization_lab.

Κώδικας SQL

```
CREATE TABLE lab_exam (
  lab_code      VARCHAR(20) NOT NULL,
  description   TEXT        NOT NULL,
  CONSTRAINT pk_lab_exam PRIMARY KEY (lab_code)
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
lab_code	VARCHAR(20)	NOT NULL	PK	Κωδικός εργαστηριακής εξέτασης. Φυσικό PK, εξωτερικά ορισμένος, μοναδικός και σταθερός.
description	TEXT	NOT NULL		Περιγραφή εξέτασης. NOT NULL γιατί κάθε κωδικός πρέπει να έχει περιγραφή.

Πίνακας 8.3: operating_room**Σκοπός**

Ο πίνακας operating_room αποθηκεύει τα χειρουργεία και τις αίθουσες επεμβάσεων του νοσοκομείου. Κάθε αίθουσα έχει τύπο, κτίριο και κατάσταση (διαθέσιμη/κατειλημμένη/υπό συντήρηση). Χρησιμοποιείται από τον πίνακα image για τη σύνδεση εικόνων αιθουσών.

Κώδικας SQL

```
CREATE TABLE operating_room (
  room_id SERIAL NOT NULL,
  name VARCHAR(100) NOT NULL,
  type VARCHAR(30) NOT NULL CHECK (type IN ('Χειρουργείο', 'Αίθουσα
επέμβασης')),
  building VARCHAR(50) NOT NULL,
  category VARCHAR(50) NOT NULL CHECK (category IN ('Διαθέσιμη',
```

```
'Κατελημμένη', 'Υπό Συντήρηση')),  
    CONSTRAINT pk_operating_room PRIMARY KEY (room_id)  
);
```

Ανάλυση Στηλών

Στήλη	Τύπος		Constraint	Αιτιολόγηση
room_id	SERIAL	NOT NULL	PK	ID αίθουσας. SERIAL για αυτόματη αύξηση.
name	VARCHAR(100)	NOT NULL		Όνομα αίθουσας. NOT NULL γιατί κάθε αίθουσα πρέπει να έχει αναγνωρίσιμο όνομα.
type	VARCHAR(30)	NOT NULL	CHECK	Τύπος αίθουσας. CHECK περιορίζει σε: 'Χειρουργείο', 'Αίθουσα επέμβασης'.
building	VARCHAR(50)	NOT NULL		Κτίριο/πτέρυγα όπου βρίσκεται η αίθουσα. NOT NULL για γεωγραφική εντόπιση.
category	VARCHAR(50)	NOT NULL	CHECK	Κατάσταση αίθουσας. CHECK περιορίζει σε: 'Διαθέσιμη', 'Κατελημμένη', 'Υπό Συντήρηση'.

Πίνακας 8.4: hospitalization_procedure

Σκοπός

Ο πίνακας hospitalization_procedure καταγράφει κάθε ιατρική πράξη που εκτελείται κατά τη νοσηλεία ενός ασθενή. Κάθε εγγραφή αντιστοιχεί σε μια συγκεκριμένη εκτέλεση πράξης (procedure_code) στο πλαίσιο μιας νοσηλείας (hospitalization_id), από έναν κύριο ιατρό (performed_by), σε συγκεκριμένη ημερομηνία (performed_date). Το trigger trg_procedure_during_hosp εξασφαλίζει ότι η πράξη γίνεται εντός του χρονικού πλαισίου της νοσηλείας.

Κώδικας SQL

```
CREATE TABLE IF NOT EXISTS hospitalization_procedure (  
    hosp_procedure_id SERIAL NOT NULL,  
    hospitalization_id INT NOT NULL,  
    procedure_code VARCHAR(20) NOT NULL,  
    performed_date DATE NOT NULL,  
    performed_by INT NOT NULL,  
    notes TEXT,  
    room INT,  
    start_time TIMESTAMP,  
    duration_minutes INT CHECK (duration_minutes > 0),  
  
    CONSTRAINT pk_hosp_proc PRIMARY KEY (hosp_procedure_id),
```



```

    CONSTRAINT fk_hp_hosp FOREIGN KEY (hospitalization_id) REFERENCES
hospitalization (hospitalization_id),
    CONSTRAINT fk_hp_proc FOREIGN KEY (procedure_code) REFERENCES
medical_procedure (procedure_code),
    CONSTRAINT fk_hp_doc FOREIGN KEY (performed_by) REFERENCES doctor
(staff_id)
);

```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
hosp_procedure_id	SERIAL	NOT NULL	PK	ID κάθε εκτελεσμένης πράξης. SERIAL γιατί μια νοσηλεία μπορεί να έχει πολλαπλές πράξεις.
hospitalization_id	INT	NOT NULL	FK → hospitalization	Σύνδεση με τη νοσηλεία. NOT NULL γιατί κάθε πράξη γίνεται υποχρεωτικά στο πλαίσιο νοσηλείας.
procedure_code	VARCHAR(20)	NOT NULL	FK → medical_procedure	Ο κωδικός της πράξης. NOT NULL γιατί δεν έχει νόημα εγγραφή χωρίς κωδικό.
performed_date	DATE	NOT NULL		Ημερομηνία εκτέλεσης της πράξης. NOT NULL, ελέγχεται από trigger ότι είναι εντός του χρονικού πλαισίου της νοσηλείας.
performed_by	INT	NOT NULL	FK → doctor	Ο κύριος εκτελών ιατρός. NOT NULL γιατί κάθε πράξη απαιτεί υπεύθυνο ιατρό.
notes	TEXT	NULL	NULL	Προαιρετικές σημειώσεις πράξης. TEXT γιατί δεν υπάρχει προκαθορισμένο μήκος.

Πίνακας 8.5: procedure_participant

Σκοπός

Ο πίνακας procedure_participant είναι junction table που υλοποιεί τη σχέση many-to-many μεταξύ ιατρικών πράξεων και μελών προσωπικού που συμμετέχουν ως βοηθοί. Ένας χειρουργός (performed_by) εκτελεί την πράξη, αλλά μπορεί να συμμετέχουν και πρόσθετα μέλη (αναισθησιολόγος, βοηθός χειρουργού, νοσηλευτής χειρουργείου) που καταγράφονται εδώ με τον ρόλο τους.

Κώδικας SQL

```

CREATE TABLE procedure_participant (
    hosp_procedure_id INT NOT NULL,
    staff_id INT NOT NULL,
    role VARCHAR(100),

```

```

    CONSTRAINT pk_procedure_participant PRIMARY KEY (hosp_procedure_id,
staff_id),
    CONSTRAINT fk_pp_procedure FOREIGN KEY (hosp_procedure_id) REFERENCES
hospitalization_procedure (hosp_procedure_id) ON DELETE CASCADE ON UPDATE
CASCADE,
    CONSTRAINT fk_pp_staff FOREIGN KEY (staff_id) REFERENCES staff (staff_id)
ON DELETE RESTRICT ON UPDATE CASCADE
);

```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
hosp_procedure_id	INT		NOT NULL PK (σύνθετο) FK → hospitalization_procedure	Σύνδεση με την πράξη. ON DELETE CASCADE: αν διαγραφεί η πράξη, αφαιρούνται αυτόματα οι συμμετέχοντες.
staff_id	INT		NOT NULL PK (σύνθετο) FK → staff	Το μέλος προσωπικού που συμμετέχει. ON DELETE RESTRICT: δεν μπορεί να διαγραφεί μέλος με συμμετοχές.
role	VARCHAR(100)		NULL	Ρόλος στην πράξη (π.χ. 'Βοηθός Χειρουργού', 'Αναισθησιολόγος'). NULL γιατί δεν είναι πάντα απαραίτητος ο ορισμός ρόλου.

Explanations

Σύνθετο PK (hosp_procedure_id, staff_id)

Εξασφαλίζει ότι το ίδιο μέλος δεν μπορεί να εμφανιστεί δύο φορές στην ίδια πράξη. Δεν απαιτείται surrogate key γιατί ο συνδυασμός είναι από μόνος του μοναδικός.

FK → staff αντί για doctor

Η σύνδεση γίνεται στον supertype staff και όχι αποκλειστικά στον doctor, γιατί μπορεί να συμμετέχουν και νοσηλευτές (π.χ. νοσηλευτής χειρουργείου) σε μια επέμβαση.

Πίνακας 8.6: hospitalization_lab

Σκοπός

Ο πίνακας hospitalization_lab καταγράφει κάθε εργαστηριακή εξέταση που παραγγέλλεται κατά τη νοσηλεία ενός ασθενή. Κάθε εγγραφή αντιστοιχεί σε μια παραγγελία εξέτασης (lab_code) από ιατρό (ordered_by) στο πλαίσιο νοσηλείας (hospitalization_id). Το αποτέλεσμα (result_value, result_date, result_unit) συμπληρώνεται αργότερα όταν είναι έτοιμο.

Κώδικας SQL

```

CREATE TABLE IF NOT EXISTS hospitalization_lab (

```

```

hosp_lab_id      SERIAL      NOT NULL,
hospitalization_id INT      NOT NULL,
lab_code         VARCHAR(20) NOT NULL,
ordered_date     DATE       NOT NULL,
result_date      DATE,
result_value     TEXT,
result_unit      VARCHAR(30),
ordered_by       INT        NOT NULL,
CONSTRAINT pk_hosp_lab PRIMARY KEY (hosp_lab_id),
CONSTRAINT fk_hl_hosp FOREIGN KEY (hospitalization_id) REFERENCES
hospitalization (hospitalization_id),
CONSTRAINT fk_hl_lab FOREIGN KEY (lab_code) REFERENCES lab_exam
(lab_code),
CONSTRAINT fk_hl_doc FOREIGN KEY (ordered_by) REFERENCES doctor
(staff_id)
);

```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
hosp_lab_id	SERIAL	NOT NULL	PK	ID κάθε παραγγελθείσας εξέτασης. SERIAL γιατί μια νοσηλεία μπορεί να έχει πολλαπλές εξετάσεις.
hospitalization_id	INT	NOT NULL	FK → hospitalization	Σύνδεση με τη νοσηλεία. NOT NULL γιατί κάθε εξέταση παραγγέλλεται στο πλαίσιο νοσηλείας.
lab_code	VARCHAR(20)	NOT NULL	FK → lab_exam	Κωδικός εξέτασης. NOT NULL γιατί δεν έχει νόημα παραγγελία χωρίς κωδικό.
ordered_date	DATE	NOT NULL		Ημερομηνία παραγγελίας. NOT NULL — ελέγχεται από trigger ότι είναι εντός του χρονικού πλαισίου της νοσηλείας.
result_date	DATE	NULL		Ημερομηνία αποτελέσματος. NULL αρχικά — συμπληρώνεται όταν είναι έτοιμο το αποτέλεσμα.
result_value	TEXT	NULL		Τιμή αποτελέσματος (αριθμητική ή κειμενική). TEXT γιατί τα αποτελέσματα μπορεί να έχουν ποικίλη μορφή.
result_unit	VARCHAR(30)	NULL		Μονάδα μέτρησης (π.χ. 'mg/dL', 'U/L'). NULL γιατί δεν έχουν πάντα μονάδα.
ordered_by	INT	NOT NULL	FK → doctor	Ο ιατρός που παρήγγειλε την εξέταση. NOT NULL γιατί κάθε εξέταση απαιτεί υπεύθυνο ιατρό.

Σχετικά Triggers

trg_procedure_during_hosp(BEFORE INSERT OR UPDATE ON hospitalization_procedure

Ελέγχει ότι η `performed_date` εμπίπτει εντός του χρονικού πλαισίου της νοσηλείας (μεταξύ `admission_date` και `discharge_date`). Απαγορεύει πράξεις πριν την εισαγωγή ή μετά την έξοδο. Δεν μπορεί να εκφραστεί ως CHECK constraint γιατί απαιτεί έλεγχο σε άλλον πίνακα (`hospitalization`).

`trg_lab_exam_during_hosp(BEFORE INSERT OR UPDATE ON hospitalization_lab)`

Ίδια λογική με `trg_procedure_during_hosp` — ελέγχει ότι η `ordered_date` εμπίπτει εντός της νοσηλείας. Απαγορεύει παραγγελία εξέτασης εκτός του χρονικού πλαισίου.

Ενότητα 9: SHIFT SCHEDULING

Η ενότητα αυτή διαχειρίζεται το σύστημα προγραμματισμού βάρδιών (εφημεριών) του νοσοκομείου. Αποτελείται από τρεις πίνακες και ένα view που μαζί καλύπτουν τον ορισμό βαρδιών, την ανάθεση προσωπικού ανά βάρδια και τον έλεγχο ορίων. Η ενότητα ενισχύεται από έξι triggers που επιβάλλουν αυτόματα όλους τους λειτουργικούς κανόνες.

Προδιαγραφές Απαιτήσεων Συστήματος

Όλα τα τμήματα του νοσοκομείου βρίσκονται σε κατάσταση εφημερίας καθημερινά (24 ώρες). Η λειτουργία τους οργανώνεται σε τρεις οκτάωρες βάρδιες: Πρωινή (07:00-15:00), Απογευματινή (15:00-23:00) και Νυχτερινή (23:00-07:00). Κάθε βάρδια πρέπει να καλύπτεται από τουλάχιστον 3 ιατρούς, 6 νοσηλευτές και 2 διοικητικούς. Αν συμμετέχει Ειδικευόμενος, πρέπει υποχρεωτικά να είναι παρών Επιμελητής Α ή Διευθυντής. Μέγιστα όρια βαρδιών ανά μήνα: Ιατροί 15, Νοσηλευτές 20, Διοικητικό Προσωπικό 25. Μεταξύ δύο διαδοχικών βαρδιών πρέπει να μεσολαβεί τουλάχιστον 8 ώρες ανάπαυσης. Κανένα μέλος δεν μπορεί να συμμετέχει σε περισσότερες από 3 συνεχόμενες νυχτερινές βάρδιες.

Πίνακας 9.1: shifts

Σκοπός

Ο πίνακας shifts αποθηκεύει τις βάρδιες κάθε τμήματος. Κάθε βάρδια ορίζεται από τον συνδυασμό τμήματος (department_name), ημερομηνίας (shift_date) και τύπου (shift_type). Το UNIQUE constraint (department_name, shift_date, shift_type) εξασφαλίζει ότι δεν μπορούν να υπάρξουν δύο ίδιες βάρδιες για το ίδιο τμήμα και ημερομηνία. Το προσωπικό ανατίθενται στις βάρδιες μέσω του junction table shift_staff.

Κώδικας SQL

```
CREATE TABLE shifts (
    shift_id SERIAL NOT NULL,
    department_name VARCHAR(100) NOT NULL,
    shift_date DATE NOT NULL,
    shift_type VARCHAR(60) NOT NULL CHECK (shift_type IN ('Πρωινή Βάρδια (07:00-15:00)', 'Απογευματινή Βάρδια (15:00-23:00)', 'Νυχτερινή Βάρδια (23:00-07:00)')),
    CONSTRAINT pk_shift PRIMARY KEY (shift_id),
    CONSTRAINT uq_shift UNIQUE (department_name, shift_date, shift_type),
    CONSTRAINT fk_shift_dept FOREIGN KEY (department_name) REFERENCES
department (department_name) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
shift_id	SERIAL	NOT NULL	PK	ID κάθε βάρδιας. SERIAL για αυτόματη αύξηση.
department_name	VARCHAR(100)	NOT NULL	FK → department	Το τμήμα στο οποίο ανήκει η βάρδια. NOT NULL γιατί κάθε βάρδια αφορά υποχρεωτικά ένα τμήμα. ON DELETE RESTRICT: το τμήμα δεν μπορεί να διαγραφεί αν έχει βαρδιές.
shift_date	DATE	NOT NULL		Ημερομηνία βάρδιας. NOT NULL γιατί κάθε βάρδια έχει συγκεκριμένη ημερομηνία.
shift_type	VARCHAR(60)	NOT NULL	CHECK	Τύπος βάρδιας. CHECK περιορίζει σε: 'Πρωινή Βάρδια (07:00-15:00)', 'Απογευματινή Βάρδια (15:00-23:00)', 'Νυχτερινή Βάρδια (23:00-07:00)'. UNIQUE (department_name, shift_date, shift_type), δεν μπορούν να υπάρχουν δύο ίδιες βάρδιες για το ίδιο τμήμα και ημερομηνία.

Explanations

UNIQUE (department_name, shift_date, shift_type)

Εξασφαλίζει ότι κάθε τμήμα μπορεί να έχει το πολύ μία βάρδια κάθε τύπου ανά ημέρα — δεν μπορούν να υπάρξουν δύο Πρωινές Βάρδιες για την Καρδιολογία την ίδια μέρα.

ON DELETE RESTRICT στο fk_shift_dept

Αν ένα τμήμα έχει καταχωρημένες βάρδιες, δεν μπορεί να διαγραφεί. Το ιστορικό βαρδιών πρέπει να διατηρείται.

Πίνακας 9.2: shift_staff

Σκοπός

Ο πίνακας shift_staff είναι το junction table που υλοποιεί τη σχέση many-to-many μεταξύ βαρδιών και μελών προσωπικού. Κάθε εγγραφή εκφράζει ότι ένα συγκεκριμένο μέλος προσωπικού συμμετέχει σε μια συγκεκριμένη βάρδια. Το σύνθετο PK (shift_id, staff_id) εξασφαλίζει ότι το ίδιο μέλος δεν μπορεί να εμφανιστεί δύο φορές στην ίδια βάρδια.

Κώδικας SQL

```
CREATE TABLE shift_staff (
    shift_id INT NOT NULL,
    staff_id INT NOT NULL,
    CONSTRAINT pk_shift_staff PRIMARY KEY (shift_id, staff_id),
    CONSTRAINT fk_ss_shift FOREIGN KEY (shift_id) REFERENCES shifts (shift_id)
ON DELETE CASCADE ON UPDATE CASCADE,
```

```
CONSTRAINT fk_ss_staff FOREIGN KEY (staff_id) REFERENCES staff (staff_id)
ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
shift_id	INT	NOT NULL	PK, FK → shifts	Σύνδεση με τη βάρδια. ON DELETE CASCADE: αν διαγραφεί η βάρδια, αφαιρούνται αυτόματα οι αναθέσεις προσωπικού.
staff_id	INT	NOT NULL	PK, FK → staff	Το μέλος προσωπικού. ON DELETE RESTRICT: μέλος με αναθέσεις σε βάρδιες δεν μπορεί να διαγραφεί. Το ίδιο μέλος δεν μπορεί να εμφανιστεί δύο φορές στην ίδια βάρδια (PK).

Explanations

ON DELETE CASCADE στο fk_ss_shift

Αν διαγραφεί μια βάρδια, διαγράφονται αυτόματα και όλες οι αναθέσεις προσωπικού σε αυτή. Η ανάθεση δεν έχει νόημα χωρίς τη βάρδια στην οποία αναφέρεται.

ON DELETE RESTRICT στο fk_ss_staff

Ένα μέλος προσωπικού δεν μπορεί να διαγραφεί αν έχει αναθέσεις σε βάρδιες. Το ιστορικό εφημεριών πρέπει να διατηρείται.

Σχετικά Triggers

Η ενότητα 9 διαθέτει 6 triggers που επιβάλλουν αυτόματα όλους τους λειτουργικούς κανόνες του προγραμματισμού βαρδιών. Κανένας από αυτούς δεν μπορεί να εκφραστεί ως CHECK constraint γιατί απαιτούν έλεγχο σε άλλους πίνακες.

trg_shift_staff_belongs_to_department(BEFORE INSERT ON shift_staff)

Ελέγχει ότι κάθε μέλος ανήκει στο τμήμα της βάρδιας: ιατροί μέσω doctor_department, νοσηλευτές μέσω nurse.department_name, διοικητικοί μέσω admin_staff.department_name. Απαγορεύει ανάθεση μέλους σε βάρδια τμήματος στο οποίο δεν ανήκει.

trg_shift_min_staff(AFTER INSERT OR DELETE ON shift_staff (DEFERRED))

Ελέγχει ότι κάθε βάρδια έχει τουλάχιστον 3 ιατρούς, 6 νοσηλευτές και 2 διοικητικούς. DEFERRABLE INITIALLY DEFERRED ώστε να εκτελείται μετά την ολοκλήρωση όλων των εισαγωγών της βάρδιας και όχι με κάθε μεμονωμένη εισαγωγή.

trg_shift_resident_needs_senior(AFTER INSERT OR DELETE ON shift_staff (DEFERRED))

Αν υπάρχει Ειδικευόμενος στη βάρδια, ελέγχει ότι υπάρχει τουλάχιστον ένας Επιμελητής Α ή Διευθυντής. DEFERRABLE INITIALLY DEFERRED για τον ίδιο λόγο με το trg_shift_min_staff.

trg_shift_monthly_limit(BEFORE INSERT ON shift_staff)

Ελέγχει ότι το μέλος δεν υπερβαίνει το μηνιαίο όριο βαρδιών (Ιατροί: 15, Νοσηλευτές: 20, Διοικητικοί: 25). Μετράει βαρδιές του ίδιου μήνα/έτους πριν την εισαγωγή.

trg_shift_rest_8h(BEFORE INSERT ON shift_staff)

Ελέγχει ότι μεταξύ δύο διαδοχικών βαρδιών του ίδιου μέλους μεσολαβούν τουλάχιστον 8 ώρες ανάπαυσης. Ελέγχει τόσο τη βάρδια που προηγείται όσο και αυτή που έπεται της νέας.

trg_shift_max_3_night(BEFORE INSERT ON shift_staff)

Ελέγχει ότι το μέλος δεν εκτελεί περισσότερες από 3 συνεχόμενες νυχτερινές βάρδιες. Ανιχνεύει αναδρομικά τις προηγούμενες νυχτερινές βάρδιες του μέλους.

Ενότητα 10: REVIEWS

Η ενότητα αυτή διαχειρίζεται τις αξιολογήσεις των ασθενών μετά την ολοκλήρωση της νοσηλείας τους. Αποτελείται από δύο πίνακες που μαζί καλύπτουν τα δύο επίπεδα αξιολόγησης που απαιτεί η επιχειρησιακή απαίτηση: την αξιολόγηση της νοσηλείας συνολικά και την αξιολόγηση κάθε ιατρού που συνταγογράφησε κατά τη νοσηλεία.

Προδιαγραφές Απαιτήσεων Συστήματος

Οι ασθενείς, με ολοκληρωμένη νοσηλεία, μπορούν να αξιολογούν την εμπειρία τους μετά την έξοδό τους. Η αξιολόγηση γίνεται με κλίμακα Likert σε δύο επίπεδα: (α) Αξιολόγηση νοσηλείας στα εξής κριτήρια: Ποιότητα νοσηλευτικής φροντίδας, Καθαριότητα, Φαγητό, Συνολική εμπειρία και (β) Αξιολόγηση ιατρού στο κριτήριο: Ποιότητα ιατρικής φροντίδας. Ο ασθενής μπορεί να αξιολογήσει κάθε ιατρό που συνταγογραφεί κατά τη νοσηλεία του. Αξιολόγηση επιτρέπεται μόνο σε ασθενείς με ολοκληρωμένη νοσηλεία.

Πίνακας 10.1: patient_hospitalization_review

Σκοπός

Ο πίνακας `patient_hospitalization_review` αποθηκεύει την αξιολόγηση κάθε νοσηλείας από τον ασθενή μετά την έξοδό του. Κάθε νοσηλεία μπορεί να έχει **το πολύ μία** αξιολόγηση (UNIQUE constraint). Η αξιολόγηση γίνεται με κλίμακα **Likert 1-5** σε τέσσερις άξονες: **ιατρική φροντίδα, νοσηλευτική φροντίδα, καθαριότητα και φαγητό**, καθώς και **συνολική βαθμολογία**.

Κώδικας SQL

```
CREATE TABLE patient_hospitalization_review (
    hospitalization_review_id SERIAL NOT NULL,
    hospitalization_id INT NOT NULL,
    nursing_care_score INT NOT NULL CHECK (nursing_care_score BETWEEN 1 AND 5),
    cleanliness_score INT NOT NULL CHECK (cleanliness_score BETWEEN 1 AND 5),
    food_score INT NOT NULL CHECK (food_score BETWEEN 1 AND 5),
    overall_score INT NOT NULL CHECK (overall_score BETWEEN 1 AND 5),
    CONSTRAINT pk_phr PRIMARY KEY (hospitalization_review_id),
    CONSTRAINT uq_phr_hosp UNIQUE (hospitalization_id),
    CONSTRAINT fk_phr_hosp FOREIGN KEY (hospitalization_id) REFERENCES
hospitalization (hospitalization_id) ON DELETE CASCADE ON UPDATE CASCADE
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
hospitalization_review_id	SERIAL	NOT NULL	PK	ID κάθε αξιολόγηση νοσηλείας. SERIAL για αυτόματη αύξηση.
hospitalization_id	INT	NOT NULL	FK → hospitalization, UNIQUE	Σύνδεση με τη νοσηλεία. NOT NULL γιατί κάθε αξιολόγηση αφορά υποχρεωτικά μια νοσηλεία. UNIQUE γιατί επιτρέπεται μία μόνο αξιολόγηση ανά νοσηλεία.
nursing_care_score	INT	NOT NULL	CHECK (1-5)	Βαθμολογία ποιότητας νοσηλευτικής φροντίδας. Κλίμακα Likert 1-5.
cleanliness_score	INT	NOT NULL	CHECK (1-5)	Βαθμολογία καθαριότητας. Κλίμακα Likert 1-5.
food_score	INT	NOT NULL	CHECK (1-5)	Βαθμολογία φαγητού. Κλίμακα Likert 1-5.
overall_score	INT	NOT NULL	CHECK (1-5)	Συνολική εμπειρία νοσηλείας. Κλίμακα Likert 1-5. Συμπληρώνεται ξεχωριστά από τον ασθενή, δεν υπολογίζεται αυτόματα από τις επιμέρους βαθμολογίες.

Explanations & Constraints

CONSTRAINT uq_hr_hosp UNIQUE (hospitalization_id)

Εξασφαλίζει ότι κάθε νοσηλεία μπορεί να έχει το πολύ μία αξιολόγηση. Δεν επιτρέπεται ο ασθενής να υποβάλει δεύτερη αξιολόγηση για την ίδια νοσηλεία.

CONSTRAINT fk_phr_hosp FOREIGN KEY (hospitalization_id) ... ON DELETE CASCADE ON UPDATE CASCADE

Ορίζει το `hospitalization_id` ως ξένο κλειδί που δείχνει στον πίνακα `hospitalization`. Εξασφαλίζει ότι δεν μπορεί να υπάρξει αξιολόγηση χωρίς αντίστοιχη νοσηλεία.

- ON DELETE CASCADE

Αν διαγραφεί η νοσηλεία, διαγράφεται αυτόματα και η αξιολόγησή της. Η αξιολόγηση δεν έχει νόημα χωρίς τη νοσηλεία στην οποία αναφέρεται.

- ON UPDATE CASCADE

Αν αλλάξει το `hospitalization_id`, η αλλαγή μεταδίδεται αυτόματα και στον `patient_hospitalization_review`. Δεν απαιτείται χειροκίνητη ενημέρωση.

overall_score

Το `overall_score` συμπληρώνεται ξεχωριστά από τον ασθενή και δεν υπολογίζεται αυτόματα από τις επιμέρους βαθμολογίες. Αυτό είναι το νόημα του συστήματος Likert, η συνολική εμπειρία είναι υποκειμενική και μπορεί να διαφέρει από τον μέσο όρο των επιμέρους.

NOT NULL (για όλα τα score)

Κάθε κριτήριο αξιολόγησης είναι υποχρεωτικό, ο ασθενής δεν μπορεί να υποβάλει μερική αξιολόγηση. Η αξιολόγηση δεν έχει νόημα αν λείπει έστω και ένα κριτήριο.

Σχετικά Triggers

Η υποβολή αξιολόγησης επιτρέπεται μόνο αφού ο ασθενής έχει εξέλθει από το νοσοκομείο, δηλαδή όταν το **discharge_date** της αντίστοιχης νοσηλείας είναι NOT NULL. Ο κανόνας αυτός δεν μπορεί να εκφραστεί ως **CHECK** constraint γιατί απαιτεί έλεγχο σε άλλον πίνακα (**hospitalization**), και εξασφαλίζεται μέσω trigger. Για πλήρη ανάλυση βλέπε **Ενότητα Triggers** → **trg_hr_after_discharge**.

Assumption

Η αξιολόγηση είναι **προαιρετική**, η εκφώνηση αναφέρει ότι οι ασθενείς **μπορούν** να αξιολογούν την εμπειρία τους, όχι ότι υποχρεούνται. Δεν υπάρχει κανένα constraint που να επιβάλλει την υποβολή αξιολόγησης μετά την έξοδο. Αν ο ασθενής επιλέξει να αξιολογήσει, τότε εφαρμόζονται οι κανόνες του πίνακα.

Πίνακας 10.2: patient_doctor_review

Σκοπός

Ο πίνακας **patient_doctor_review** αποθηκεύει την αξιολόγηση κάθε γιατρού που συνταγογράφησε κατά τη νοσηλεία του ασθενή. Ο ασθενής μπορεί να αξιολογήσει κάθε γιατρό που εξέδωσε συνταγή κατά τη νοσηλεία του, με βαθμολογία ποιότητας ιατρικής φροντίδας σε κλίμακα Likert 1-5. Η αξιολόγηση είναι προαιρετική και επιτρέπεται μόνο μετά την ολοκλήρωση της νοσηλείας.

Κώδικας SQL

```
CREATE TABLE patient_doctor_review (
    doctor_review_id SERIAL NOT NULL,
    hospitalization_id INT NOT NULL,
    doctor_id INT NOT NULL,
    medical_care_score INT NOT NULL CHECK (medical_care_score BETWEEN 1 AND 5),
    CONSTRAINT pk_patient_doctor_review PRIMARY KEY (doctor_review_id),
    CONSTRAINT fk_dr_hosp FOREIGN KEY (hospitalization_id) REFERENCES
hospitalization (hospitalization_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_dr_doctor FOREIGN KEY (doctor_id) REFERENCES doctor
(staff_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT uq_dr_hosp_doctor UNIQUE (hospitalization_id, doctor_id)
);
```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
doctor_review_id	SERIAL	NOT NULL	PK	ID κάθε αξιολόγησης γιατρού. SERIAL για αυτόματη αύξηση.

Στήλη	Τύπος		Constraint	Αιτιολόγηση
hospitalization_id	INT	NOT NULL	FK → hospitalization	Σύνδεση με τη νοσηλεία. NOT NULL γιατί η αξιολόγηση γίνεται πάντα στο πλαίσιο συγκεκριμένης νοσηλείας.
doctor_id	INT	NOT NULL	FK → doctor	Ο ιατρός που αξιολογείται. NOT NULL γιατί κάθε αξιολόγηση αφορά υποχρεωτικά έναν ιατρό.
medical_care_score	INT	NOT NULL	CHECK (1-5)	Βαθμολογία ποιότητας ιατρικής φροντίδας. Κλίμακα Likert 1-5. NOT NULL γιατί η αξιολόγηση δεν έχει νόημα χωρίς βαθμολογία.

Explanations & Constraints

CONSTRAINT uq_dr_hosp_doctor UNIQUE (hospitalization_id, doctor_id)

Το UNIQUE δεν ορίζεται σε κάθε στήλη ξεχωριστά αλλά στον **συνδυασμό** (**hospitalization_id**, **doctor_id**). Αυτό σημαίνει ότι ο ασθενής μπορεί να αξιολογήσει πολλούς γιατρούς στην ίδια νοσηλεία, αλλά κάθε ιατρό μία φορά. Ο ίδιος ιατρός μπορεί να αξιολογηθεί ξανά σε διαφορετική νοσηλεία.

CONSTRAINT fk_dr_hosp ... ON DELETE CASCADE ON UPDATE CASCADE

Ορίζει το **hospitalization_id** ως ξένο κλειδί που δείχνει στον πίνακα **hospitalization**. Αν διαγραφεί η νοσηλεία, διαγράφονται αυτόματα και όλες οι αξιολογήσεις ιατρών που αναφέρονται σε αυτή. Αν αλλάξει το **hospitalization_id**, η αλλαγή μεταδίδεται αυτόματα.

CONSTRAINT fk_dr_doctor ... ON DELETE RESTRICT ON UPDATE CASCADE

Ορίζει το **doctor_id** ως ξένο κλειδί που δείχνει στον πίνακα **doctor**. Αν ένας ιατρός προσπαθήσει να διαγραφεί ενώ υπάρχουν αξιολογήσεις του, η διαγραφή απαγορεύεται. Το **ιστορικό αξιολογήσεων πρέπει να διατηρείται**.

Σχετικά Triggers

Ο πίνακας αυτός σχετίζεται με δύο triggers που δεν μπορούν να εκφραστούν ως CHECK constraints γιατί απαιτούν έλεγχο σε άλλους πίνακες.

Η υποβολή αξιολόγησης επιτρέπεται μόνο αφού ο ασθενής έχει εξέλθει από το νοσοκομείο, δηλαδή όταν το **discharge_date** της αντίστοιχης νοσηλείας είναι NOT NULL. Απαιτεί έλεγχο στον πίνακα **hospitalization**. **Ενότητα Triggers** → **trg_dr_after_discharge**

Η αξιολόγηση επιτρέπεται μόνο για ιατρό που έχει συνταγογραφήσει κατά τη νοσηλεία. Απαιτεί έλεγχο στον πίνακα **prescription**. **Ενότητα Triggers** → **trg_dr_prescribed_doctor**

Ενότητα 11: MEDIA

Η ενότητα αυτή διαχειρίζεται τις εικόνες που συνδέονται με τις οντότητες του συστήματος. Αποτελείται από έναν πίνακα (image) που αποθηκεύει URLs εικόνων και τη σύνδεσή τους με οποιαδήποτε οντότητα της βάσης: τμήματα, προσωπικό, ασθενείς, αίθουσες, φάρμακα, δραστικές ουσίες, αλλεργίες και κλίνες.

Προδιαγραφές Απαιτήσεων Συστήματος

Κάθε οντότητα του συστήματος (τμήμα, ιατρός, ασθενής, φάρμακο κ.ά.) μπορεί να έχει συνδεδεμένες εικόνες. Κάθε εικόνα ανήκει αποκλειστικά σε μία οντότητα, δεν επιτρέπεται εικόνα που αναφέρεται ταυτόχρονα σε δύο διαφορετικές οντότητες. Επίσης, κάθε εικόνα πρέπει να ανήκει σε τουλάχιστον μία οντότητα, δεν επιτρέπονται εικόνες χωρίς σύνδεση. Και οι δύο αυτοί κανόνες επιβάλλονται αυτόματα από το trigger `trg_image_entity_check`.

Πίνακας 11.1: image

Σκοπός

Ο πίνακας image αποθηκεύει URLs εικόνων και τη σύνδεσή τους με τις οντότητες του συστήματος. Κάθε εγγραφή αντιστοιχεί σε μία εικόνα που ανήκει ακριβώς σε μία οντότητα, εκφραζόμενη μέσω ενός εκ των FK (department_name, staff_id, patient_id, room_id, drug_code, active_substance, allergy_id, bed_id). Η επιλογή του «πλατιού» πίνακα με πολλά nullable FKs αντί για ξεχωριστούς πίνακες ανά οντότητα αποτρέπει διπλασιασμό δομής και επιτρέπει ενιαία διαχείριση εικόνων.

Κώδικας SQL

```
CREATE TABLE image (  
    image_id INT GENERATED ALWAYS AS IDENTITY,  
    is_public BOOLEAN DEFAULT TRUE,  
    department_name VARCHAR(100) DEFAULT NULL,  
    staff_id INT DEFAULT NULL,  
    patient_id INT DEFAULT NULL,  
    room_id INT DEFAULT NULL,  
    drug_code VARCHAR(30) DEFAULT NULL,  
    active_substance VARCHAR(200) DEFAULT NULL,  
    allergy_id INT DEFAULT NULL,  
    bed_id INT DEFAULT NULL,  
    url VARCHAR(255) NOT NULL,  
    caption VARCHAR(255) NOT NULL,  
    CONSTRAINT pk_image PRIMARY KEY (image_id),  
    CONSTRAINT fk_img_dept FOREIGN KEY (department_name) REFERENCES department  
(department_name) ON DELETE SET NULL ON UPDATE CASCADE,  
    CONSTRAINT fk_img_staff FOREIGN KEY (staff_id) REFERENCES staff (staff_id)
```

```

ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_img_patient FOREIGN KEY (patient_id) REFERENCES patient
(patient_id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_img_room FOREIGN KEY (room_id) REFERENCES operating_room
(room_id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_img_drug FOREIGN KEY (drug_code) REFERENCES drug (drug_code)
ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_img_subst FOREIGN KEY (active_substance) REFERENCES
active_substance (name) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_img_allergy FOREIGN KEY (allergy_id) REFERENCES
patient_allergy (allergy_id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_img_bed FOREIGN KEY (bed_id) REFERENCES department_beds
(bed_id) ON DELETE SET NULL ON UPDATE CASCADE
);

```

Ανάλυση Στηλών & Constraints

Στήλη	Τύπος		Constraint	Αιτιολόγηση
image_id	INT IDENTITY	NOT NULL	PK	ID κάθε εικόνας. GENERATED ALWAYS AS IDENTITY, δεν επιτρέπεται χειροκίνητη εισαγωγή τιμής.
is_public	BOOLEAN		DEFAULT TRUE	Ορατότητα εικόνας. DEFAULT TRUE γιατί η συντριπτική πλειονότητα των εικόνων (τμήματα, φάρμακα, εξοπλισμός) είναι δημόσιες. NULL δεν επιτρέπεται.
department_name	VARCHAR (100)	NULL	FK → department	Σύνδεση με τμήμα. NULL αν η εικόνα δεν αφορά τμήμα. ON DELETE SET NULL: αν διαγραφεί το τμήμα, η εικόνα παραμένει χωρίς σύνδεση.
staff_id	INT	NULL	FK → staff	Σύνδεση με μέλος προσωπικού. NULL αν η εικόνα δεν αφορά προσωπικό. ON DELETE SET NULL.
patient_id	INT	NULL	FK → patient	Σύνδεση με ασθενή. NULL αν η εικόνα δεν αφορά ασθενή. ON DELETE SET NULL.
room_id	INT	NULL	FK → operating_room	Σύνδεση με αίθουσα επέμβασης. NULL αν η εικόνα δεν αφορά αίθουσα. ON DELETE SET NULL.
drug_code	VARCHAR (30)	NULL	FK → drug	Σύνδεση με φάρμακο. NULL αν η εικόνα δεν αφορά φάρμακο. ON DELETE SET NULL.

Στήλη	Τύπος		Constraint	Αιτιολόγηση
active_substance	VARCHAR (200)	NULL	FK → active_substance	Σύνδεση με δραστική ουσία. NULL αν η εικόνα δεν αφορά δραστική ουσία. ON DELETE SET NULL.
allergy_id	INT	NULL	FK → patient_allergy	Σύνδεση με αλλεργία ασθενή. NULL αν η εικόνα δεν αφορά αλλεργία. ON DELETE SET NULL.
bed_id	INT	NULL	FK → department_beds	Σύνδεση με κλίνη. NULL αν η εικόνα δεν αφορά κλίνη. ON DELETE SET NULL.
url	VARCHAR (255)	NOT NULL		URL της εικόνας. NOT NULL γιατί χωρίς URL η εγγραφή είναι άχρηστη.
caption	VARCHAR (1000)	NOT NULL		Λεζάντα εικόνας. NOT NULL γιατί κάθε εικόνα πρέπει να έχει περιγραφή για προσβασιμότητα και αναζήτηση.

Explanations & Constraints

nullable

Η εναλλακτική θα ήταν ξεχωριστοί πίνακες ανά οντότητα (department_image, staff_image, drug_image κ.ά.). Αυτό θα σήμαινε 8+ επιπλέον πίνακες με πανομοιότυπη δομή. Ο ενιαίος πίνακας image με nullable FKs είναι πιο συντηρήσιμος, επιτρέπει ενιαία αναζήτηση και η επιχειρησιακή λογική «ακριβώς μία σύνδεση» επιβάλλεται από trigger.

ON DELETE SET NULL

Αν διαγραφεί οποιαδήποτε οντότητα (π.χ. ένας ιατρός αποχωρήσει, ένα φάρμακο αποσυρθεί), η εικόνα δεν διαγράφεται — το αντίστοιχο FK γίνεται NULL. Αυτό επιτρέπει τη διατήρηση του αρχείου εικόνων ακόμα και μετά τη διαγραφή της συνδεδεμένης οντότητας. Ωστόσο, μια εικόνα χωρίς σύνδεση (όλα NULL) εντοπίζεται από το trigger και μπορεί να καθαριστεί.

GENERATED ALWAYS AS IDENTITY

Χρησιμοποιείται αντί για SERIAL γιατί είναι το σύγχρονο πρότυπο SQL. Το ALWAYS εξασφαλίζει ότι δεν μπορεί να εισαχθεί χειροκίνητα τιμή για το image_id, αποτρέποντας συγκρούσεις κατά τη φόρτωση δεδομένων.

is_public DEFAULT TRUE

Η συντριπτική πλειονότητα των εικόνων (φωτογραφίες τμημάτων, εξοπλισμού, φαρμάκων) προορίζεται για δημόσια προβολή στην ιστοσελίδα του νοσοκομείου. Μόνο εικόνες ασθενών ή ευαίσθητου υλικού ορίζονται ως μη δημόσιες (is_public = FALSE).

Σχετικά Triggers

trg_image_entity_check – BEFORE INSERT OR UPDATE ON image

Μετράει πόσα από τα 8 nullable FK πεδία (department_name, staff_id, patient_id, room_id, drug_code, active_substance, allergy_id, bed_id) είναι NOT NULL. Αν η μέτρηση είναι 0, απαγορεύει την εισαγωγή: η εικόνα δεν μπορεί να υπάρχει χωρίς σύνδεση σε οντότητα. Αν η μέτρηση είναι > 1, απαγορεύει επίσης: η εικόνα δεν μπορεί να ανήκει σε περισσότερες από μία οντότητες. Η λογική αυτή δεν μπορεί να

εκφραστεί ως CHECK constraint γιατί αφορά πολλαπλά πεδία ταυτόχρονα και απαιτεί αριθμητική σύγκριση του πλήθους NOT NULL πεδίων.

Assumption

Οι εικόνες αποθηκεύονται εξωτερικά (cloud storage, CDN) και η βάση κρατάει μόνο το URL. Αυτό είναι η βέλτιστη πρακτική για παραγωγικά συστήματα: αποφεύγεται η αποθήκευση binary δεδομένων στη βάση, που θα επιβάρυνε σημαντικά το μέγεθός της και την απόδοση των queries. Το caption είναι NOT NULL για λόγους προσβασιμότητας (accessibility) — κάθε εικόνα πρέπει να έχει εναλλακτικό κείμενο.

Κεφάλαιο Γ: Αναλυτική Τεκμηρίωση Views

Η βάση δεδομένων του νοσοκομείου «Υγειόπολης» ορίζει 14 ενεργά views (ένα παραμένει σε σχόλιο) οργανωμένα ανά λειτουργική ενότητα. Κάθε view αποτελεί εικονικό πίνακα χωρίς αποθηκευμένα δεδομένα εκτελείται ζωντανά κατά κάθε κλήση και εξυπηρετεί τόσο το REST API του UI όσο και τα ερωτήματα SQL του Κεφαλαίου Ε.

View	Ενότητα	Σκοπός (σύνοψη)
v_department_full_profile	Departments	Πλήρες προφίλ τμήματος: διευθυντής, ιατροί, νοσηλευτές, διοικητικοί
v_available_beds	Departments	Διαθέσιμες κλίνες ανά τμήμα με window function
v_staff_full_profile	Staff	Ενοποιημένο προφίλ προσωπικού (ISA ιεραρχία)
v_department_directors	Staff	Διευθυντές ανά τμήμα
v_patient_full_profile	Patients	Πλήρες προφίλ ασθενή: δημογραφικά, αλλεργίες, νοσηλείες
v_triage_queue	Triage	Ουρά αναμονής ΤΕΠ (εκκρεμείς διαλογές)
v_active_hospitalizations	Hospitalization	Ενεργές νοσηλείες με θεράποντα ιατρό
v_hospitalization_cost_breakdown	Hospitalization	Ανάλυση κόστους ΚΕΝ ανά νοσηλεία
v_patient_prescriptions	Prescriptions	Συνταγές με πλήρη στοιχεία ασθενή/ιατρού/φαρμάκου
v_triage_nurses	Shifts	Νοσηλευτές σε ενεργή βάρδια τρέχουσας ώρας
v_monthly_shift_summary	Shifts	Μηνιαία σύνοψη βαρδιών ανά μέλος
v_hospitalization_review_avg	Reviews	Μέση βαθμολογία αξιολόγησης ανά νοσηλεία
v_doctor_review_avg	Reviews	Μέση βαθμολογία αξιολόγησης ανά ιατρό

Γ.1 Ενότητα 1 - Hospitalization Departments

v_department_full_profile

Παρέχει πλήρη εικόνα κάθε τμήματος σε ένα μόνο ερώτημα: ποιος το διευθύνει, ποιοι γιατροί/νοσηλευτές/διοικητικοί ανήκουν σε αυτό. Οι λίστες προσωπικού ενοποιούνται με STRING_AGG μέσω correlated subqueries.

```
CREATE VIEW v_department_full_profile AS
SELECT
    d.department_name,
    d.director_id,
```

```

s.first_name AS director_first_name,
s.last_name AS director_last_name,
-- Προσωπικό τμήματος
(SELECT STRING_AGG(first_name || ' ' || last_name || ' (' || staff_type ||
'),' , ', ' ')
FROM staff st JOIN doctor_department dd ON st.staff_id = dd.doctor_id
WHERE dd.department_name = d.department_name) AS doctors,
(SELECT STRING_AGG(first_name || ' ' || last_name || ' (' || staff_type ||
'),' , ', ' ')
FROM staff st JOIN nurse n ON st.staff_id = n.staff_id
WHERE n.department_name = d.department_name) AS nurses,
(SELECT STRING_AGG(first_name || ' ' || last_name || ' (' || staff_type ||
'),' , ', ' ')
FROM staff st JOIN admin_staff a ON st.staff_id = a.staff_id
WHERE a.department_name = d.department_name) AS admin_staff
FROM department d
JOIN staff s ON d.director_id = s.staff_id;

```

Σχεδιαστικές Επιλογές

Correlated subqueries (αντί JOIN + GROUP BY):

Επιτρέπουν μία γραμμή ανά τμήμα χωρίς GROUP BY στο κύριο query.

STRING_AGG:

ενοποιεί πολλαπλές εγγραφές (ιατροί/νοσηλευτές) σε μία συμβολοσειρά, κατάλληλο για UI display.

Χρήση

Dashboard τμημάτων, λίστα τμημάτων στο API endpoint /api/departments.

v_available_beds

Εμφανίζει μόνο τις κλίνες με status='διαθέσιμη' που δεν είναι ανατεθειμένες σε ενεργή νοσηλεία.

Χρησιμοποιεί window function για αριθμό διαθέσιμων κλινών ανά τμήμα.

```

CREATE VIEW v_available_beds AS
SELECT
    db.bed_id,
    db.department_name,
    db.type AS bed_type,
    db.status,
    d.floor_building,

```

```
d.bed_capacity,
COUNT(*) OVER (PARTITION BY db.department_name) AS available_beds_in_dept
FROM department_beds db
JOIN department d ON db.department_name = d.department_name
WHERE db.status = 'διαθέσιμη' AND db.bed_id NOT IN (SELECT h.bed_id FROM
hospitalization h WHERE h.discharge_date IS NULL)
ORDER BY db.department_name, db.type, db.bed_id;
```

Σχεδιαστικές Επιλογές

Διπλό φίλτρο ασφαλείας:

Status = 'διαθέσιμη' ΚΑΙ NOT IN (ενεργές νοσηλείες). Αποτρέπει ανάθεση κλίνης που εμφανίζεται διαθέσιμη αλλά έχει ενεργή νοσηλεία λόγω race condition.

COUNT(*) OVER PARTITION BY:

Παρέχει πλήθος διαθέσιμων κλινών ανά τμήμα σε κάθε γραμμή χωρίς GROUP BY, ιδανικό για UI dropdown που χρειάζεται και την κλίνη και το πλήθος.

Χρήση:

Φόρμα εισαγωγής νοσηλείας (επιλογή κλίνης), Q08 έλεγχος διαθεσιμότητας.

Γ.2 Ενότητα 2 — Staff

v_staff_full_profile

Το πιο σύνθετο view της βάσης. Ενοποιεί τον supertype πίνακα staff με τους τρεις subtypes (doctor, nurse, admin_staff) μέσω LEFT JOINS, εκθέτοντας όλες τις επιμέρους στήλες σε ένα ενιαίο προφίλ. Υπολογίζει δυναμικά την ηλικία και επιλύει τον supervisor μέσω self-join.

```
CREATE VIEW v_staff_full_profile AS
SELECT
    s.staff_id,
    s.amka,
    s.first_name,
    s.last_name,
    s.email,
    s.phone,
    s.hire_date,
    s.staff_type,
    DATE_PART('year', AGE(CURRENT_DATE, s.date_of_birth)) AS age,
    -- Ιατρός
    d.license_number AS doctor_license,
```

```

d.specialty AS doctor_specialty,
d.rank AS doctor_rank,
sup.staff_id AS supervisor_id,
sup_s.first_name AS supervisor_first_name,
sup_s.last_name AS supervisor_last_name,
-- Νοσηλεύτης
n.nurse_rank,
n.department_name AS nurse_department,
-- Διοικητικό
a.role AS admin_role,
a.office AS admin_office,
a.department_name AS admin_department,
-- Τμήματα ιατρών
dd.department_name AS doctor_department,
-- Ενιαίος ρόλος
CASE
    WHEN d.staff_id IS NOT NULL THEN 'Ιατρός'
    WHEN n.staff_id IS NOT NULL THEN 'Νοσηλεύτης'
    WHEN a.staff_id IS NOT NULL THEN 'Διοικητικό'
    ELSE 'Άγνωστο'
END AS resolved_role
FROM staff s
LEFT JOIN doctor d ON s.staff_id = d.staff_id
LEFT JOIN doctor sup ON d.supervisor_id = sup.staff_id
LEFT JOIN staff sup_s ON sup.staff_id = sup_s.staff_id
LEFT JOIN nurse n ON s.staff_id = n.staff_id
LEFT JOIN admin_staff a ON s.staff_id = a.staff_id
LEFT JOIN doctor_department dd ON d.staff_id = dd.doctor_id;

```

Σχεδιαστικές Επιλογές

LEFT JOINS (όχι INNER):

Κάθε μέλος staff ανήκει σε ακριβώς ένα subtype, τα άλλα δύο LEFT JOINS επιστρέφουν NULL, εξασφαλίζοντας ότι δεν χάνεται καμία εγγραφή.

Self-join για supervisor:

JOIN doctor sup → JOIN staff sup_s επιλύει ονοματεπώνυμο επόπτη σε ένα πέρασμα.

CASE WHEN IS NOT NULL:

Εντοπίζει τον ενεργό subtype και εκθέτει readable resolved_role.

v_department_directors

Απλό αναφορικό view: εμφανίζει ονοματεπώνυμο και βαθμίδα Διευθυντή για κάθε τμήμα. Χρησιμοποιείται σε dropdowns κατά τη δημιουργία νέου τμήματος ή επεξεργασία διευθυντή.

```
CREATE VIEW v_department_directors AS
SELECT
    d.department_name,
    doc.staff_id,
    s.first_name,
    s.last_name,
    doc.rank
FROM department d
JOIN doctor doc ON d.director_id = doc.staff_id
JOIN staff s ON s.staff_id = doc.staff_id;
```

Γ.3 Ενότητα 4 — Patients

v_patient_full_profile

Το κεντρικό view της ενότητας Patients. Ενοποιεί δημογραφικά, αλλεργίες (STRING_AGG), πρόσωπα επαφής, ιστορικό νοσηλειών και τελευταίο βάρος/ύψος από triage. Χρησιμοποιεί correlated subqueries για weight/height ώστε να αποφύγει GROUP BY σε πολύπλοκο JOIN.

```
CREATE VIEW v_patient_full_profile AS
SELECT
    p.patient_id,
    p.amka,
    p.first_name,
    p.last_name,
    p.fathers_name,
    p.gender,
    p.street,
    p.city,
    p.postal_code,
    p.phone,
    p.email,
    p.occupation,
    p.nationality,
    c.name AS nationality_name,
```

```

    i.name AS insurer_name,
    i.type AS insurer_type,

    EXTRACT(YEAR FROM AGE(p.dob))::INT AS age,

    (SELECT weight FROM triage WHERE patient_id = p.patient_id ORDER BY
arrival_time DESC LIMIT 1) AS last_weight,
    (SELECT height FROM triage WHERE patient_id = p.patient_id ORDER BY
arrival_time DESC LIMIT 1) AS last_height,

    STRING_AGG(DISTINCT pa.active_substance, ', ') AS allergies,

    STRING_AGG(DISTINCT ec.first_name || ' ' || ec.last_name || ' (' ||
COALESCE(pec.relation, '?') || ')', ' | '
) AS emergency_contacts,

    COUNT(DISTINCT h.hospitalization_id) AS total_hospitalizations,
    MAX(h.admission_date) AS last_admission_date

FROM patient p
LEFT JOIN country c ON p.nationality = c.country_code
LEFT JOIN insurer i ON p.insurer_id = i.insurer_id
LEFT JOIN patient_allergy pa ON p.patient_id = pa.patient_id
LEFT JOIN patient_emergency_contact pec ON p.patient_id = pec.patient_id
LEFT JOIN emergency_contact ec ON pec.contact_id = ec.contact_id
LEFT JOIN hospitalization h ON p.patient_id = h.patient_id
GROUP BY
    p.patient_id, p.amka, p.first_name, p.last_name, p.fathers_name,
    p.dob, p.gender, p.street, p.city, p.postal_code, p.phone, p.email,
    p.occupation, p.nationality, c.name, i.name, i.type;

```

Σχεδιαστικές Επιλογές

STRING_AGG(DISTINCT ...):

Η διπλή LEFT JOIN (allergy + emergency_contact) μπορεί να πολλαπλασιάσει γραμμές, το DISTINCT εξαλείφει διπλότυπα χωρίς υποερώτημα.

Correlated subqueries για weight/height:

Αποφεύγει GROUP BY στο κύριο query για πεδία που χρειάζονται ORDER BY (LIMIT 1). Δεν είναι δυνατό να ενσωματωθεί στο GROUP BY αρχιτεκτονικά.

Γ.4 Ενότητα 5 — Triage

v_triage_queue

Εμφανίζει την ουρά αναμονής ΤΕΠ: αποκλειστικά εγγραφές με outcome IS NULL, ταξινομημένες κατά urgency_level (ASC) και arrival_time (ASC), **FIFO εντός επιπέδου επείγοντος**.

```
CREATE VIEW v_triage_queue AS
SELECT
    t.triage_id,
    t.patient_id,
    p.first_name,
    p.last_name,
    p.amka,
    t.arrival_time,
    t.urgency_level,
    t.symptoms,
    t.nurse_id,
    s.first_name AS nurse_first_name,
    s.last_name AS nurse_last_name
FROM triage t
JOIN patient p ON t.patient_id = p.patient_id
JOIN staff s ON t.nurse_id = s.staff_id
WHERE t.outcome IS NULL
ORDER BY t.urgency_level, t.arrival_time;
```

Σχεδιαστικές Επιλογές

WHERE outcome IS NULL:

Φιλτράρει μόνο εκκρεμείς διαλογές. Μόλις συμπληρωθεί το outcome, εξαφανίζεται αυτόματα από την ουρά.

ORDER BY urgency_level, arrival_time:

Εφαρμόζει κλινικό πρωτόκολλο τριάζ, κρίσιμα πρώτα, ίδια επείγοντα FIFO.

Γ.5 Ενότητα 6 — Hospitalization

v_active_hospitalizations

Εμφανίζει μόνο ενεργές νοσηλείες (discharge_date IS NULL) με ονοματεπώνυμο ασθενή, τύπο κλίνης και θεράποντα γιατρό μέσω LEFT JOIN στον hospitalization_staff (role='attending_doctor').

```
CREATE VIEW v_active_hospitalizations AS
```

```

SELECT
    h.hospitalization_id,
    h.patient_id,
    p.first_name AS patient_first_name,
    p.last_name AS patient_last_name,
    p.amka AS patient_amka,
    h.department_name,
    h.bed_id,
    db.type AS bed_type,
    h.admission_date,
    h.admission_diag_icd10,
    h.ken_code,
    hs.staff_id AS attending_doctor_id,
    s.first_name AS doctor_first_name,
    s.last_name AS doctor_last_name
FROM hospitalization h
JOIN patient p ON h.patient_id = p.patient_id
JOIN department_beds db ON h.bed_id = db.bed_id
LEFT JOIN hospitalization_staff hs ON hs.hospitalization_id =
h.hospitalization_id AND hs.role = 'attending_doctor'
LEFT JOIN staff s ON hs.staff_id = s.staff_id
WHERE h.discharge_date IS NULL;

```

Σχεδιαστικές Επιλογές

LEFT JOIN hospitalization_staff:

Νοσηλεία μπορεί να μην έχει ακόμα ανατεθεί θεράποντα, το LEFT JOIN εξασφαλίζει εμφάνιση ακόμα και χωρίς ιατρό.

Φίλτρο role='attending_doctor':

Αποφεύγει πολλαπλές γραμμές ανά νοσηλεία (ένας attending, ένας consultant κ.λπ.).

WHERE discharge_date IS NULL:

Αποκλείει ιστορικές νοσηλείες για αποδοτικό index scan (idx_hospitalization_discharge_date).

v_hospitalization_cost_breakdown

Παρέχει αναλυτική κοστολόγηση ανά νοσηλεία: base_cost KEN, extra_days πέραν ΜΔΝ, ken_total και stored_total_cost. Δεν συμπεριλαμβάνει κόστος ιατρικών πράξεων (πεδίο δεν υπάρχει στο τρέχον schema, ορίζεται ως 0 με σχόλιο FIXED).


```

CREATE VIEW v_hospitalization_cost_breakdown AS
SELECT
    h.hospitalization_id,
    h.patient_id,
    p.first_name AS patient_first_name,
    p.last_name AS patient_last_name,
    h.department_name,
    h.admission_date,
    h.discharge_date,
    h.ken_code,
    k.base_cost AS ken_base_cost,
    k.avg_duration_days AS ken_avg_days,
    k.daily_extra AS ken_daily_extra,
    GREATEST(0, (h.discharge_date - h.admission_date) - k.avg_duration_days) AS
extra_days,
    COALESCE(k.base_cost + GREATEST(0, (h.discharge_date - h.admission_date) -
k.avg_duration_days) * k.daily_extra, 0) AS ken_total,
    0::NUMERIC AS procedures_cost,
    h.total_cost AS stored_total_cost
FROM hospitalization h
JOIN patient p ON h.patient_id = p.patient_id
LEFT JOIN ken_code k ON h.ken_code = k.ken_code
LEFT JOIN hospitalization_lab le ON le.hospitalization_id =
h.hospitalization_id
GROUP BY
    h.hospitalization_id, h.patient_id, p.first_name, p.last_name,
    h.department_name, h.admission_date, h.discharge_date, h.ken_code,
    k.base_cost, k.avg_duration_days, k.daily_extra, h.total_cost;

```

Σχεδιαστικές Επιλογές

GREATEST(0, ...):

Εξασφαλίζει μη-αρνητικό extra_days, αν παραμονή < ΜΔΝ, extra_days=0.

COALESCE(..., 0):

Νοσηλείες χωρίς KEN (ken_code IS NULL) εμφανίζουν 0 αντί NULL στο ken_total.

LEFT JOIN ken_code: Ενεργές νοσηλείες μπορεί να μην έχουν ακόμα KEN — το LEFT JOIN αποτρέπει εξαφάνισή τους.

Γ.6 Ενότητα 8 — Prescriptions

v_patient_prescriptions

Ενοποιημένο αρχείο συνταγών: ονοματεπώνυμο ασθενή, νοσηλεία (τμήμα + ημερομηνίες), πλήρη στοιχεία ιατρού (ειδικότητα, βαθμίδα), φάρμακο, δοσολογία, διάρκεια. Υπολογίζει duration_days αριθμητικά.

```
CREATE OR REPLACE VIEW v_patient_prescriptions AS
SELECT
    pr.prescription_id,
    p.patient_id,
    p.first_name AS patient_first_name,
    p.last_name AS patient_last_name,
    p.amka AS patient_amka,
    pr.hospitalization_id,
    h.department_name AS hospitalization_dept,
    h.admission_date,
    h.discharge_date,
    s.staff_id AS doctor_id,
    s.first_name AS doctor_first_name,
    s.last_name AS doctor_last_name,
    d.specialty AS doctor_specialty,
    d.rank AS doctor_rank,
    dr.drug_code,
    dr.name AS drug_name,
    pr.dosage,
    pr.frequency,
    pr.start_date,
    pr.end_date,
    (pr.end_date - pr.start_date) AS duration_days
FROM prescription pr
JOIN patient p ON pr.patient_id = p.patient_id
JOIN hospitalization h ON pr.hospitalization_id = h.hospitalization_id
JOIN staff s ON pr.doctor_id = s.staff_id
JOIN doctor d ON pr.doctor_id = d.staff_id
JOIN drug dr ON pr.drug_code = dr.drug_code
ORDER BY pr.patient_id, pr.start_date;
```

Σχεδιαστικές Επιλογές

INNER JOINS:

Κάθε συνταγή έχει υποχρεωτικά ασθενή, νοσηλεία, ιατρό και φάρμακο (NOT NULL FKs). Το INNER JOIN εξαλείφει ορφανές εγγραφές.

(end_date - start_date) AS duration_days:

Αριθμητική αφαίρεση DATE επιστρέφει INT (ημέρες) στη PostgreSQL — χρήσιμο για φιλτράρισμα μακράς αγωγής.

Γ.7 Ενότητα 9 — Shift Scheduling

v_triage_nurses

Εμφανίζει νοσηλευτές που βρίσκονται σε ενεργή βάρδια τη στιγμή της κλήσης. Ελέγχει CURRENT_DATE και CURRENT_TIME έναντι του τύπου βάρδιας (Π/A/N) για αυτόματη φιλτράρισμα.

```
CREATE VIEW v_triage_nurses AS
SELECT
    s.staff_id,
    s.first_name,
    s.last_name,
    n.nurse_rank,
    n.department_name,
    sh.shift_type,
    sh.shift_date
FROM staff s
JOIN nurse n ON s.staff_id = n.staff_id
JOIN shift_staff ss ON s.staff_id = ss.staff_id
JOIN shifts sh ON ss.shift_id = sh.shift_id
WHERE sh.shift_date = CURRENT_DATE
    AND (
        (sh.shift_type = 'Π' AND CURRENT_TIME BETWEEN '07:00' AND '15:00')
        OR
        (sh.shift_type = 'Α' AND CURRENT_TIME BETWEEN '15:00' AND '23:00')
        OR
        (sh.shift_type = 'Ν' AND (CURRENT_TIME >= '23:00' OR CURRENT_TIME <=
'07:00'))
    )
ORDER BY n.nurse_rank DESC, s.last_name, s.first_name;
```

Σχεδιαστικές Επιλογές

CURRENT_DATE / CURRENT_TIME:

To view επιστρέφει πάντα ζωντανά δεδομένα — κατάλληλο για real-time φόρμα τριάζ.

OR για νυχτερινή βάρδια:

Η νυχτερινή (23:00-07:00) διασχίζει τα μεσάνυχτα, οπότε απαιτείται σύνθετη συνθήκη.

ORDER BY nurse_rank DESC:

Εμφανίζει πρώτα ανώτερους νοσηλευτές για προτεραιοποίηση στο dropdown.

v_monthly_shift_summary

Παρέχει μηνιαία σύνοψη βαρδιών ανά μέλος, με ανάλυση ανά τύπο (Πρωινή/Απογευματινή/Νυχτερινή) μέσω conditional aggregation. Χρησιμοποιείται για έλεγχο τήρησης μηνιαίων ορίων.

```
CREATE VIEW v_monthly_shift_summary AS
SELECT
    ss.staff_id,
    s.first_name,
    s.last_name,
    s.staff_type,
    EXTRACT(YEAR FROM sh.shift_date)::INT AS year,
    EXTRACT(MONTH FROM sh.shift_date)::INT AS month,
    COUNT(*) AS shift_count,
    COUNT(*) FILTER (WHERE sh.shift_type = 'Π') AS morning_shifts,
    COUNT(*) FILTER (WHERE sh.shift_type = 'Α') AS afternoon_shifts,
    COUNT(*) FILTER (WHERE sh.shift_type = 'Ν') AS night_shifts
FROM shift_staff ss
JOIN staff s ON ss.staff_id = s.staff_id
JOIN shifts sh ON ss.shift_id = sh.shift_id
GROUP BY
    ss.staff_id, s.first_name, s.last_name, s.staff_type,
    EXTRACT(YEAR FROM sh.shift_date),
    EXTRACT(MONTH FROM sh.shift_date);
```

Σχεδιαστικές Επιλογές**COUNT(*) FILTER (WHERE ...):**

Conditional aggregation σε ένα GROUP BY αντί για 3 υποερωτήματα, αποδοτικότερο.

EXTRACT(YEAR/MONTH) αντί DATE_TRUNC:

Επιστρέφει ακέραιους άμεσα αναγνώσιμους χωρίς μετατροπή.

Regular VIEW (όχι MATERIALIZED):

Ανανεώνεται αυτόματα — κάθε εγγραφή στο shift_staff αντικατοπτρίζεται αμέσως.

Γ.8 Ενότητα 10 — Reviews

v_hospitalization_review_avg

Υπολογίζει μέση βαθμολογία ανά νοσηλεία από τους τέσσερις άξονες αξιολόγησης (nursing_care, cleanliness, food, overall). Χρησιμοποιεί AVG() σε κάθε ξεχωριστή στήλη και αριθμητικό μέσο των τεσσάρων.

```
CREATE VIEW v_hospitalization_review_avg AS
SELECT
    hospitalization_id,
    ROUND(
        (AVG(nursing_care_score) + AVG(cleanliness_score) + AVG(food_score) +
        AVG(overall_score)) / 4.0, 2
    ) AS avg_score
FROM patient_hospitalization_review
GROUP BY hospitalization_id;
```

Σχεδιαστικές Επιλογές

Άθροιση 4 AVG() / 4.0:

Αντί ROUND(AVG(a+b+c+d)/4), ο δεύτερος τρόπος θα έδινε λανθασμένο αποτέλεσμα αν κάποιο score ήταν NULL (SUM θα δώσει NULL).

UNIQUE constraint στον πίνακα:

Κάθε νοσηλεία έχει μία μόνο αξιολόγηση, η GROUP BY είναι ισοδύναμη με απλό SELECT.

v_doctor_review_avg

Μέση βαθμολογία medical_care_score ανά ιατρό, από όλες τις αξιολογήσεις που έχει λάβει κατά ποικίλες νοσηλείες.

```
CREATE VIEW v_doctor_review_avg AS
SELECT
    doctor_id,
    ROUND(AVG(medical_care_score), 2) AS avg_score
FROM patient_doctor_review
GROUP BY doctor_id;
```

Σχεδιαστικές Επιλογές

Απλό GROUP BY doctor_id:

Ένας ιατρός μπορεί να αξιολογηθεί πολλές φορές (διαφορετικές νοσηλείες), το AVG() συγκεντρώνει σωστά.

Κεφάλαιο Δ: Triggers - Πλήρης Αναφορά

Η ενότητα αυτή παρέχει συγκεντρωτική αναφορά όλων των 42 triggers της βάσης, οργανωμένη ανά λειτουργική ενότητα. Κάθε trigger τεκμηριώνεται με: σκοπό, συμβάν ενεργοποίησης, λογική υλοποίησης και σχέση με άλλα triggers.

#	Trigger	Ενότητα	Συμβάν	Σκοπός (σύνοψη)
01	trg_department_director_check	Departments	BEFORE INSERT/UPDATE ON department	Ελέγχει rank=Διευθυντής και ανήκει στο τμήμα
02	trg_doctor_specialty_in_department	Departments	BEFORE INSERT/UPDATE ON doctor_department	Ελέγχει ειδικότητα συμβατή με τμήμα
03	trg_one_director_per_department	Departments	BEFORE INSERT/UPDATE ON doctor	Μέγιστο 1 Διευθυντής ανά τμήμα
04	trg_staff_has_subtype (DEFERRABLE)	Staff	AFTER INSERT ON staff	Κάθε staff έχει subtype εγγραφή
05	trg_doctor_matches_type	Staff	BEFORE INSERT/UPDATE ON doctor	staff_type='doctor' για εγγραφή σε doctor
06	trg_nurse_matches_type	Staff	BEFORE INSERT/UPDATE ON nurse	staff_type='nurse' για εγγραφή σε nurse
07	trg_admin_matches_type	Staff	BEFORE INSERT/UPDATE ON admin_staff	staff_type='admin' για εγγραφή σε admin_staff
08	trg_doctor_supervisor_check	Staff	BEFORE INSERT/UPDATE ON doctor	Επόπτης ανώτερης βαθμίδας — αποτρέπει αυτο-επιτήρηση
09	trg_supervisor_same_department (DEFERRABLE)	Staff	AFTER INSERT/UPDATE ON doctor	Επόπτης ίδιο τμήμα με ιατρό
10	trg_patient_updated_at	Patients	BEFORE UPDATE ON patient	Ενημερώνει updated_at αυτόματα
11	trg_patient_discharge_check	Patients	BEFORE UPDATE ON hospitalization	Πληρότητα δεδομένων πριν έξοδο
12	trg_patient_prescription_check	Patients	BEFORE INSERT ON prescription	Ελάχιστα δεδομένα ασθενή πριν συνταγή

#	Trigger	Ενότητα	Συμβάν	Σκοπός (σύνοψη)
13	trg_patient_amka_gr_check	Patients	BEFORE INSERT/UPDATE ON patient	Έλληνας = υποχρεωτικό ΑΜΚΑ
14	trg_triage_nurse_in_emergency	Triage	BEFORE INSERT/UPDATE ON triage	Νοσηλευτής ΤΕΠ σε ενεργή βάρδια
15	trg_triage_single_active	Triage	BEFORE INSERT ON triage	Ένα ενεργό triage ανά ασθενή
16	trg_triage_create_hospitalization	Triage	AFTER UPDATE ON triage	Αυτόματη νοσηλεία όταν outcome=παραπομπή
17	trg_hosp_bed_dept_match	Hospitalization	BEFORE INSERT/UPDATE ON hospitalization	Κλίνη ανήκει στο σωστό τμήμα
18	trg_hosp_bed_status	Hospitalization	BEFORE INSERT / AFTER UPDATE	Κλίνη διαθέσιμη → κατειλημμένη → διαθέσιμη
19	trg_hosp_bed_overlap	Hospitalization	BEFORE INSERT/UPDATE ON hospitalization	Απαγορεύει διπλοκράτηση κλίνης
20	trg_hosp_discharge_diagnosis	Hospitalization	BEFORE UPDATE (discharge_date)	ICD-10 εξόδου + ΚΕΝ πριν έξοδο
21	trg_hosp_total_cost	Hospitalization	AFTER UPDATE (discharge_date)	Αυτόματος υπολογισμός κόστους
23	trg_procedure_during_hosp	Lab & Procedures	BEFORE INSERT/UPDATE ON hosp_procedure	Πράξη εντός χρονικού πλαίσιου νοσηλείας
24	trg_lab_exam_during_hosp	Lab & Procedures	BEFORE INSERT/UPDATE ON hosp_lab	Εξέταση εντός χρονικού πλαίσιου νοσηλείας
25	trg_allergy_check	Prescriptions	BEFORE INSERT ON prescription	Απαγορεύει συνταγή αλλεργιογόνου φαρμάκου
26	trg_prescription_doctor_guard	Prescriptions	BEFORE INSERT ON prescription	Ιατρός ανήκει σε hospitalization_staff
27	trg_prescription_within_hosp	Prescriptions	BEFORE INSERT/UPDATE ON prescription	Ημερομηνίες εντός νοσηλείας
28	trg_shift_staff_belongs_to_department	Shifts	BEFORE INSERT ON shift_staff	Μέλος ανήκει στο τμήμα της βάρδιας

#	Trigger	Ενότητα	Συμβάν	Σκοπός (σύνοψη)
29	trg_shift_min_staff (DEFERRABLE)	Shifts	AFTER INSERT/DELETE ON shift_staff	Ελάχ. 3 ιατροί, 6 νοσηλ., 2 διοικ.
30	trg_shift_resident_needs_senior (DEFERRABLE)	Shifts	AFTER INSERT/DELETE ON shift_staff	Ειδικευόμενος → Επιμελητής Α/Διευθυντής παρών
31	trg_shift_monthly_limit	Shifts	BEFORE INSERT ON shift_staff	Μηνιαίο όριο βαρδιών ανά τύπο
32	trg_shift_rest_8h	Shifts	BEFORE INSERT ON shift_staff	8ω ανάπαυση μεταξύ βαρδιών
33	trg_shift_max_3_night	Shifts	BEFORE INSERT ON shift_staff	Μέγιστο 3 συνεχ. νυχτερινές βάρδιες
34	trg_hr_after_discharge	Reviews	BEFORE INSERT ON hosp_review	Αξιολόγηση μόνο μετά την έξοδο
35	trg_dr_after_discharge	Reviews	BEFORE INSERT ON doctor_review	Αξιολόγηση ιατρού μετά έξοδο
36	trg_dr_prescribed_doctor	Reviews	BEFORE INSERT ON doctor_review	Ιατρός πράγματι συνταγογράφησε
37	trg_image_entity_check	Media	BEFORE INSERT/UPDATE ON image	Polymorphic integrity check
38	trg_refresh_hospitalization_patient_view	Hospitalization	AFTER INSERT/UPDATE/DELETE	Ανανεώνει materialized view

Δ.1 Ενότητα 1 — Hospital Departments

#01 | trg_department_director_check

Επαληθεύει ότι ο director_id που τίθεται σε ένα τμήμα είναι πράγματι ιατρός με rank='Διευθυντής' και δεν διευθύνει ήδη άλλο τμήμα.

```
CREATE OR REPLACE FUNCTION fn_department_director_check()
RETURNS TRIGGER AS $$
DECLARE
    v_rank VARCHAR(30);
    v_other_dept_name VARCHAR(100);
BEGIN
```

```
IF NEW.director_id IS NOT NULL THEN

    SELECT "rank" INTO v_rank
    FROM doctor
    WHERE staff_id = NEW.director_id;

    IF v_rank IS NULL THEN
        RAISE EXCEPTION
            'Ο staff_id=% δεν είναι ιατρός και δεν μπορεί να οριστεί Διευθυντής
            τμήματος.',
            NEW.director_id;
    END IF;

    IF v_rank <> 'Διευθυντής' THEN
        RAISE EXCEPTION
            'Ο ιατρός (staff_id=%) έχει βαθμό "%" – απαιτείται βαθμός
            "Διευθυντής".',
            NEW.director_id, v_rank;
    END IF;

    SELECT department_name INTO v_other_dept_name
    FROM department
    WHERE director_id = NEW.director_id
        AND department_name <> NEW.department_name
    LIMIT 1;

    IF FOUND THEN
        RAISE EXCEPTION
            'Ο ιατρός (staff_id=%) ήδη διευθύνει το τμήμα "%'.',
            NEW.director_id, v_other_dept_name;
    END IF;

END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```

DROP TRIGGER IF EXISTS trg_department_director_check ON department;

CREATE TRIGGER trg_department_director_check
BEFORE INSERT OR UPDATE ON department
FOR EACH ROW
EXECUTE FUNCTION fn_department_director_check();

```

Ο έλεγχος γίνεται στον πίνακα department (BEFORE), πριν ο Διευθυντής εισαχθεί στο doctor_department, εξου και η ανάγκη για DEFERRABLE trigger (#09) για circular dependency.

#02 | trg_doctor_specialty_in_department

Αποτρέπει ανάθεση ιατρού σε τμήμα όταν η ειδικότητά του δεν υπάρχει στο department_specialty του τμήματος. Αναζητά specialty του ιατρού στον doctor, έπειτα ελέγχει αν υπάρχει στο department_specialty για το δοθέν τμήμα. Εξασφαλίζει ότι, π.χ., ένας Καρδιολόγος δεν ανατίθεται στην Ορθοπαιδική.

```

CREATE OR REPLACE FUNCTION fn_check_doctor_specialty_in_department()
RETURNS TRIGGER AS $$
DECLARE
    v_specialty VARCHAR(100);
BEGIN
    SELECT specialty INTO v_specialty
    FROM doctor WHERE staff_id = NEW.doctor_id;

    IF NOT EXISTS (
        SELECT 1
        FROM department_specialty
        WHERE department_name = NEW.department_name
        AND specialty = v_specialty
    ) THEN
        RAISE EXCEPTION
            'Η ειδικότητα "%" δεν επιτρέπεται στο τμήμα "%".',
            v_specialty, NEW.department_name;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_doctor_specialty_in_department

```

```
BEFORE INSERT OR UPDATE ON doctor_department
FOR EACH ROW EXECUTE FUNCTION fn_check_doctor_specialty_in_department();
```

#03 | trg_one_director_per_department

Εξασφαλίζει ότι σε κάθε τμήμα υπάρχει το πολύ ένας Διευθυντής ιατρός. Αν ο νέος ιατρός έχει rank='Διευθυντής', ελέγχει αν υπάρχει ήδη άλλος Διευθυντής στο ίδιο τμήμα μέσω JOIN doctor_department + doctor. Ενεργοποιείται και κατά UPDATE ώστε να αποτρέπει αναβάθμιση ιατρού σε Διευθυντή αν υπάρχει ήδη.

```
CREATE OR REPLACE FUNCTION fn_check_one_director_per_department()
RETURNS TRIGGER AS $$
BEGIN
    IF (SELECT rank FROM doctor WHERE staff_id = NEW.doctor_id) = 'Διευθυντής'
    THEN
        IF EXISTS (
            SELECT 1
            FROM doctor_department dd
            JOIN doctor d ON dd.doctor_id = d.staff_id
            WHERE dd.department_name = NEW.department_name
            AND d.rank = 'Διευθυντής'
            AND dd.doctor_id <> NEW.doctor_id
        ) THEN
            RAISE EXCEPTION
                'Το τμήμα "%" έχει ήδη Διευθυντή.',
                NEW.department_name;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_one_director_per_department
BEFORE INSERT OR UPDATE ON doctor_department
FOR EACH ROW EXECUTE FUNCTION fn_check_one_director_per_department();
```

Δ.2 Ενότητα 2 — Staff ISA Integrity

#04 | trg_staff_has_subtype (CONSTRAINT TRIGGER — DEFERRABLE)

Επιβάλλει ότι κάθε εγγραφή στον staff ανήκει σε τουλάχιστον έναν subtype πίνακα (doctor/nurse/admin_staff). Το trigger είναι DEFERRABLE INITIALLY DEFERRED, εκτελείται στο τέλος του transaction, όχι άμεσα. Αυτό επιτρέπει την εισαγωγή staff ΠΙΝ την εγγραφή subtype εντός του ίδιου transaction (π.χ. σε stored procedure sp_hire_staff). Χωρίς DEFERRABLE, κάθε INSERT staff θα αποτύγχανε αμέσως.

```
CREATE OR REPLACE FUNCTION fn_check_staff_has_subtype()
RETURNS TRIGGER AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM doctor WHERE staff_id = NEW.staff_id) AND
       NOT EXISTS (SELECT 1 FROM nurse WHERE staff_id = NEW.staff_id) AND
       NOT EXISTS (SELECT 1 FROM admin_staff WHERE staff_id = NEW.staff_id)
    THEN
        RAISE EXCEPTION
            'Staff % δεν ανήκει σε κανένα subtype.',
            NEW.staff_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_staff_has_subtype
AFTER INSERT OR UPDATE ON staff
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION fn_check_staff_has_subtype();
```

#05/#06/#07 | trg_doctor/nurse/admin_matches_type

Αποτρέπει εγγραφή σε subtype πίνακα (doctor/nurse/admin_staff) εάν το staff_type δεν αντιστοιχεί. Και τα τρία triggers καλούν την ίδια συνάρτηση fn_check_subtype_matches_type() που χρησιμοποιεί TG_TABLE_NAME για να προσδιορίσει τον πίνακα.

```
CREATE OR REPLACE FUNCTION fn_check_subtype_matches_type()
RETURNS TRIGGER AS $$
DECLARE
```

```

    v_staff_type VARCHAR(25);
BEGIN
    SELECT staff_type INTO v_staff_type
    FROM staff WHERE staff_id = NEW.staff_id;

    IF TG_TABLE_NAME = 'doctor' AND v_staff_type <> 'Ιατρός' THEN
        RAISE EXCEPTION
            'Ο staff % δεν είναι ιατρός (staff_type=%).',
            NEW.staff_id, v_staff_type;
    END IF;

    IF TG_TABLE_NAME = 'nurse' AND v_staff_type <> 'Νοσηλεύτης' THEN
        RAISE EXCEPTION
            'Ο staff % δεν είναι νοσηλεύτης (staff_type=%).',
            NEW.staff_id, v_staff_type;
    END IF;

    IF TG_TABLE_NAME = 'admin_staff' AND v_staff_type <> 'Διοικητικό Προσωπικό'
THEN
        RAISE EXCEPTION
            'Ο staff % δεν είναι διοικητικό προσωπικό (staff_type=%).',
            NEW.staff_id, v_staff_type;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_doctor_matches_type
BEFORE INSERT OR UPDATE ON doctor
FOR EACH ROW EXECUTE FUNCTION fn_check_subtype_matches_type();

CREATE TRIGGER trg_nurse_matches_type
BEFORE INSERT OR UPDATE ON nurse
FOR EACH ROW EXECUTE FUNCTION fn_check_subtype_matches_type();

CREATE TRIGGER trg_admin_matches_type
BEFORE INSERT OR UPDATE ON admin_staff

```

```
FOR EACH ROW EXECUTE FUNCTION fn_check_subtype_matches_type();
```

#08 | trg_doctor_supervisor_check

Σκοπός: Επιβάλλει δύο κανόνες εποπτείας: (α) ο επόπτης πρέπει να είναι ανώτερης βαθμίδας και (β) δεν υπάρχει κυκλική αλυσίδα εποπτείας. (α) Ορίζει rank_order (Ειδικευόμενος=1, Επιμελητής B=2, A=3, Διευθυντής=4) και ελέγχει supervisor_order > rank_order. (β) Διατρέχει αναδρομικά την αλυσίδα supervisor_id ελέγχοντας αν εμφανιστεί το ίδιο staff_id (κύκλος) με βάθος > 20. Το αναδρομικό CTE του Q13 βασίζεται στο γεγονός ότι αυτός ο trigger εγγυάται DAG (directed acyclic graph) χωρίς κύκλους.

```
CREATE OR REPLACE FUNCTION fn_doctor_supervisor_check()
RETURNS TRIGGER AS $$
DECLARE
    v_check_id INT;
    v_depth INT := 0;
    v_supervisor_rank VARCHAR(30);
    v_rank_order INT;
    v_supervisor_order INT;
BEGIN
    IF NEW.supervisor_id IS NOT NULL THEN

        -- Κυκλική αλυσίδα εποπτείας
        v_check_id := NEW.supervisor_id;
        LOOP
            SELECT supervisor_id INTO v_check_id
            FROM doctor WHERE staff_id = v_check_id;

            EXIT WHEN v_check_id IS NULL;

            v_depth := v_depth + 1;
            IF v_depth > 20 THEN
                RAISE EXCEPTION
                    'Κυκλική αλυσίδα εποπτείας ανιχνεύθηκε για staff_id=%',
                    NEW.staff_id;
            END IF;

            IF v_check_id = NEW.staff_id THEN
                RAISE EXCEPTION
                    'Κυκλική αλυσίδα εποπτείας: staff_id=% εμφανίζεται στην
```

```

αλυσίδα.',
        NEW.staff_id;
    END IF;
END LOOP;

-- Επόπτης πρέπει να είναι ανώτερης βαθμίδας
SELECT rank INTO v_supervisor_rank
FROM doctor WHERE staff_id = NEW.supervisor_id;

v_rank_order := CASE NEW.rank
    WHEN 'Ειδικευόμενος' THEN 1
    WHEN 'Επιμελητής B' THEN 2
    WHEN 'Επιμελητής A' THEN 3
    WHEN 'Διευθυντής' THEN 4
END;

v_supervisor_order := CASE v_supervisor_rank
    WHEN 'Ειδικευόμενος' THEN 1
    WHEN 'Επιμελητής B' THEN 2
    WHEN 'Επιμελητής A' THEN 3
    WHEN 'Διευθυντής' THEN 4
END;

IF v_supervisor_order <= v_rank_order THEN
    RAISE EXCEPTION
        'Ο επόπτης (rank=%) πρέπει να είναι ανώτερης βαθμίδας από τον
εποπτευόμενο (rank=%).',
        v_supervisor_rank, NEW.rank;
END IF;

END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_doctor_supervisor_check
BEFORE INSERT OR UPDATE ON doctor

```



```
FOR EACH ROW EXECUTE FUNCTION fn_doctor_supervisor_check();
```

#09 | trg_supervisor_same_department (CONSTRAINT — DEFERRABLE)

Ελέγχει ότι επόπτης και εποπτευόμενος ανήκουν στο ίδιο τμήμα. DEFERRABLE για το ίδιο λόγο με το #04 — κατά μαζική εισαγωγή νέου τμήματος, ο Διευθυντής και οι υφιστάμενοι εισάγονται σειριακά στο ίδιο transaction. Αν ο υφιστάμενος εισαχθεί πρώτα, το doctor_department του Διευθυντή δεν υπάρχει ακόμα.

```
CREATE OR REPLACE FUNCTION fn_check_supervisor_same_department()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.supervisor_id IS NULL THEN
        RETURN NEW;
    END IF;

    -- Αν ο ιατρός δεν έχει ακόμα τμήμα, παράλειψε
    IF NOT EXISTS (
        SELECT 1 FROM doctor_department WHERE doctor_id = NEW.staff_id
    ) THEN
        RETURN NEW;
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM doctor_department dd1
        JOIN doctor_department dd2 ON dd1.department_name = dd2.department_name
        WHERE dd1.doctor_id = NEW.staff_id
        AND    dd2.doctor_id = NEW.supervisor_id
    ) THEN
        RAISE EXCEPTION
            'Ο επόπτης (staff_id=%) δεν ανήκει στο ίδιο τμήμα με τον εποπτευόμενο (staff_id=%).',
            NEW.supervisor_id, NEW.staff_id;
    END IF;

    RETURN NEW;
END;
```

```

$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_supervisor_same_department
AFTER INSERT OR UPDATE OF supervisor_id ON doctor
DEFERRABLE INITIALLY DEFERRED

```

Δ.3 Ενότητα 4 — Patients

#10 | trg_patient_updated_at

Ενημερώνει αυτόματα το πεδίο updated_at κάθε φορά που τροποποιείται μία εγγραφή στον patient. Απλή ανάθεση NEW.updated_at. Εξασφαλίζει audit trail χωρίς χειροκίνητη παρέμβαση του εφαρμογιστή.

```

CREATE OR REPLACE FUNCTION fn_patient_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at := CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_patient_updated_at
BEFORE UPDATE ON patient
FOR EACH ROW EXECUTE FUNCTION fn_patient_updated_at();

```

#11 | trg_patient_discharge_check

Σκοπός: Εμποδίζει την έξοδο ασθενή αν τα δημογραφικά του δεδομένα δεν είναι πλήρη. Ο patient ορίζεται με nullable πεδία για emergency εισαγωγή. Ο έλεγχος πληρότητας αναβάλλεται σκόπιμα έως την έξοδο μέσω αυτού του trigger.

```

CREATE OR REPLACE FUNCTION fn_patient_discharge_check()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.discharge_date IS NULL OR OLD.discharge_date IS NOT NULL THEN
        RETURN NEW;
    END IF;

```

```
    PERFORM fn_patient_completeness_check(NEW.patient_id);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_patient_discharge_check
BEFORE UPDATE OF discharge_date ON hospitalization
FOR EACH ROW EXECUTE FUNCTION fn_patient_discharge_check();
```

#12 | trg_patient_prescription_check

Σκοπός: Ελέγχει ελάχιστη πληρότητα ασθενή (AMKA + ονοματεπώνυμο) πριν από συνταγογράφηση.

```
CREATE OR REPLACE FUNCTION fn_patient_prescription_check()
RETURNS TRIGGER AS $$
BEGIN
    PERFORM fn_patient_completeness_check(NEW.patient_id);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_patient_prescription_check
BEFORE INSERT ON prescription
FOR EACH ROW EXECUTE FUNCTION fn_patient_prescription_check();
```

#13 | trg_patient_amka_gr_check

Σκοπός: Εξασφαλίζει ότι ασθενής με nationality='GR' έχει πάντα AMKA.

```
CREATE OR REPLACE FUNCTION fn_patient_amka_gr_check()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.nationality = 'GR' AND NEW.amka IS NULL THEN
        IF EXISTS (SELECT 1 FROM hospitalization WHERE patient_id =
NEW.patient_id) OR
```

```

        EXISTS (SELECT 1 FROM prescription WHERE patient_id =
NEW.patient_id)
        THEN
            RAISE EXCEPTION
                'Ο ασθενής (patient_id=%) έχει ελληνική υπηκοότητα και πρέπει να
έχει ΑΜΚΑ.',
                NEW.patient_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_patient_amka_gr_check
BEFORE UPDATE OF nationality, amka ON patient
FOR EACH ROW EXECUTE FUNCTION fn_patient_amka_gr_check();

```

Δ.4 Ενότητα 5 — Triage

#14 | trg_triage_nurse_in_emergency

Ελέγχει ότι ο νοσηλευτής διαλογής ανήκει στο ΤΕΠ και βρίσκεται σε ενεργή βάρδια. Αλληλεπίδραση με view: Συμπληρώνει το v_triage_nurses — το trigger αποτρέπει εισαγωγή με νοσηλευτή εκτός ΤΕΠ, ενώ το view εμφανίζει μόνο αυτούς που επιτρέπονται.

```

CREATE OR REPLACE FUNCTION fn_triage_nurse_in_emergency()
RETURNS TRIGGER AS $$
DECLARE
    v_department_name VARCHAR(100);
BEGIN
    SELECT department_name INTO v_department_name
    FROM nurse WHERE staff_id = NEW.nurse_id;

    IF v_department_name <> 'Επείγοντα Περιστατικά' THEN
        RAISE EXCEPTION
            'Ο νοσηλευτής (staff_id=%) δεν ανήκει στα Επείγοντα Περιστατικά

```

```

(τμήμα=%).',
        NEW.nurse_id, v_department_name;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_triage_nurse_in_emergency
BEFORE INSERT OR UPDATE OF nurse_id ON triage
FOR EACH ROW EXECUTE FUNCTION fn_triage_nurse_in_emergency();

```

#15 | trg_triage_single_active

Αποτρέπει ταυτόχρονες ενεργές διαλογές για τον ίδιο ασθενή (outcome IS NULL).

```

CREATE OR REPLACE FUNCTION fn_triage_single_active()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM triage
        WHERE patient_id = NEW.patient_id
        AND outcome IS NULL
        AND triage_id <> COALESCE(NEW.triage_id, -1)
    ) THEN
        RAISE EXCEPTION
            'Ο ασθενής (patient_id=%) έχει ήδη ανοιχτό triage στην ουρά.',
            NEW.patient_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_triage_single_active
BEFORE INSERT ON triage
FOR EACH ROW EXECUTE FUNCTION fn_triage_single_active();

```

#16 | trg_triage_create_hospitalization

Αυτοματοποιεί τη ροή ΤΕΠ → Νοσηλεία — μόλις το outcome γίνει 'παραπομπή για νοσηλεία', δημιουργείται εγγραφή νοσηλείας. Αυτός ο trigger δημιουργεί κενή νοσηλεία χωρίς κλίνη ή KEN — τα συμπληρώνει η εφαρμογή στη συνέχεια. Αλληλεπίδραση με sp_triage_admission: η stored procedure καλεί τον trigger έμμεσα μέσω UPDATE triage.

```
CREATE OR REPLACE FUNCTION fn_triage_create_hospitalization()
RETURNS TRIGGER AS $$
DECLARE
    v_bed_number INT;
BEGIN
    IF NEW.outcome = 'παραπομπή για νοσηλεία' AND
        (OLD.outcome IS NULL OR OLD.outcome <> 'παραπομπή για νοσηλεία')
    THEN
        IF NOT EXISTS (
            SELECT 1 FROM department_beds
            WHERE department_name = NEW.referred_dept_name
            AND status = 'διαθέσιμη'
        ) THEN
            RAISE EXCEPTION
                'Δεν υπάρχει διαθέσιμη κλίνη στο τμήμα "%".',
                NEW.referred_dept_name;
        END IF;

        SELECT bed_id INTO v_bed_number
        FROM department_beds
        WHERE department_name = NEW.referred_dept_name
        AND status = 'διαθέσιμη'
        ORDER BY bed_id
        LIMIT 1;

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_triage_create_hospitalization
```

```
AFTER UPDATE OF outcome ON triage
FOR EACH ROW EXECUTE FUNCTION fn_triage_create_hospitalization();
```

Δ.5 Ενότητα 6 — Hospitalization

#17 | trg_hosp_bed_dept_match

Εξασφαλίζει ότι η κλίνη που ανατίθεται σε νοσηλεία ανήκει πράγματι στο τμήμα της νοσηλείας (σύνθετο FK check).

```
CREATE OR REPLACE FUNCTION fn_hosp_bed_dept_match()
RETURNS TRIGGER AS $$
DECLARE
    v_bed_dept VARCHAR(100);
BEGIN
    SELECT department_name INTO v_bed_dept
    FROM department_beds WHERE bed_id = NEW.bed_id;

    IF v_bed_dept <> NEW.department_name THEN
        RAISE EXCEPTION
            'Η κλίνη (bed_id=%) ανήκει στο τμήμα "%" – δεν ταιριάζει με τη νοσηλεία
            (department="%").',
            NEW.bed_id, v_bed_dept, NEW.department_name;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_hosp_bed_dept_match
BEFORE INSERT OR UPDATE OF bed_id, department_name ON hospitalization
FOR EACH ROW EXECUTE FUNCTION fn_hosp_bed_dept_match();
```

#18 | trg_hosp_bed_status

Διαχειρίζεται αυτόματα την κατάσταση κλίνης κατά εισαγωγή και έξοδο ασθενή. Αλληλεπίδραση με v_available_beds: Το view φιλτράρει status='διαθέσιμη'. Μόλις εκτελεστεί ο trigger, η κλίνη εξαφανίζεται/εμφανίζεται αυτόματα από το view.

```
CREATE OR REPLACE FUNCTION fn_hosp_bed_status()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        IF NEW.discharge_date IS NULL THEN
            UPDATE department_beds
            SET status = 'κατειλημμένη'
            WHERE bed_id = NEW.bed_id;
        END IF;

    ELSIF TG_OP = 'UPDATE' THEN
        UPDATE department_beds b
        SET status =
            CASE
                WHEN EXISTS (
                    SELECT 1
                    FROM hospitalization h
                    WHERE h.bed_id = (NEW.bed_id, OLD.bed_id)
                        AND h.discharge_date IS NULL
                )
                THEN 'κατειλημμένη'
                ELSE 'διαθέσιμη'
            END
        WHERE b.bed_id = (NEW.bed_id, OLD.bed_id);

    ELSIF TG_OP = 'DELETE' THEN
        UPDATE department_beds b
        SET status =
            CASE
                WHEN EXISTS (
                    SELECT 1
                    FROM hospitalization h
                    WHERE h.bed_id = OLD.bed_id
```



```

        AND h.discharge_date IS NULL
    )
    THEN 'κατειλημμένη'
    ELSE 'διαθέσιμη'
END
WHERE b.bed_id = OLD.bed_id;
END IF;

RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_hosp_bed_status
AFTER INSERT OR UPDATE ON hospitalization
FOR EACH ROW EXECUTE FUNCTION fn_hosp_bed_status();

```

#19 | trg_hosp_bed_overlap

Αποτρέπει διπλοκράτηση ελέγχει ότι η κλίνη δεν έχει ήδη ενεργή νοσηλεία (discharge_date IS NULL) σε άλλη εγγραφή. Σχέση με #18: Το #18 ενημερώνει το status, το #19 ελέγχει απευθείας στον πίνακα hospitalization, ανεξάρτητο δεύτερο επίπεδο προστασίας.

```

CREATE OR REPLACE FUNCTION fn_hosp_bed_overlap()
RETURNS TRIGGER AS $$
DECLARE
    v_overlap_count INT;
BEGIN
    SELECT COUNT(*) INTO v_overlap_count
    FROM hospitalization
    WHERE bed_id = NEW.bed_id
    AND hospitalization_id <> COALESCE(NEW.hospitalization_id, -1)
    AND admission_date <= COALESCE(NEW.discharge_date, 'infinity'::DATE)
    AND COALESCE(discharge_date, 'infinity'::DATE) >= NEW.admission_date;

    IF v_overlap_count > 0 THEN
        RAISE EXCEPTION
            'Επικάλυψη νοσηλειών: κλίνη bed_id=% είναι ήδη κατειλημμένη για αυτή την

```

```

περίοδο.',
        NEW.bed_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_hosp_bed_overlap
BEFORE INSERT OR UPDATE ON hospitalization
FOR EACH ROW EXECUTE FUNCTION fn_hosp_bed_overlap();

```

#20 | trg_hosp_discharge_diagnosis

Αποτρέπει έξοδο ασθενή χωρίς κωδικοποίηση διάγνωσης εξόδου (ICD-10) και κωδικό KEN. Σχέση με `sp_discharge_patient`: Η stored procedure παρέχει `p_discharge_icd10` και `p_ken_code` ως παραμέτρους — ο trigger τα επαληθεύει σε επίπεδο βάσης.

```

CREATE OR REPLACE FUNCTION fn_hosp_discharge_diagnosis()
RETURNS TRIGGER AS $$
DECLARE
    v_icd_exists INT;
BEGIN
    IF NEW.discharge_date IS NOT NULL AND OLD.discharge_date IS NULL THEN
        IF NEW.discharge_diag_icd10 IS NULL THEN
            RAISE EXCEPTION
                'Δεν επιτρέπεται έξοδος (hospitalization_id=%) χωρίς κωδικό ICD-10
                διάγνωσης εξόδου.',
                NEW.hospitalization_id;
        END IF;
        SELECT COUNT(*) INTO v_icd_exists
        FROM icd10_code WHERE icd10_code = NEW.discharge_diag_icd10;
        IF v_icd_exists = 0 THEN
            RAISE EXCEPTION
                'Ο κωδικός ICD-10 "%" δεν υπάρχει στον πίνακα icd10_code
                (hospitalization_id=%).',
                NEW.discharge_diag_icd10, NEW.hospitalization_id;

```

```

        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_hosp_discharge_diagnosis
BEFORE UPDATE ON hospitalization
FOR EACH ROW EXECUTE FUNCTION fn_hosp_discharge_diagnosis();

```

#21 | trg_hosp_total_cost

Υπολογίζει αυτόματα το total_cost κατά την έξοδο ασθενή. Αν παραμονή $\leq \text{ΜΔΝ}$ $\rightarrow$ total_cost = base_cost. Αν παραμονή $> \text{ΜΔΝ}$ $\rightarrow$ extra_days $\times$ daily_extra. Η GREATEST(0,...) εξασφαλίζει μηδενικό επιπλέον κόστος για σύντομες νοσηλείες. Αλληλεπίδραση: Χρησιμοποιείται από v_hospitalization_cost_breakdown και Q01.

```

CREATE OR REPLACE FUNCTION fn_hosp_total_cost()
RETURNS TRIGGER AS $$
DECLARE
    v_base_cost DECIMAL(10,2);
    v_avg_duration DECIMAL(5,2);
    v_daily_extra DECIMAL(10,2);
    v_days_stayed DECIMAL(10,2);
    v_extra_days DECIMAL(10,2);
BEGIN
    IF NEW.discharge_date IS NOT NULL AND OLD.discharge_date IS NULL THEN
        IF NEW.ken_code IS NOT NULL THEN
            SELECT base_cost, avg_duration_days, daily_extra
            INTO v_base_cost, v_avg_duration, v_daily_extra
            FROM ken_code WHERE ken_code = NEW.ken_code;

            v_days_stayed := (NEW.discharge_date -
NEW.admission_date)::DECIMAL;
            v_extra_days := GREATEST(0, v_days_stayed - v_avg_duration);
            NEW.total_cost := v_base_cost + (v_extra_days * v_daily_extra);
        ELSE
            NEW.total_cost := NULL;
        END IF;
    END IF;

```

```

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_hosp_total_cost
BEFORE UPDATE ON hospitalization
FOR EACH ROW EXECUTE FUNCTION fn_hosp_total_cost();

```

Δ.6 Ενότητα 7 — Lab & Procedures

#23 | trg_procedure_during_hosp

Εξασφαλίζει ότι η ημερομηνία εκτέλεσης ιατρικής πράξης (performed_date) εμπίπτει εντός [admission_date, discharge_date] της νοσηλείας.

```

CREATE OR REPLACE FUNCTION fn_procedure_during_hosp()
RETURNS TRIGGER AS $$
DECLARE
    v_admission_date DATE;
    v_discharge_date DATE;
BEGIN
    SELECT admission_date, discharge_date
    INTO v_admission_date, v_discharge_date
    FROM hospitalization WHERE hospitalization_id = NEW.hospitalization_id;

    IF NEW.performed_date < v_admission_date THEN
        RAISE EXCEPTION
            'Η πράξη (hosp_procedure_id=%) γίνεται πριν την εισαγωγή (%).',
            NEW.hosp_procedure_id, v_admission_date;
    END IF;

    IF v_discharge_date IS NOT NULL AND NEW.performed_date > v_discharge_date
THEN
        RAISE EXCEPTION
            'Η πράξη (hosp_procedure_id=%) γίνεται μετά την έξοδο (%).',

```

```

        NEW.hosp_procedure_id, v_discharge_date;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- FIXED
CREATE TRIGGER trg_procedure_during_hosp
BEFORE INSERT OR UPDATE ON hospitalization_procedure
FOR EACH ROW EXECUTE FUNCTION fn_procedure_during_hosp();

```

#24 | trg_lab_exam_during_hosp

Ίδια λογική με #23 για εργαστηριακές εξετάσεις (ordered_date εντός νοσηλείας).

```

CREATE OR REPLACE FUNCTION fn_lab_exam_during_hosp()
RETURNS TRIGGER AS $$
DECLARE
    v_admission_date DATE;
    v_discharge_date DATE;
BEGIN
    SELECT admission_date, discharge_date
    INTO v_admission_date, v_discharge_date
    FROM hospitalization WHERE hospitalization_id = NEW.hospitalization_id;

    IF NEW.ordered_date::DATE < v_admission_date THEN
        RAISE EXCEPTION
            'Η εξέταση (exam_id=%) γίνεται πριν την εισαγωγή (%).',
            NEW.hosp_lab_id, v_admission_date;
    END IF;

    IF v_discharge_date IS NOT NULL AND
        NEW.ordered_date::DATE > v_discharge_date THEN
        RAISE EXCEPTION
            'Η εξέταση (exam_id=%) γίνεται μετά την έξοδο (%).',
            NEW.hosp_lab_id, v_discharge_date;
    END IF;
END;

```

```

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_lab_exam_during_hosp
BEFORE INSERT OR UPDATE ON hospitalization_lab -- FIXED
FOR EACH ROW EXECUTE FUNCTION fn_lab_exam_during_hosp();

```

Δ.7 Triggers Procedures

#25 | trg_surgery_room_overlap

Αποτρέπει ταυτόχρονη χρήση της ίδιας αίθουσας χειρουργείου από δύο επεμβάσεις. Χρησιμοποιεί interval overlap check με `NEW.duration_minutes || ' minutes'::INTERVAL`. Ελέγχει και κατά UPDATE ώστε να μην μπορεί να αλλάξει η ώρα επέμβασης αν δημιουργεί σύγκρουση.

```

CREATE OR REPLACE FUNCTION fn_surgery_room_overlap()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.room IS NULL OR NEW.start_time IS NULL OR NEW.duration_minutes IS
    NULL THEN
        RETURN NEW;
    END IF;

    IF EXISTS (
        SELECT 1
        FROM hospitalization_procedure
        WHERE room = NEW.room
            AND hosp_procedure_id <> COALESCE(NEW.hosp_procedure_id, -1)
            AND start_time < (NEW.start_time + (NEW.duration_minutes || '
minutes')::INTERVAL)
            AND (start_time + (duration_minutes || ' minutes')::INTERVAL) >
NEW.start_time
        ) THEN
        RAISE EXCEPTION
            'Επικάλυψη αίθουσας: room_id=% είναι ήδη κατειλημμένη αυτή την ώρα.',

```

```

        NEW.room;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_surgery_room_overlap
BEFORE INSERT OR UPDATE ON hospitalization_procedure
FOR EACH ROW EXECUTE FUNCTION fn_surgery_room_overlap();

```

#26 | trg_surgery_doctor_overlap

Αποτρέπει τον κύριο χειρουργό (performed_by) να εκτελεί δύο ταυτόχρονες επεμβάσεις.

```

CREATE OR REPLACE FUNCTION fn_surgery_doctor_overlap()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.start_time IS NULL OR NEW.duration_minutes IS NULL THEN
        RETURN NEW;
    END IF;

    -- Έλεγχος κύριου χειρουργού (performed_by)
    IF EXISTS (
        SELECT 1
        FROM hospitalization_procedure
        WHERE performed_by = NEW.performed_by
            AND hosp_procedure_id <> COALESCE(NEW.hosp_procedure_id, -1)
            AND start_time < (NEW.start_time + (NEW.duration_minutes || '
minutes')::INTERVAL)
            AND (start_time + (duration_minutes || ' minutes')::INTERVAL) >
NEW.start_time
    ) THEN
        RAISE EXCEPTION
            'Επικάλυψη χειρουργού: ο ιατρός (staff_id=%) συμμετέχει ήδη σε άλλη
επέμβαση αυτή την ώρα.',
            NEW.performed_by;
    END IF;
END;

```

```

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_surgery_doctor_overlap
BEFORE INSERT OR UPDATE ON hospitalization_procedure
FOR EACH ROW EXECUTE FUNCTION fn_surgery_doctor_overlap();

```

#27 | trg_surgery_assistant_overlap

Αποτρέπει βοηθό ιατρό/νοσηλεύτη (procedure_participant) να συμμετέχει σε δύο ταυτόχρονες επεμβάσεις.

```

CREATE OR REPLACE FUNCTION fn_surgery_assistant_overlap()
RETURNS TRIGGER AS $$
DECLARE
    v_start_time TIMESTAMP;
    v_duration INT;
BEGIN
    SELECT start_time, duration_minutes
    INTO v_start_time, v_duration
    FROM hospitalization_procedure
    WHERE hosp_procedure_id = NEW.hosp_procedure_id;

    IF v_start_time IS NULL OR v_duration IS NULL THEN
        RETURN NEW;
    END IF;

    IF EXISTS (
        SELECT 1
        FROM procedure_participant pp
        JOIN hospitalization_procedure hp ON pp.hosp_procedure_id =
hp.hosp_procedure_id
        WHERE pp.staff_id = NEW.staff_id
        AND pp.hosp_procedure_id <> NEW.hosp_procedure_id
    )

```



```

        AND hp.start_time < (v_start_time + (v_duration || '
minutes')::INTERVAL)
        AND (hp.start_time + (hp.duration_minutes || ' minutes')::INTERVAL) >
v_start_time
    ) THEN
        RAISE EXCEPTION
            'Επικάλυψη βοηθού: ο staff (staff_id=%) συμμετέχει ήδη σε άλλη
επέμβαση αυτή την ώρα.',
            NEW.staff_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_surgery_assistant_overlap
BEFORE INSERT ON procedure_participant
FOR EACH ROW EXECUTE FUNCTION fn_surgery_assistant_overlap();

```

Δ.8 Ενότητα 8 — Prescriptions

#28 | trg_allergy_check

Το κρίσιμότερο trigger ασφαλείας — αποτρέπει συνταγογράφηση φαρμάκου που περιέχει δραστική ουσία στην οποία ο ασθενής είναι αλλεργικός. Αλυσίδα τιμής: drug → drug_substance → active_substance ↔ patient_allergy. Η χρήση INNER JOIN εξασφαλίζει ότι μόνο φάρμακα με γνωστές δραστικές ουσίες ελέγχονται. Αλληλεπίδραση με Q07 και Q10.

```

CREATE OR REPLACE FUNCTION fn_allergy_check()
RETURNS TRIGGER AS $$
DECLARE
    v_conflicting_substance VARCHAR(200);
BEGIN
    SELECT ds.active_substance_name INTO v_conflicting_substance
    FROM drug_substance ds
    JOIN patient_allergy pa ON pa.active_substance = ds.active_substance_name
    WHERE ds.drug_code = NEW.drug_code

```

```

AND pa.patient_id = NEW.patient_id
LIMIT 1;

IF FOUND THEN
    RAISE EXCEPTION
        'Απαγορευμένη συνταγογράφηση: ο ασθενής (patient_id=%) έχει αλλεργία
στη δραστική ουσία "%" (drug_code=%).',
        NEW.patient_id, v_conflicting_substance, NEW.drug_code;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_allergy_check
BEFORE INSERT ON prescription
FOR EACH ROW EXECUTE FUNCTION fn_allergy_check();

```

#29 | trg_prescription_doctor_guard

Σκοπός: Αποτρέπει ιατρό που δεν ανήκει στο hospitalization_staff της νοσηλείας να συνταγογραφήσει.

```

CREATE OR REPLACE FUNCTION fn_prescription_doctor_guard()
RETURNS TRIGGER AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM hospitalization_staff
        WHERE hospitalization_id = NEW.hospitalization_id
            AND staff_id = NEW.doctor_id
    ) THEN
        RAISE EXCEPTION
            'Μη εξουσιοδοτημένη συνταγογράφηση: ο ιατρός (staff_id=%) '
            'δεν ανήκει στο ιατρικό team της νοσηλείας (hospitalization_id=%).',
            NEW.doctor_id, NEW.hospitalization_id;
    END IF;
END;

```

```

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prescription_doctor_guard
BEFORE INSERT ON prescription
FOR EACH ROW EXECUTE FUNCTION fn_prescription_doctor_guard();

```

#30 | trg_prescription_within_hosp

Εξασφαλίζει ότι start_date και end_date συνταγής εμπίπτουν εντός του χρονικού πλαισίου νοσηλείας.

```

CREATE OR REPLACE FUNCTION fn_prescription_within_hosp()
RETURNS TRIGGER AS $$
DECLARE
    v_admission_date DATE;
    v_discharge_date DATE;
BEGIN
    SELECT admission_date, discharge_date
    INTO v_admission_date, v_discharge_date
    FROM hospitalization WHERE hospitalization_id = NEW.hospitalization_id;

    IF NEW.start_date < v_admission_date THEN
        RAISE EXCEPTION
            'Η ημερομηνία έναρξης συνταγής (%) είναι πριν την εισαγωγή (%).',
            NEW.start_date, v_admission_date;
    END IF;

    IF v_discharge_date IS NOT NULL AND NEW.end_date > v_discharge_date THEN
        RAISE EXCEPTION
            'Η ημερομηνία λήξης συνταγής (%) είναι μετά την έξοδο (%).',
            NEW.end_date, v_discharge_date;
    END IF;

```

```

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prescription_within_hosp
BEFORE INSERT ON prescription
FOR EACH ROW EXECUTE FUNCTION fn_prescription_within_hosp();

```

Δ.9 Ενότητα 9 — Shift Scheduling

Έξι triggers επιβάλλουν αυτόματα όλους τους επιχειρησιακούς κανόνες βαρδιών. Τα δύο από αυτά (#32, #33) είναι DEFERRABLE INITIALLY DEFERRED για να επιτρέπουν bulk εισαγωγή βάρδιας.

#31 | trg_shift_staff_belongs_to_department

Αποτρέπει ανάθεση μέλους σε βάρδια τμήματος στο οποίο δεν ανήκει.

```

CREATE OR REPLACE FUNCTION fn_shift_staff_belongs_to_department()
RETURNS TRIGGER AS $$
DECLARE
    v_department_name VARCHAR(100);
    v_staff_type VARCHAR(25);
BEGIN
    SELECT department_name INTO v_department_name
    FROM shifts WHERE shift_id = NEW.shift_id;

    SELECT staff_type INTO v_staff_type
    FROM staff WHERE staff_id = NEW.staff_id;

    IF v_staff_type = 'Ιατρός' THEN
        IF NOT EXISTS (
            SELECT 1 FROM doctor_department
            WHERE doctor_id = NEW.staff_id
            AND department_name = v_department_name
        ) THEN
            RAISE EXCEPTION
                'Ο ιατρός (staff_id=%) δεν ανήκει στο τμήμα %.',

```

```

        NEW.staff_id, v_department_name;
    END IF;

    ELSIF v_staff_type = 'Νοσηλεύτης' THEN
        IF NOT EXISTS (
            SELECT 1 FROM nurse
            WHERE staff_id = NEW.staff_id
            AND department_name = v_department_name
        ) THEN
            RAISE EXCEPTION
                'Ο νοσηλεύτης (staff_id=%) δεν ανήκει στο τμήμα %.',
                NEW.staff_id, v_department_name;
        END IF;

    ELSIF v_staff_type = 'Διοικητικό Προσωπικό' THEN
        IF NOT EXISTS (
            SELECT 1 FROM admin_staff
            WHERE staff_id = NEW.staff_id
            AND department_name = v_department_name
        ) THEN
            RAISE EXCEPTION
                'Ο διοικητικός (staff_id=%) δεν ανήκει στο τμήμα %.',
                NEW.staff_id, v_department_name;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_shift_staff_belongs_to_department
BEFORE INSERT ON shift_staff
FOR EACH ROW EXECUTE FUNCTION fn_shift_staff_belongs_to_department();

```

#32 | trg_shift_min_staff (DEFERRABLE)

Εξασφαλίζει ότι κάθε βάρδια έχει τουλάχιστον 3 ιατρούς, 6 νοσηλευτές και 2 διοικητικούς. Κατά bulk εισαγωγή βάρδιας, τα μέλη εισάγονται ένα-ένα. Μετά το πρώτο INSERT, η βάρδια δεν πληρεί ακόμα το ελάχιστο. Το DEFERRABLE επιτρέπει τον τελικό έλεγχο μόνο στο COMMIT.

```
CREATE OR REPLACE FUNCTION fn_shift_min_staff()
RETURNS TRIGGER AS $$
DECLARE
    v_shift_id INT;
    v_doctor_count INT;
    v_nurse_count INT;
    v_admin_count INT;
BEGIN
    v_shift_id := CASE TG_OP WHEN 'DELETE' THEN OLD.shift_id ELSE NEW.shift_id
END;

    SELECT COUNT(*) INTO v_doctor_count
    FROM shift_staff ss
    JOIN staff s ON ss.staff_id = s.staff_id
    WHERE ss.shift_id = v_shift_id
    AND s.staff_type = 'Ιατρός';

    SELECT COUNT(*) INTO v_nurse_count
    FROM shift_staff ss
    JOIN staff s ON ss.staff_id = s.staff_id
    WHERE ss.shift_id = v_shift_id
    AND s.staff_type = 'Νοσηλευτής';

    SELECT COUNT(*) INTO v_admin_count
    FROM shift_staff ss
    JOIN staff s ON ss.staff_id = s.staff_id
    WHERE ss.shift_id = v_shift_id
    AND s.staff_type = 'Διοικητικό Προσωπικό';

    IF v_doctor_count < 3 THEN
        RAISE EXCEPTION
            'Βάρδια % έχει % ιατρούς – απαιτούνται τουλάχιστον 3.',
            v_shift_id, v_doctor_count;
    END IF;
```

```

IF v_nurse_count < 6 THEN
    RAISE EXCEPTION
        'Βάρδια % έχει % νοσηλευτές – απαιτούνται τουλάχιστον 6.',
        v_shift_id, v_nurse_count;
END IF;

IF v_admin_count < 2 THEN
    RAISE EXCEPTION
        'Βάρδια % έχει % διοικητικούς – απαιτούνται τουλάχιστον 2.',
        v_shift_id, v_admin_count;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_shift_min_staff
AFTER INSERT OR DELETE ON shift_staff
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION fn_shift_min_staff();

```

#33 | trg_shift_resident_needs_senior (DEFERRABLE)

Αν υπάρχει Ειδικευόμενος στη βάρδια, πρέπει να υπάρχει τουλάχιστον ένας Επιμελητής Α ή Διευθυντής.

```

CREATE OR REPLACE FUNCTION fn_shift_resident_needs_senior()
RETURNS TRIGGER AS $$
DECLARE
    v_shift_id INT;
    v_has_resident BOOLEAN;
    v_has_senior BOOLEAN;
BEGIN
    v_shift_id := CASE TG_OP WHEN 'DELETE' THEN OLD.shift_id ELSE NEW.shift_id
END;

    SELECT EXISTS (
        SELECT 1

```

```

        FROM shift_staff ss
        JOIN doctor d ON ss.staff_id = d.staff_id
        WHERE ss.shift_id = v_shift_id
        AND d.rank = 'Ειδικευόμενος'
    ) INTO v_has_resident;

    IF v_has_resident THEN
        SELECT EXISTS (
            SELECT 1
            FROM shift_staff ss
            JOIN doctor d ON ss.staff_id = d.staff_id
            WHERE ss.shift_id = v_shift_id
            AND d.rank IN ('Επιμελητής A', 'Διευθυντής')
        ) INTO v_has_senior;

        IF NOT v_has_senior THEN
            RAISE EXCEPTION
                'Βάρδια % έχει Ειδικευόμενο αλλά δεν έχει Επιμελητή A ή Διευθυντή.',
                v_shift_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_shift_resident_needs_senior
AFTER INSERT OR DELETE ON shift_staff
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION fn_shift_resident_needs_senior();

```

#34 | trg_shift_monthly_limit

Αποτρέπει υπέρβαση μηνιαίου ορίου βαρδιών (Ιατροί: 15, Νοσηλευτές: 20, Διοικητικοί: 25).

```

CREATE OR REPLACE FUNCTION fn_shift_monthly_limit()
RETURNS TRIGGER AS $$

```



```

DECLARE
    v_shift_date DATE;
    v_shift_month INT;
    v_shift_year INT;
    v_shift_count INT;
    v_monthly_limit INT;
    v_staff_type VARCHAR(25);
BEGIN
    SELECT shift_date INTO v_shift_date
    FROM shifts WHERE shift_id = NEW.shift_id;

    v_shift_month := EXTRACT(MONTH FROM v_shift_date);
    v_shift_year := EXTRACT(YEAR FROM v_shift_date);

    SELECT staff_type INTO v_staff_type
    FROM staff WHERE staff_id = NEW.staff_id;

    v_monthly_limit := CASE v_staff_type
        WHEN 'Ιατρός' THEN 15
        WHEN 'Νοσηλεύτης' THEN 20
        WHEN 'Διοικητικό Προσωπικό' THEN 25
    END;

    SELECT COUNT(*) INTO v_shift_count
    FROM shift_staff ss
    JOIN shifts s ON ss.shift_id = s.shift_id
    WHERE ss.staff_id = NEW.staff_id
    AND EXTRACT(MONTH FROM s.shift_date) = v_shift_month
    AND EXTRACT(YEAR FROM s.shift_date) = v_shift_year;

    IF v_shift_count >= v_monthly_limit THEN
        RAISE EXCEPTION
            'Υπέρβαση μηνιαίου ορίου: staff_id=% έχει % βαρδιές τον %/% (όριο:
            %).',
            NEW.staff_id, v_shift_count, v_shift_month, v_shift_year,
            v_monthly_limit;
    END IF;

```

```

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_shift_monthly_limit
BEFORE INSERT ON shift_staff
FOR EACH ROW EXECUTE FUNCTION fn_shift_monthly_limit();

```

#35 | trg_shift_rest_8h

Εξασφαλίζει ελάχιστη ανάπαυση 8 ωρών μεταξύ διαδοχικών βαρδιών. Υπολογίζει start_time της νέας βάρδιας από shift_type ('Π'=07:00, 'Α'=15:00, 'Ν'=23:00) και shift_date. Ελέγχει και τις δύο κατευθύνσεις (πριν/μετά).

```

CREATE OR REPLACE FUNCTION fn_shift_rest_8h()
RETURNS TRIGGER AS $$
DECLARE
    v_new_date DATE;
    v_new_type VARCHAR(60);
    v_new_start TIMESTAMP;
    v_new_end TIMESTAMP;
    rec RECORD;
    v_prev_end TIMESTAMP;
    v_next_start TIMESTAMP;
BEGIN
    SELECT shift_date, shift_type INTO v_new_date, v_new_type
    FROM shifts WHERE shift_id = NEW.shift_id;

    v_new_start := CASE v_new_type
        WHEN 'Π' THEN (v_new_date::TEXT || ' 07:00')::TIMESTAMP
        WHEN 'Α' THEN (v_new_date::TEXT || ' 15:00')::TIMESTAMP
        WHEN 'Ν' THEN (v_new_date::TEXT || ' 23:00')::TIMESTAMP
    END;

    v_new_end := CASE v_new_type
        WHEN 'Π' THEN (v_new_date::TEXT || ' 15:00')::TIMESTAMP
        WHEN 'Α' THEN (v_new_date::TEXT || ' 23:00')::TIMESTAMP

```

```

        WHEN 'N' THEN ((v_new_date + INTERVAL '1 day')::TEXT || '
07:00')::TIMESTAMP
    END;

    FOR rec IN
        SELECT s.shift_date, s.shift_type
        FROM shift_staff ss
        JOIN shifts s ON ss.shift_id = s.shift_id
        WHERE ss.staff_id = NEW.staff_id AND s.shift_date BETWEEN v_new_date -
INTERVAL '2 days' AND v_new_date + INTERVAL '2 days'
    LOOP
        v_prev_end := CASE rec.shift_type
            WHEN 'Π' THEN (rec.shift_date::TEXT || ' 15:00')::TIMESTAMP
            WHEN 'Α' THEN (rec.shift_date::TEXT || ' 23:00')::TIMESTAMP
            WHEN 'Ν' THEN ((rec.shift_date + INTERVAL '1 day')::TEXT || '
07:00')::TIMESTAMP
        END;

        v_next_start := CASE rec.shift_type
            WHEN 'Π' THEN (rec.shift_date::TEXT || ' 07:00')::TIMESTAMP
            WHEN 'Α' THEN (rec.shift_date::TEXT || ' 15:00')::TIMESTAMP
            WHEN 'Ν' THEN (rec.shift_date::TEXT || ' 23:00')::TIMESTAMP
        END;

        IF v_prev_end <= v_new_start AND
            EXTRACT(EPOCH FROM (v_new_start - v_prev_end)) / 3600 < 8 THEN
            RAISE EXCEPTION
                'Παραβίαση 8ω ανάπαυσης: staff_id=% έχει μόνο % ώρες πριν τη νέα
βάρδια.',
                NEW.staff_id,
                ROUND((EXTRACT(EPOCH FROM (v_new_start - v_prev_end)) /
3600)::NUMERIC, 1);
        END IF;

        IF v_new_end <= v_next_start AND
            EXTRACT(EPOCH FROM (v_next_start - v_new_end)) / 3600 < 8 THEN
            RAISE EXCEPTION
                'Παραβίαση 8ω ανάπαυσης: staff_id=% θα έχει μόνο % ώρες μετά τη

```

```

νέα βάρδια.',
        NEW.staff_id,
        ROUND((EXTRACT(EPOCH FROM (v_next_start - v_new_end)) /
3600)::NUMERIC, 1);
    END IF;
END LOOP;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_shift_rest_8h
BEFORE INSERT ON shift_staff
FOR EACH ROW EXECUTE FUNCTION fn_shift_rest_8h();

```

#36 | trg_shift_max_3_night

Αποτρέπει πάνω από 3 συνεχόμενες νυχτερινές βάρδιες.

```

CREATE OR REPLACE FUNCTION fn_shift_max_3_night()
RETURNS TRIGGER AS $$
DECLARE
    v_new_date DATE;
    v_new_type VARCHAR(60);
    v_consecutive INT := 1;
    v_check_date DATE;
BEGIN
    SELECT shift_date, shift_type INTO v_new_date, v_new_type
    FROM shifts WHERE shift_id = NEW.shift_id;

    IF v_new_type <> 'N' THEN
        RETURN NEW;
    END IF;

    v_check_date := v_new_date - INTERVAL '1 day';

    LOOP
        EXIT WHEN v_consecutive > 3;
    
```

```

    IF EXISTS (
        SELECT 1
        FROM shift_staff ss
        JOIN shifts s ON ss.shift_id = s.shift_id
        WHERE ss.staff_id = NEW.staff_id
        AND s.shift_type = 'N'
        AND s.shift_date = v_check_date
    ) THEN
        v_consecutive := v_consecutive + 1;
        v_check_date := v_check_date - INTERVAL '1 day';
    ELSE
        EXIT;
    END IF;
END LOOP;

IF v_consecutive > 3 THEN
    RAISE EXCEPTION
        'Παραβίαση ορίου: staff_id=% θα έχει % συνεχόμενες νυχτερινές (μέγιστο
3).',
        NEW.staff_id, v_consecutive;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_shift_max_3_night
BEFORE INSERT ON shift_staff
FOR EACH ROW EXECUTE FUNCTION fn_shift_max_3_night();

```

Δ.10 Ενότητα 10 — Reviews

#37 | trg_hr_after_discharge

Αποτρέπει αξιολόγηση νοσηλείας αν δεν έχει συμπληρωθεί discharge_date.

```

CREATE OR REPLACE FUNCTION fn_hr_after_discharge()
RETURNS TRIGGER AS $$

```

```

DECLARE
    v_discharge_date DATE;
BEGIN
    SELECT discharge_date INTO v_discharge_date
    FROM hospitalization WHERE hospitalization_id = NEW.hospitalization_id;

    IF v_discharge_date IS NULL THEN
        RAISE EXCEPTION
            'Δεν επιτρέπεται αξιολόγηση νοσηλείας (hospitalization_id=%): ο
ασθενής δεν έχει εξέλθει ακόμα.',
            NEW.hospitalization_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_hr_after_discharge
BEFORE INSERT ON patient_hospitalization_review
FOR EACH ROW EXECUTE FUNCTION fn_hr_after_discharge();

```

#38 | trg_dr_after_discharge

Ίδια λογική με #37 για αξιολόγηση ιατρού.

```

CREATE OR REPLACE FUNCTION fn_dr_after_discharge()
RETURNS TRIGGER AS $$
DECLARE
    v_discharge_date DATE;
BEGIN
    SELECT discharge_date INTO v_discharge_date
    FROM hospitalization WHERE hospitalization_id = NEW.hospitalization_id;

    IF v_discharge_date IS NULL THEN
        RAISE EXCEPTION
            'Δεν επιτρέπεται αξιολόγηση ιατρού (hospitalization_id=%): ο ασθενής
δεν έχει εξέλθει ακόμα.',
            NEW.hospitalization_id;
    END IF;

```

```

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_dr_after_discharge
BEFORE INSERT ON patient_doctor_review
FOR EACH ROW EXECUTE FUNCTION fn_dr_after_discharge();

```

#39 | trg_dr_prescribed_doctor

Αποτρέπει αξιολόγηση ιατρού που δεν έχει συνταγογραφήσει στη συγκεκριμένη νοσηλεία.

```

CREATE OR REPLACE FUNCTION fn_dr_prescribed_doctor()
RETURNS TRIGGER AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM prescription
    WHERE hospitalization_id = NEW.hospitalization_id
    AND doctor_id = NEW.doctor_id;

    IF v_count = 0 THEN
        RAISE EXCEPTION
            'Δεν επιτρέπεται αξιολόγηση: ο ιατρός (doctor_id=%) δεν έχει
            συνταγογραφήσει κατά τη νοσηλεία (hospitalization_id=%).',
            NEW.doctor_id, NEW.hospitalization_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_dr_prescribed_doctor
BEFORE INSERT ON patient_doctor_review
FOR EACH ROW EXECUTE FUNCTION fn_dr_prescribed_doctor();

```

Δ.11 Ενότητα 11 — Media

#40 | trg_image_entity_check

Επαληθεύει ότι το entity_id αντιστοιχεί σε υπαρκτή εγγραφή για το δηλωθέν entity_type (polymorphic association χωρίς FK).

```
CREATE OR REPLACE FUNCTION fn_image_entity_check()
RETURNS TRIGGER AS $$
DECLARE
    v_count INT := 0;
BEGIN
    IF NEW.department_name IS NOT NULL THEN v_count := v_count + 1; END IF;
    IF NEW.staff_id IS NOT NULL THEN v_count := v_count + 1; END IF;
    IF NEW.patient_id IS NOT NULL THEN v_count := v_count + 1; END IF;
    IF NEW.room_id IS NOT NULL THEN v_count := v_count + 1; END IF;
    IF NEW.drug_code IS NOT NULL THEN v_count := v_count + 1; END IF;
    IF NEW.active_substance IS NOT NULL THEN v_count := v_count + 1; END IF;
    IF NEW.allergy_id IS NOT NULL THEN v_count := v_count + 1; END IF;
    IF NEW.bed_id IS NOT NULL THEN v_count := v_count + 1; END IF;

    IF v_count = 0 THEN
        RAISE EXCEPTION
            'Η εικόνα (image_id=%) πρέπει να ανήκει σε τουλάχιστον μία οντότητα.',
            NEW.image_id;
    END IF;

    IF v_count > 1 THEN
        RAISE EXCEPTION
            'Η εικόνα (image_id=%) δεν μπορεί να ανήκει σε περισσότερες από μία
            οντότητες (count=%).',
            NEW.image_id, v_count;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_image_entity_check
```



```
BEFORE INSERT OR UPDATE ON image
FOR EACH ROW EXECUTE FUNCTION fn_image_entity_check();
```

Κεφάλαιο Ε: Ευρετήρια (Indexes)

Η ενότητα αυτή τεκμηριώνει το σύνολο των ευρετηρίων (indexes) που ορίζονται στη βάση δεδομένων, σύμφωνα με την απαίτηση της εκφώνησης: «Να ορίσετε κατάλληλα ευρετήρια για τους πίνακες της ΒΔ και να δικαιολογήσετε την επιλογή σας με βάση τη χρησιμότητά τους για τα ερωτήματα στα οποία χρησιμοποιούνται.»

Αρχές Επιλογής: Κάθε index επιλέχθηκε βάσει τεσσάρων κριτηρίων: (α) συχνότητα εμφάνισης στηλής σε WHERE/JOIN/ORDER BY, (β) selectivity (χαμηλή cardinality → μικρό όφελος), (γ) αποδέκτης ερωτημάτων (Q01–Q15) και (δ) κόστος συντήρησης (κάθε INSERT/UPDATE ενημερώνει τα indexes).

Τεχνική Σημείωση — B-tree Indexes

Όλα τα indexes είναι B-tree (προεπιλογή PostgreSQL), κατάλληλα για: =, <, >, BETWEEN, IN, ORDER BY, LIKE 'prefix%'.

Δεν χρησιμοποιούνται GIN/GiST/Hash γιατί τα queries της βάσης δεν απαιτούν full-text search ή spatial queries.

Σημείωση: Τα PRIMARY KEY και UNIQUE constraints δημιουργούν αυτόματα B-tree index — δεν επαναλαμβάνονται εδώ.

Ε.1 Ενότητα 0 — Reference Data

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers που ωφελούνται	Αιτιολόγηση
idx_country_name	country(name)	Q01 (insurer name display), v_patient_full_profile	Αναζήτηση χώρας βάσει ονόματος (όχι κωδικού). Χωρίς index → Seq Scan σε 250+ χώρες.
idx_insurer_type	insurer(type)	Q01 (GROUP BY insurer type), v_hospitalization_cost_breakdown	Ομαδοποίηση/φιλτράρισμα ανά τύπο φορέα (Δημόσιος/Ιδιωτικός/ΕΟΧ). 4 διακριτές τιμές — χαμηλή selectivity αλλά χρήσιμο για partition scans.

Ε.2 Ενότητα 1 — Hospital Departments

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_department_floor	department(floor_building)	Αναζήτηση τμημάτων ανά όροφο	Εσωτερικές αναζητήσεις ανά φυσική θέση τμήματος.
idx_department_director	department(director_id)	trg_department_director_check, v_department_directors	Ο trigger ελέγχει director_id σε κάθε INSERT/UPDATE department — κρίσιμο για απόδοση.

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_department_specialty_specialty	department_specialty(specialty)	trg_doctor_specialty_in_department	Ο trigger διατρέχει department_specialty WHERE specialty=X σε κάθε ανάθεση ιατρού σε τμήμα.
idx_department_beds_department	department_beds(department_name)	v_available_beds, Q08, trg_hosp_bed_dept_match	Πιο συχνό φίλτρο: 'δώσε μου τις κλίνες τμήματος X'. Κρίσιμο για διαδικασία εισαγωγής νοσηλείας.
idx_department_beds_status	department_beds(status)	v_available_beds, trg_hosp_bed_status	Φίλτρο status='διαθέσιμη' χρησιμοποιείται σε κάθε φόρτωση φόρμας νοσηλείας. Χωρίς index → Seq Scan σε 400+ κλίνες.
idx_department_beds_type	department_beds(type)	v_available_beds (ORDER BY type)	Ταξινόμηση κλινών ανά τύπο (μονόκλινο/δίκλινο κ.λπ.) στο view.

E.3 Ενότητα 2 — Staff

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_staff_last_name	staff(last_name)	Αναζήτηση προσωπικού, Q08	Full-text αναζήτηση ονοματεπωνύμου στο UI. Χωρίς index → Seq Scan σε 291 εγγραφές (μικρό αλλά κρίσιμο για UX).
idx_staff_staff_type	staff(staff_type)	Q08, Q12, trg_shift_min_staff	Φιλτράρισμα ανά τύπο (Ιατρός/Νοσηλεύτης/Διοικητικό) σε πολλά queries. Μόνο 3 τιμές — χαμηλή selectivity αλλά υψηλή συχνότητα χρήσης.
idx_staff_hire_date	staff(hire_date)	Στατιστικά πρόσληψης, χρονολογικές αναφορές	Range queries ανά ημερομηνία πρόσληψης.
idx_doctor_specialty	doctor(specialty)	Q02 (WHERE specialty=X), Q05, Q12	Κεντρικό φίλτρο Q02 — αναζήτηση ιατρών ανά ειδικότητα. Χρησιμοποιείται επίσης στη φόρμα ανάθεσης τμήματος.
idx_doctor_rank	doctor(rank)	trg_doctor_supervisor_check, Q02, Q05, Q12	Έλεγχος rank σε triggers και GROUP BY ανά βαθμίδα. 4 διακριτές τιμές.

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_doctor_supervisor	doctor(supervisor_id)	Q13 (αναδρομικό CTE), trg_supervisor_same_department	Το αναδρομικό CTE Q13 διατρέπει supervisor_id επαναλαμβανόμενα — ο index επιταχύνει κάθε βήμα αναδρόμησης.
idx_doctor_department_department	doctor_department(department_name)	Q02, Q12, trg_shift_staff_belongs_to_department	JOIN doctor_department WHERE department_name=X — πολύ συχνό pattern.
idx_nurse_department	nurse(department_name)	Q12, trg_shift_staff_belongs_to_department, v_triage_nurses	Φίλτρο νοσηλευτών ανά τμήμα σε πολλά triggers και views.
idx_nurse_rank	nurse(nurse_rank)	Q12 (subcategory ανάλυση)	Ομαδοποίηση νοσηλευτών ανά βαθμίδα στο Q12.
idx_admin_staff_department	admin_staff(department_name)	Q12, trg_shift_staff_belongs_to_department	Ίδια λογική με nurse_department.
idx_admin_staff_role	admin_staff(role)	Q12 (subcategory ανάλυση)	Ομαδοποίηση διοικητικών ανά ρόλο στο Q12.

E.4 Ενότητα 3 — Pharmacy

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_drug_name	drug(name)	Searchable dropdown φαρμάκων στο UI	Full-text αναζήτηση φαρμάκου βάσει ονόματος. Κρίσιμο για UX κατά συνταγογράφηση.
idx_drug_substance_substance	drug_substance(active_substance_name)	trg_allergy_check, Q07, Q10	Ο trigger trg_allergy_check εκτελείται σε κάθε INSERT prescription: διατρέπει drug_substance WHERE active_substance_name IN (...). Χωρίς index → Seq Scan σε 40 εγγραφές × 450 prescriptions = βαρύ.

E.5 Ενότητα 4 — Patients

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_patient_last_name	patient(last_name)	Αναζήτηση ασθενών στο UI	Αναζήτηση ονόματος στη λίστα ασθενών. Χωρίς index → Seq Scan σε 310 εγγραφές.
idx_patient_city	patient(city)	Γεωγραφική ανάλυση ασθενών	Range queries ανά πόλη.
idx_patient_nationality	patient(nationality)	v_patient_full_profile JOIN country	JOIN με country βάσει nationality κώδικα.
idx_patient_insurer	patient(insurer_id)	Q01 (JOIN insurer), v_patient_full_profile	Κεντρικό JOIN Q01: patient → insurer. Εκτελείται σε κάθε οικονομική αναφορά.
idx_patient_created_at	patient(created_at)	Χρονολογικές αναφορές εγγραφής	Range scan ανά ημερομηνία εγγραφής.
idx_patient_allergy_patient	patient_allergy(patient_id)	trg_allergy_check, Q07	Κεντρικό lookup: 'ποιες αλλεργίες έχει ο ασθενής X;' Εκτελείται σε κάθε συνταγογράφηση.
idx_patient_allergy_substance	patient_allergy(active_substance)	trg_allergy_check, Q07	JOIN από active_substance → patient_allergy. Χρησιμοποιείται στην ανάστροφη αναζήτηση Q07.

E.6 Ενότητα 5 — Triage

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_triage_patient	triage(patient_id)	trg_triage_single_active, v_patient_full_profile, Q03	Ο trigger #15 εκτελεί SELECT WHERE patient_id=NEW.patient_id AND outcome IS NULL σε κάθε triage. Κρίσιμο.
idx_triage_nurse	triage(nurse_id)	trg_triage_nurse_in_emergency, v_triage_nurses	Έλεγχος βάρδιας νοσηλεύτη σε κάθε τριάζ.
idx_triage_arrival_time	triage(arrival_time)	Q15 (AVG wait time), v_triage_queue ORDER BY	Ο υπολογισμός μέσου χρόνου αναμονής Q15 διατρέχει arrival_time. Επίσης ORDER BY στο view.
idx_triage_urgency	triage(urgency_level)	Q15 (GROUP BY urgency_level), v_triage_queue ORDER BY	Το Q15 ομαδοποιεί κατά urgency_level — index επιταχύνει GROUP BY scan.

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_triage_referred_department	triage(referred_dept_name)	Q15 (GROUP BY referred_dept_name)	Ο Q15 ομαδοποιεί και κατά τμήμα παραπομπής.

E.7 Ενότητα 6 — Hospitalization

Η ενότητα Hospitalization έχει τους περισσότερους indexes λόγω της κεντρικής θέσης του πίνακα — συμμετέχει σε Q01, Q03, Q09, Q11, Q14, Q15 και σχεδόν όλα τα triggers.

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_ken_code_description	ken_code(description)	Q01 (GROUP BY description)	Full-text lookup KEN βάσει περιγραφής.
idx_icd10_category	icd10_code(category)	Q14 (GROUP BY category), v_hospitalization_cost_breakdown	Το Q14 ομαδοποιεί 5.670 ICD-10 κωδικούς ανά category — κρίσιμος index.
idx_hospitalization_patient	hospitalization(patient_id)	Q03, Q09, Q15, v_active_hospitalizations	JOIN hospitalization→patient σε σχεδόν κάθε query. Υψηλή συχνότητα χρήσης.
idx_hospitalization_bed	hospitalization(bed_id)	trg_hosp_bed_overlap, trg_hosp_bed_status	Ο trigger #19 εκτελεί SELECT WHERE bed_id=X AND discharge_date IS NULL σε κάθε εισαγωγή νοσηλείας.
idx_hospitalization_department	hospitalization(department_name)	Q01, Q03, Q12, Q15	GROUP BY department_name σε Q01, Q03 — υψηλή συχνότητα.
idx_hospitalization_admission_date	hospitalization(admission_date)	Q09, Q14, Q15, trg_procedure_during_hosp	Temporal range queries και checks — admission_date BETWEEN/> σε πολλά σημεία.
idx_hospitalization_discharge_date	hospitalization(discharge_date)	Q01 (WHERE IS NOT NULL), v_active_hospitalizations (IS NULL), Q09, Q11	Κρίσιμος index: WHERE discharge_date IS NULL για ενεργές νοσηλείες χρησιμοποιείται παντού. IS NULL scan είναι αποδοτικός με B-tree.
idx_hospitalization_ken_code	hospitalization(ken_code)	Q01 (JOIN ken_code), trg_hosp_total_cost	JOIN hospitalization→ken_code σε Q01 και trigger υπολογισμού κόστους.
idx_hospitalization_admission_icd10	hospitalization(admission_diag_icd10)	Q14 (JOIN icd10_code)	Q14 ομαδοποιεί βάσει κατηγορίας ICD-10 εισαγωγής.

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_hospitalization_discharge_icd10	hospitalization(discharge_diag_icd10)	trg_hosp_discharge_diagnosis, αναφορές εξόδου	Ο trigger ελέγχει discharge_diag_icd10 IS NULL σε κάθε έξοδο.
idx_hosp_staff_hospitalization	hospitalization_staff(hospitalization_id)	trg_prescription_doctor_guard, Q12	Lookup 'ποιο προσωπικό ανήκει σε νοσηλεία X;' — εκτελείται σε κάθε νέα συνταγή.
idx_hosp_staff_staff	hospitalization_staff(staff_id)	Q02, Q12	Lookup 'σε ποιες νοσηλείες συμμετέχει ο staff X;'
idx_hosp_staff_role	hospitalization_staff(role)	v_active_hospitalizations (WHERE role='attending_doctor')	Φίλτρο ρόλου στο view ενεργών νοσηλειών.

E.8 Ενότητα 7 — Lab & Procedures

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_medical_procedure_category	medical_procedure(category)	Q05 (WHERE category='XEIPOYP GIKEΣ')	Φίλτρο κατηγορίας πράξης στο Q05 — 5.394 εγγραφές χωρίς index = full scan.
idx_hosp_procedure_hospitalization	hospitalization_procedure(hospitalization_id)	Q05, Q11, Q12, trg_procedure_during_hosp	Κεντρικό JOIN. Επίσης ο trigger ελέγχει hospitalization_id σε κάθε εισαγωγή πράξης.
idx_hosp_procedure_doctor	hospitalization_procedure(performed_by)	Q02, Q05, Q11 (COUNT ανά ιατρό)	Τα Q02, Q05, Q11 ομαδοποιούν πράξεις ανά performed_by — κρίσιμος index για GROUP BY.
idx_hosp_procedure_date	hospitalization_procedure(performed_date)	Q11 (WHERE EXTRACT(YEAR)=2025), Q05	Το Q11 φιλτράρει βάσει έτους εκτέλεσης — χωρίς index = Seq Scan σε 334 εγγραφές × EXTRACT.
idx_lab_exam_hospitalization	hospitalization_lab(hospitalization_id)	trg_lab_exam_during_hosp, αναφορές εξετάσεων	JOIN hospitalization_lab→hospitalization σε trigger και αναφορές.
idx_lab_exam_doctor	hospitalization_lab(ordered_by)	Αναφορές παραγγελιών ανά ιατρό	Lookup εξετάσεων ανά παραγγέλλοντα ιατρό.
idx_procedure_participant_staff	procedure_participant(staff_id)	trg_surgery_assistant_overlap	Ο νέος trigger ελέγχει staff_id σε procedure_participant για επικάλυψη βοηθού.

E.9 Ενότητα 8 — Prescriptions

Ο πίνακας prescription συμμετέχει σε Q07, Q10, trg_allergy_check, trg_prescription_doctor_guard. Έχει indexes σε όλα τα FKs και τα πεδία ημερομηνιών.

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_prescription_doctor	prescription(doctor_id)	Q02 (εφημερίες+επεμβάσεις), trg_dr_prescribed_doctor	Ο trigger #39 ελέγχει EXISTS WHERE doctor_id=X AND hospitalization_id=Y σε κάθε αξιολόγηση.
idx_prescription_patient	prescription(patient_id)	Q07, Q10, trg_patient_prescription_check	JOIN prescription→patient, lookup αλλεργιών ανά ασθενή.
idx_prescription_drug	prescription(drug_code)	Q07, Q10 (ζεύγη δραστικών), trg_allergy_check	Το Q10 αναλύει drug_code για δραστικές ουσίες — κρίσιμο για CTE substance_per_prescription.
idx_prescription_hospitalization	prescription(hospitalization_id)	trg_prescription_doctor_guard, Q10	JOIN prescription→hospitalization για έλεγχο ανήκει σε team.
idx_prescription_start_date	prescription(start_date)	Αναζήτηση ενεργών συνταγών ανά ημερομηνία	Range scan: 'ποιες συνταγές ξεκινούν μετά ημερομηνία X;'
idx_prescription_end_date	prescription(end_date)	trg_prescription_within_hosp, ενεργές συνταγές	Range scan: 'ποιες συνταγές λήγουν πριν ημερομηνία X;'

E.10 Ενότητα 9 — Shifts

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_shifts_department	shifts(department_name)	Q08, Q12, trg_shift_staff_belongs_to_department	Κεντρικό φίλτρο Q08: 'βάρδιες τμήματος X ημερομηνίας Y'. JOIN με department.
idx_shifts_date	shifts(shift_date)	Q02 (EXTRACT YEAR), Q08 (=date), Q12 (BETWEEN)	Πολλά queries φιλτράρουν βάσει ημερομηνίας βάρδιας — κρίσιμος index για range scans.
idx_shifts_type	shifts(shift_type)	v_triage_nurses, v_monthly_shift_summary	Φιλτράρισμα ανά τύπο (Π/Α/Ν) στα views εφημεριών.
idx_shift_staff_staff	shift_staff(staff_id)	Q02, Q08, trg_shift_monthly_load	Triggers εφημεριών (#34, #35, #36) ελέγχουν βαρδιές μέλους

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
		mit, trg_shift_rest_8h	staff_id=X — εκτελούνται σε κάθε ανάθεση.

E.11 Ενότητα 10–11 — Reviews & Media

Index	Πίνακας, Στήλη	Ερωτήματα / Triggers	Αιτιολόγηση
idx_patient_doctor_review_doctor	patient_doctor_review(doctor_id)	v_doctor_review_avg (GROUP BY doctor_id)	GROUP BY doctor_id σε 291 ιατρούς × πολλαπλές αξιολογήσεις.
idx_image_patient	image(patient_id)	Καρτέλα ασθενή (εικόνα προφίλ)	Lookup εικόνας βάσει patient_id.
idx_image_staff	image(staff_id)	Προφίλ προσωπικού	Lookup εικόνας βάσει staff_id.
idx_image_public	image(is_public)	Φιλτράρισμα δημόσιων/ιδιωτικών εικόνων	Χαμηλή selectivity (boolean) — χρήσιμο για bitmap index scan.

E.12 Συνοπτικός Πίνακας Αντιστοίχισης Index ↔ Query

Ο παρακάτω πίνακας δείχνει ποια ερωτήματα (Q01–Q15) ωφελούνται περισσότερο από ποιους indexes:

Ερώτημα	Κρίσιμοι Indexes	Λόγος
Q01	idx_hospitalization_department, idx_hospitalization_discharge_date, idx_hospitalization_ken_code, idx_patient_insurer	GROUP BY department+year+KEN, WHERE discharge IS NOT NULL, JOIN insurer
Q02	idx_doctor_specialty, idx_shift_staff_staff, idx_shifts_date, idx_hosp_procedure_doctor	WHERE specialty=X, EXISTS shifts, COUNT procedures
Q03	idx_hospitalization_patient, idx_hospitalization_department	GROUP BY patient+department, HAVING COUNT>3
Q05	idx_hosp_procedure_doctor, idx_hosp_procedure_date, idx_medical_procedure_category	JOIN+COUNT procedures, WHERE YEAR=X, WHERE category=X
Q07	idx_patient_allergy_substance, idx_drug_substance_substance	LEFT JOIN allergy+drug_substance GROUP BY substance

Ερώτ ημα	Κρίσιμοι Indexes	Λόγος
Q08	idx_shifts_department, idx_shifts_date, idx_shift_staff_staff	NOT EXISTS subquery: shifts WHERE date=X AND dept=X
Q09	idx_hospitalization_patient, idx_hospitalization_discharge_date	CTE: GROUP BY patient+year, WHERE discharge IS NOT NULL
Q10	idx_prescription_drug, idx_prescription_patient	CTE: JOIN drug_substance, self-join substance pairs
Q11	idx_hosp_procedure_doctor, idx_hosp_procedure_date	CTE: GROUP BY performed_by, WHERE YEAR=2025
Q12	idx_shifts_date, idx_staff_staff_type, idx_doctor_rank, idx_nurse_rank	BETWEEN dates, LEFT JOINs subtypes, CASE subcategory
Q13	idx_doctor_supervisor	RECURSIVE CTE: JOIN doctor ON supervisor_id
Q14	idx_icd10_category, idx_hospitalization_admission_date	CTE: GROUP BY category+year, JOIN icd10
Q15	idx_triage_urgency, idx_triage_arrival_time, idx_hospitalization_discharge_date	GROUP BY urgency+dept, AVG arrival_time, LEFT JOIN hosp

Κεφάλαιο ΣΤ: Ερωτήματα SQL

Παραδοχές για τα Ερωτήματα

Οι παρακάτω παραδοχές αφορούν την ερμηνεία διφορούμενων όρων στα Q1–Q15.

Ερώτημα	Παραδοχή
Q1	Η «πρόσθετη χρέωση» υπολογίζεται ως $(actual_days - avg_duration_days) \times (base_cost / avg_duration_days)$ όταν $actual_days > avg_duration_days$, αλλιώς 0.
Q4	Η «Συνολική εντύπωση νοσηλείας» αντλείται από <code>patient_hospitalization_review.overall_score</code> των νοσηλείων όπου ο ιατρός είχε συνταγογραφήσει.
Q5	«Νέοι ιατροί <35 ετών»: η ηλικία υπολογίζεται από <code>date_of_birth</code> με reference date την τρέχουσα.
Q6	Ο «μέσος όρος αξιολόγησης» = ΜΟ όλων των κριτηρίων (<code>nursing_care</code> , <code>cleanliness</code> , <code>food</code> , <code>overall</code>) από <code>patient_hospitalization_review</code> .
Q10	Δύο φάρμακα θεωρούνται «ταυτόχρονα» όταν έχουν επικαλυπτόμενο διάστημα [<code>start_date</code> , <code>end_date</code>] στην ίδια νοσηλεία.
Q11	«Τρέχον έτος» = έτος της τρέχουσας ημερομηνίας (<code>EXTRACT(YEAR FROM CURRENT_DATE)</code>).
Q14	Ως «κατηγορία ICD-10» χρησιμοποιείται το πεδίο <code>category</code> του <code>icd10_code</code> , όχι το πρώτο γράμμα του κωδικού.
Q15	Ο «χρόνος αναμονής» <code>triage</code> = <code>admission_date - arrival_time</code> για όσους νοσηλεύτηκαν.

Q01: Έσοδα ανά Τμήμα, Έτος και KEN Κωδικός

Σκοπός

Το ερώτημα υπολογίζει τα συνολικά έσοδα του νοσοκομείου με τριπλή ανάλυση: ανά τμήμα, ανά έτος και ανά KEN κωδικό. Παράλληλα αναλύει την πρόσθετη χρέωση λόγω υπέρβασης της μέσης διάρκειας νοσηλείας (ΜΔΝ) και κατανέμει τις νοσηλείες ανά ασφαλιστικό φορέα, παρέχοντας πλήρη οικονομική εικόνα του νοσοκομείου.

Κώδικας SQL

```
SELECT
    h.department_name,
    EXTRACT(YEAR FROM h.discharge_date)::INT      AS year,
    h.ken_code,
    k.description                                  AS ken_description,
    i.name                                          AS insurer_name,
```

```

        i.type                                AS insurer_type,
        k.base_cost                           AS ken_base_cost,
        SUM(
            GREATEST(0, (h.discharge_date - h.admission_date) -
k.avg_duration_days)
            * k.daily_extra
        )                                     AS total_extra_charge,
        COUNT(h.hospitalization_id)          AS hosp_count,
        SUM(h.total_cost)                     AS total_revenue

FROM hospitalization h
JOIN patient p      ON h.patient_id      = p.patient_id
JOIN insurer i      ON p.insurer_id      = i.insurer_id
JOIN ken_code k      ON h.ken_code        = k.ken_code
WHERE h.discharge_date IS NOT NULL
GROUP BY
    h.department_name,
    EXTRACT(YEAR FROM h.discharge_date),
    h.ken_code,
    k.description,
    k.base_cost,
    k.avg_duration_days,
    k.daily_extra,
    i.name,
    i.type
ORDER BY
    year DESC,
    h.department_name,
    total_revenue DESC;

```

Ανάλυση Επίλυσης

Joins:

Συνδέεται ο hospitalization με patient (για τον ασφαλιστικό φορέα), insurer (για τον τύπο και το όνομα φορέα) και ken_code (για τα οικονομικά δεδομένα KEN). Όλα τα JOIN είναι INNER, ώστε να εμφανίζονται μόνο νοσηλείες με πλήρη δεδομένα.

GREATEST(0, ...):

Υπολογίζει τις επιπλέον ημέρες πέρα από τη μέση διάρκεια. Το GREATEST εξασφαλίζει ότι αν η παραμονή είναι εντός ΜΔΝ, το αποτέλεσμα είναι 0 και όχι αρνητικό.

WHERE discharge_date IS NOT NULL:

Φιλτράρει μόνο ολοκληρωμένες νοσηλείες. Ενεργές νοσηλείες χωρίς ημερομηνία εξόδου εξαιρούνται από την οικονομική ανάλυση.

GROUP BY:

Η ομαδοποίηση γίνεται σε όλα τα μη-συγκεντρωτικά πεδία, συμπεριλαμβανομένων των k.base_cost, k.avg_duration_days, k.daily_extra που χρειάζονται για τον υπολογισμό αλλά δεν αποτελούν aggregate functions.

ORDER BY year DESC:

Τα αποτελέσματα ταξινομούνται από το πιο πρόσφατο έτος, διευκολύνοντας την ανάλυση των τελευταίων οικονομικών δεδομένων.

Q02: Ιατροί ανά Ειδικότητα με Εφημερίες και Επεμβάσεις

Σκοπός

Το ερώτημα εντοπίζει όλους τους ιατρούς συγκεκριμένης ειδικότητας (παράμετρος), δείχνει αν είχαν εφημερία στο τρέχον έτος και μετρά πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί. Αποτελεί εργαλείο διοίκησης ανθρώπινου δυναμικού.

Κώδικας SQL

```
SELECT
    st.staff_id,
    st.first_name,
    st.last_name,
    doc.specialty,
    doc.rank,
    CASE
        WHEN EXISTS (
            SELECT 1
            FROM shift_staff ss
            JOIN shifts sh ON ss.shift_id = sh.shift_id
            WHERE ss.staff_id = st.staff_id
              AND EXTRACT(YEAR FROM sh.shift_date) = EXTRACT(YEAR FROM
CURRENT_DATE)
        ) THEN 'NAI'
        ELSE 'OXI'
    END AS had_shift_this_year,
```

```

COUNT(hp.hosp_procedure_id) AS lead_doctor_procedures
FROM staff st
JOIN doctor doc ON st.staff_id = doc.staff_id
LEFT JOIN hospitalization_procedure hp ON hp.performed_by = st.staff_id
WHERE doc.specialty = 'Καρδιολογία' -- ειδικότητα εδώ
GROUP BY
    st.staff_id,
    st.first_name,
    st.last_name,
    doc.specialty,
    doc.rank
ORDER BY
    lead_doctor_procedures DESC,
    st.last_name;

```

Ανάλυση Επίλυσης

EXISTS (correlated subquery):

Ελέγχει αν ο ιατρός έχει εφημερία στο τρέχον έτος χρησιμοποιώντας CURRENT_DATE, χωρίς να κάνει COUNT, αρκεί η ύπαρξη μίας εγγραφής για αποτέλεσμα «ΝΑΙ». Είναι αποδοτικότερο από LEFT JOIN + CASE.

LEFT JOIN hospitalization_procedure:

Χρησιμοποιείται LEFT JOIN ώστε να εμφανίζονται και ιατροί που δεν έχουν εκτελέσει καμία επέμβαση (COUNT = 0). INNER JOIN θα τους απέκλειε.

CASE WHEN EXISTS ... THEN:

Μετατρέπει boolean αποτέλεσμα σε εμφανίσιμη ετικέτα «ΝΑΙ»/«ΟΧΙ» για τον χρήστη.

Q03: Ασθενείς με Πολλαπλές Νοσηλείες στο Ίδιο Τμήμα

Σκοπός

Εντοπίζει ασθενείς που έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, εμφανίζοντας τον συνολικό αριθμό νοσηλείων και το συνολικό κόστος. Αποτελεί εργαλείο εντοπισμού χρόνιων ασθενών υψηλής χρήσης υπηρεσιών.

Κώδικας SQL

```
SELECT
```

```

    pt.patient_id,
    pt.first_name,
    pt.last_name,
    hp.department_name,
    COUNT(hp.hospitalization_id) AS hosp_count,
    SUM(hp.total_cost)          AS total_cost
FROM hospitalization hp
JOIN patient pt ON hp.patient_id = pt.patient_id
WHERE hp.discharge_date IS NOT NULL
GROUP BY
    pt.patient_id,
    pt.first_name,
    pt.last_name,
    hp.department_name
HAVING COUNT(hp.hospitalization_id) > 3
ORDER BY
    hosp_count DESC,
    pt.last_name;

```

Ανάλυση Επίλυσης

HAVING COUNT > 3:

Το φίλτρο τοποθετείται σε HAVING και όχι σε WHERE επειδή αφορά αποτέλεσμα aggregate function. Εφαρμόζεται μετά την ομαδοποίηση, κρατώντας μόνο συνδυασμούς ασθενή-τμήματος με περισσότερες από 3 νοσηλείες.

GROUP BY ασθενής + τμήμα:

Η ομαδοποίηση γίνεται ανά ζεύγος (patient_id, department_name) ώστε ο ίδιος ασθενής να μπορεί να εμφανίζεται σε διαφορετικά τμήματα ανεξάρτητα.

WHERE discharge_date IS NOT NULL:

Εξαιρούνται ενεργές νοσηλείες για ακριβή μέτρηση ολοκληρωμένων επισκέψεων.

Q04: Ερώτημα με EXPLAIN/EXPLAIN ANALYZE

Σκοπός

Ανάλυση εκτέλεσης (execution plan) ενός σύνθετου ερωτήματος με χρήση EXPLAIN ANALYZE για βελτιστοποίηση απόδοσης. Εμφανίζει κόστος εκτέλεσης, αριθμό εγγραφών και χρόνο εκτέλεσης για κάθε κόμβο του query plan.

Κώδικας SQL

```
--(α) το αποτέλεσμα EXPLAIN/EXPLAIN ANALYZE
EXPLAIN ANALYZE
SELECT
st.staff_id,
st.first_name,
st.last_name,
doc.rank,
h.discharge_date,
hr.overall_score,
ROUND(AVG(pr.medical_care_score)OVER(PARTITION BY doc.staff_id), 2) AS
doc_review,
COUNT(pr.doctor_review_id)OVER (PARTITION BY doc.staff_id) AS review_count
FROM doctor doc
JOIN staff st ON doc.staff_id = st.staff_id
JOIN hospitalization_staff hs ON doc.staff_id = hs.staff_id
JOIN hospitalization h ON hs.hospitalization_id = h.hospitalization_id
LEFT JOIN patient_doctor_review pr
    ON doc.staff_id = pr.doctor_id
    AND h.hospitalization_id = pr.hospitalization_id
LEFT JOIN patient_hospitalization_review hr
    ON h.hospitalization_id = hr.hospitalization_id
WHERE h.discharge_date IS NOT NULL
    AND doc.staff_id = 80
ORDER BY review_count DESC;

--(β) εναλλακτική έκδοση με FORCE INDEX ή hint
SET enable_seqscan = OFF;
EXPLAIN ANALYZE
SELECT
st.staff_id,
st.first_name,
st.last_name,
doc.rank,
h.discharge_date,
hr.overall_score,
ROUND(AVG(pr.medical_care_score)OVER(PARTITION BY doc.staff_id), 2) AS
```



```

doc_review,
COUNT(pr.doctor_review_id)OVER (PARTITION BY doc.staff_id) AS review_count
FROM doctor doc
JOIN staff st ON doc.staff_id = st.staff_id
JOIN hospitalization_staff hs ON doc.staff_id = hs.staff_id
JOIN hospitalization h ON hs.hospitalization_id = h.hospitalization_id
LEFT JOIN patient_doctor_review pr
    ON doc.staff_id = pr.doctor_id
    AND h.hospitalization_id = pr.hospitalization_id
LEFT JOIN patient_hospitalization_review hr
    ON h.hospitalization_id = hr.hospitalization_id
WHERE h.discharge_date IS NOT NULL
    AND doc.staff_id = 80
ORDER BY review_count DESC;

SET enable_seqscan = ON;

```

Q05: Νέοι Ιατροί (< 35 ετών) με τις Περισσότερες Χειρουργικές Επεμβάσεις

Σκοπός

Εντοπίζει τους ιατρούς ηλικίας κάτω των 35 ετών που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί. Αποτελεί εργαλείο αξιολόγησης νέου ιατρικού προσωπικού.

Κώδικας SQL

```

SELECT
    st.staff_id,
    st.first_name,
    st.last_name,
    EXTRACT(YEAR FROM AGE(st.date_of_birth))::INT AS age,
    doc.rank,
    COUNT(hp.hosp_procedure_id) AS procedures_count
FROM staff st
JOIN doctor doc ON st.staff_id = doc.staff_id
JOIN hospitalization_procedure hp ON hp.performed_by = doc.staff_id

```

```
JOIN medical_procedure mp ON hp.procedure_code = mp.procedure_code
WHERE EXTRACT(YEAR FROM AGE(st.date_of_birth)) < 35
      AND mp.category = 'ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ'
GROUP BY
    st.staff_id,
    st.first_name,
    st.last_name,
    st.date_of_birth,
    doc.rank
ORDER BY
    procedures_count DESC,
    age ASC;
```

Ανάλυση Επίλυσης

EXTRACT(YEAR FROM AGE(...)):

Υπολογίζει την τρέχουσα ηλικία δυναμικά. Η AGE() επιστρέφει διάστημα (interval) και το EXTRACT(YEAR) εξάγει τα έτη, εξασφαλίζοντας σωστή σύγκριση ακόμη και αν η ηλικία αλλάξει μετά την ημερομηνία γέννησης.

JOIN medical_procedure:

Απαραίτητο για φιλτράρισμα κατά κατηγορία επέμβασης. Χρησιμοποιείται INNER JOIN ώστε να μετρούνται μόνο χειρουργικές επεμβάσεις.

ORDER BY procedures_count DESC, age ASC:

Πρωτεύουσα ταξινόμηση κατά αριθμό επεμβάσεων (περισσότερες πρώτα) και δευτερεύουσα κατά ηλικία (νεότεροι πρώτα) για ισοβαθμίες.

Q06: Ερώτημα με EXPLAIN/EXPLAIN ANALYZE - Δεύτερο

Σκοπός

Δεύτερη ανάλυση εκτέλεσης — εφαρμόστε EXPLAIN ANALYZE σε ένα ερώτημα που περιλαμβάνει CTEs ή window functions (π.χ. Q15 ή Q13) για σύγκριση κόστους εκτέλεσης.

Κώδικας SQL

```
--α) EXPLAIN/EXPLAIN ANALYZE
EXPLAIN ANALYZE
SELECT
    p.patient_id,
```

```

    p.first_name,
    p.last_name,
    h.hospitalization_id,
    h.admission_date,
    h.discharge_date,
    h.admission_diag_icd10,
    h.discharge_diag_icd10,
    h.total_cost,
    hr.overall_score,
    ROUND(AVG(hr.overall_score) OVER (PARTITION BY p.patient_id), 2) AS
avg_review,
    COUNT(h.hospitalization_id) OVER (PARTITION BY p.patient_id)      AS
hosp_count
FROM patient p
JOIN hospitalization h ON p.patient_id = h.patient_id
LEFT JOIN patient_hospitalization_review hr
    ON h.hospitalization_id = hr.hospitalization_id
WHERE h.discharge_date IS NOT NULL
    AND p.patient_id = 101
ORDER BY h.admission_date DESC;

--β) εναλλακτική έκδοση με FORCE INDEX ή hint,
SET enable_seqscan = OFF;
EXPLAIN ANALYZE
SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    h.hospitalization_id,
    h.admission_date,
    h.discharge_date,
    h.admission_diag_icd10,
    h.discharge_diag_icd10,
    h.total_cost,
    hr.overall_score,

    ROUND(AVG(hr.overall_score) OVER (PARTITION BY p.patient_id), 2) AS
avg_review,

```

```

COUNT(h.hospitalization_id) OVER (PARTITION BY p.patient_id) AS
hosp_count
FROM patient p
JOIN hospitalization h ON p.patient_id = h.patient_id
LEFT JOIN patient_hospitalization_review hr
    ON h.hospitalization_id = hr.hospitalization_id
WHERE h.discharge_date IS NOT NULL
    AND p.patient_id = 101
ORDER BY h.admission_date DESC;

SET enable_seqscan = ON;

```

Q07: Δραστικές Ουσίες: Αλλεργίες Ασθενών και Φάρμακα

Σκοπός

Για κάθε δραστική ουσία, το ερώτημα μετρά πόσοι ασθενείς έχουν δηλώσει αλλεργία και πόσα φάρμακα την περιέχουν. Ταξινομεί κατά πλήθος αλλεργικών ασθενών, εντοπίζοντας τις πιο προβληματικές δραστικές ουσίες για ασφαλή συνταγογράφηση.

Κώδικας SQL

```

SELECT
    a.name,
    COUNT(DISTINCT pa.patient_id) AS patient_count,
    COUNT(DISTINCT ds.drug_code) AS drug_count
FROM active_substance a
LEFT JOIN patient_allergy pa ON a.name = pa.active_substance
LEFT JOIN drug_substance ds ON a.name = ds.active_substance_name
GROUP BY a.name
ORDER BY patient_count DESC;

```

Ανάλυση Επίλυσης

LEFT JOIN (×2):

Και τα δύο JOIN είναι LEFT ώστε να εμφανίζονται όλες οι δραστικές ουσίες, ακόμη και αν δεν έχει δηλωθεί αλλεργία σε αυτές ή δεν υπάρχει φάρμακο που να τις περιέχει. Αυτό επιτρέπει πλήρη κατάλογο.

COUNT(DISTINCT ...):

Χρησιμοποιείται DISTINCT σε αμφότερα τα COUNT γιατί η διπλή LEFT JOIN μπορεί να προκαλέσει

πολλαπλασιασμό γραμμών (cartesian effect). Χωρίς DISTINCT, ένας ασθενής με 5 φάρμακα θα μετρηθεί 5 φορές.

Q08: Προσωπικό χωρίς Εφημερία σε Συγκεκριμένη Ημερομηνία και Τμήμα

Σκοπός

Εντοπίζει το σύνολο του προσωπικού (ιατροί, νοσηλευτές, διοικητικοί) που δεν έχουν προγραμματισμένη εφημερία σε δεδομένη ημερομηνία και τμήμα. Αποτελεί εργαλείο διαχείρισης διαθεσιμότητας προσωπικού.

Κώδικας SQL

```
SELECT      st.staff_id, st.first_name, st.last_name, st.staff_type
FROM staff st
WHERE NOT EXISTS (
    SELECT 1
    FROM shift_staff ss
    JOIN shifts sh ON ss.shift_id = sh.shift_id
    WHERE ss.staff_id      = st.staff_id
          AND sh.shift_date = '2026-03-10'      -- η ημερομηνία
          AND sh.department_name = 'Καρδιολογία' -- το τμήμα
) ORDER BY st.staff_type, st.last_name;
```

Ανάλυση Επίλυσης

NOT EXISTS (correlated subquery):

Για κάθε μέλος προσωπικού ελέγχει αν υπάρχει εφημερία στη δεδομένη ημερομηνία και τμήμα. NOT EXISTS επιστρέφει αληθές αν δεν βρεθεί καμία εγγραφή. Είναι ισοδύναμο με LEFT JOIN ... WHERE IS NULL αλλά πιο εκφραστικό.

Χωρίς φίλτρο staff_type:

Το ερώτημα επιστρέφει όλο το προσωπικό ανεξαρτήτως κατηγορίας (ιατροί, νοσηλευτές, διοικητικοί), με ταξινόμηση κατά staff_type για εύκολη ανάγνωση.

Q09: Ασθενείς με Ίδιο Αριθμό Νοσηλευτικών Ημερών

Σκοπός

Εντοπίζει ασθενείς που νοσηλεύτηκαν τον ίδιο συνολικό αριθμό ημερών μέσα στο ίδιο έτος, με συνολική διάρκεια άνω των 15 ημερών. Χρησιμοποιείται για στατιστική ανάλυση και εύρεση ομοιοτήτων προτύπων νοσηλείας.

Κώδικας SQL

```
WITH hosp_days AS (  
    SELECT  
        p.patient_id, p.first_name, p.last_name,  
        SUM((h.discharge_date - h.admission_date)::INT) AS total_days,  
        EXTRACT(YEAR FROM h.discharge_date)::INT AS year  
    FROM hospitalization h  
    JOIN patient p ON h.patient_id = p.patient_id  
    WHERE h.discharge_date IS NOT NULL  
    GROUP BY p.patient_id, p.first_name, p.last_name, year  
    HAVING SUM((h.discharge_date - h.admission_date)::INT) > 15  
)  
SELECT hd1.patient_id, hd1.first_name, hd1.last_name, hd1.total_days, hd1.year  
FROM hosp_days hd1  
WHERE EXISTS (  
    SELECT 1  
    FROM hosp_days hd2  
    WHERE hd1.total_days = hd2.total_days  
        AND hd1.patient_id <> hd2.patient_id  
        AND hd1.year = hd2.year  
) ORDER BY hd1.year DESC, hd1.total_days;
```

Ανάλυση Επίλυσης

CTE hosp_days:

Υπολογίζει πρώτα το σύνολο ημερών ανά ασθενή ανά έτος. Η χρήση CTE απλοποιεί τη δομή και επιτρέπει αναφορά στο ίδιο CTE δύο φορές στο κυρίως ερώτημα (self-join μέσω EXISTS).

HAVING > 15:

Φιλτράρει εγγραφές που δεν πληρούν το κατώφλι ημερών ήδη από το CTE, μειώνοντας το πλήθος εγγραφών πριν το EXISTS.

EXISTS με hd1.patient_id <> hd2.patient_id:

Εξασφαλίζει ότι κάθε ασθενής συγκρίνεται μόνο με διαφορετικούς ασθενείς, αποτρέπει αυτοσύγκριση.

Q10: Top-3 Ζεύγη Δραστικών Ουσιών που Συνταγογραφήθηκαν Μαζί

Σκοπός

Εντοπίζει τα 3 πιο συχνά ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία. Αποτελεί εργαλείο φαρμακοεπαγρύπνησης για εντοπισμό πιθανών αλληλεπιδράσεων φαρμάκων.

Κώδικας SQL

```
WITH substance_per_prescription AS (  
    SELECT pr.patient_id, pr.hospitalization_id, pr.drug_code,  
           ds.active_substance_name AS substance  
    FROM prescription pr  
    JOIN drug_substance ds ON pr.drug_code = ds.drug_code  
),  
substance_pairs AS (  
    SELECT sp1.substance AS substance_a,  
           sp2.substance AS substance_b,  
           sp1.patient_id, sp2.hospitalization_id  
    FROM substance_per_prescription sp1  
    JOIN substance_per_prescription sp2  
      ON sp1.patient_id = sp2.patient_id  
     AND sp1.hospitalization_id = sp2.hospitalization_id  
     AND sp1.substance < sp2.substance  
)  
SELECT substance_a, substance_b, COUNT(*) AS frequency  
FROM substance_pairs  
GROUP BY substance_a, substance_b  
ORDER BY frequency DESC  
LIMIT 3;
```

Ανάλυση Επίλυσης

CTE **substance_per_prescription**:

Αποσυνθέτει κάθε συνταγή στις δραστικές ουσίες που περιέχει μέσω JOIN με drug_substance. Ένα φάρμακο με 3 ουσίες δίνει 3 εγγραφές.

Self-join με **sp1.substance < sp2.substance**:

Δημιουργεί μοναδικά ζεύγη (A, B) χωρίς αντίστροφα (B, A). Η συνθήκη < εξασφαλίζει αλφαβητική σειρά και αποτρέπει διπλές καταχωρίσεις.

LIMIT 3:

Επιστρέφει μόνο τα 3 πιο συχνά ζεύγη μετά ταξινόμηση κατά frequency DESC.

Q11: Ιατροί με Τουλάχιστον 5 Λιγότερες Επεμβάσεις από τον Πρώτο

Σκοπός

Εντοπίζει ιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος (στην υλοποίηση, χρησιμοποιείται το 2025 για εμφάνιση αποτελεσμάτων). Αποτελεί εργαλείο ανάλυσης φόρτου εργασίας και αξιολόγησης αποδοτικότητας.

Κώδικας SQL

```
WITH procedure_count AS (  
    SELECT hp.performed_by AS doctor_id,  
           COUNT(hp.hosp_procedure_id) AS num_of_procedures  
    FROM hospitalization_procedure hp  
    WHERE EXTRACT(YEAR FROM hp.performed_date) = '2025'  
    GROUP BY hp.performed_by  
) ,  
max_procedures AS (  
    SELECT MAX(num_of_procedures) AS max_count  
    FROM procedure_count  
)  
SELECT  
    pc.doctor_id, s.first_name, s.last_name, d.rank,  
    pc.num_of_procedures, mp.max_count,  
    (mp.max_count - pc.num_of_procedures) AS difference  
FROM procedure_count pc  
JOIN max_procedures mp ON TRUE  
JOIN staff s ON pc.doctor_id = s.staff_id  
JOIN doctor d ON pc.doctor_id = d.staff_id  
WHERE (mp.max_count - pc.num_of_procedures) >= 5  
ORDER BY pc.num_of_procedures DESC;
```

Ανάλυση Επίλυσης

Δύο CTEs σε αλυσίδα:

Το procedure_count υπολογίζει τις επεμβάσεις ανά ιατρό, και το max_procedures διαβάζει τη μέγιστη τιμή από το πρώτο CTE. Η αλυσιδωτή δομή διαχωρίζει την υπολογιστική λογική.

JOIN max_procedures ON TRUE:

Cross join με ένα CTE που επιστρέφει ακριβώς μία γραμμή. Εξυπηρετεί τη διανομή της τιμής max_count σε κάθε γραμμή χωρίς subquery στο SELECT.

WHERE difference >= 5:

Φιλτράρει μετά τη σύνδεση, κρατώντας μόνο ιατρούς με διαφορά >= 5 επεμβάσεις από τον πρώτο.

Το `WHERE EXTRACT(YEAR FROM hp.performed_date) = '2025'` αντί για `EXTRACT(YEAR FROM CURRENT_DATE)` είναι σκόπιμο για αποτελέσματα ως επιλογή σχεδιασμού.

Q12: Κατανομή Προσωπικού ανά Τμήμα, Βάρδια και Υποκατηγορία

Σκοπός

Εμφανίζει τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκατηγορία: ιατροί ανά ειδικότητα, νοσηλευτές ανά βαθμίδα, διοικητικό προσωπικό ανά ρόλο. Αποτελεί εργαλείο προγραμματισμού βαρδιών.

Κώδικας SQL

```
SELECT
    sh.department_name, sh.shift_date, sh.shift_type,
    s.staff_type,
    CASE
        WHEN doc.staff_id IS NOT NULL THEN doc.specialty
        WHEN n.staff_id IS NOT NULL THEN n.nurse_rank
        WHEN a.staff_id IS NOT NULL THEN a.role
    END AS subcategory,
    COUNT(ss.staff_id) AS staff_count
FROM shifts sh
JOIN shift_staff ss ON sh.shift_id = ss.shift_id
JOIN staff s ON ss.staff_id = s.staff_id
LEFT JOIN doctor doc ON s.staff_id = doc.staff_id
LEFT JOIN nurse n ON s.staff_id = n.staff_id
LEFT JOIN admin_staff a ON s.staff_id = a.staff_id
WHERE sh.shift_date BETWEEN '2025-03-03' AND '2025-03-09'
GROUP BY sh.department_name, sh.shift_date, sh.shift_type, s.staff_type,
subcategory
ORDER BY sh.shift_date, sh.department_name, sh.shift_type, s.staff_type,
subcategory;
```

Ανάλυση Επίλυσης

LEFT JOIN σε doctor, nurse, admin_staff:

Ο πίνακας staff υλοποιεί Table Inheritance Pattern — κάθε μέλος είναι εγγεγραμμένο στον staff αλλά και σε έναν από τους εξειδικευμένους πίνακες. Τα LEFT JOIN επιτρέπουν την ταυτόχρονη ανάκτηση δεδομένων και από τους τρεις τύπους.

CASE WHEN IS NOT NULL:

Εντοπίζει σε ποιον υποπίνακα ανήκει κάθε μέλος και επιστρέφει την κατάλληλη υποκατηγορία (specialty / nurse_rank / role). Μόνο ένα από τα τρία JOIN θα επιστρέψει εγγραφή για κάθε staff_id.

BETWEEN για εβδομάδα:

Φιλτράρει συγκεκριμένη εβδομάδα. Παράμετρος που μπορεί εύκολα να τροποποιηθεί.

Q13: Ιεραρχία Εποπτείας Ιατρών (Αναδρομικό CTE)

Σκοπός

Αναπαράγει ολόκληρη την ιεραρχία εποπτείας για κάθε ιατρό, από τον ίδιο ως την κορυφή (Διευθυντής), με ένδειξη επιπέδου σε κάθε βαθμίδα. Αποτελεί εργαλείο οργανωτικής δομής και εντοπισμού αλυσίδων αναφοράς.

Κώδικας SQL

```
WITH RECURSIVE supervision_hierarchy AS (
  -- Base case: κάθε γιατρός ως αφετηρία
  SELECT d.staff_id AS doctor_id,
         s.first_name || ' ' || s.last_name AS doctor_name,
         d.rank AS doctor_rank, d.supervisor_id,
         0 AS level, d.staff_id AS root_id
  FROM doctor d JOIN staff s ON d.staff_id = s.staff_id
  UNION ALL
  -- Recursive: ανέβα στην ιεραρχία
  SELECT sup.staff_id, ss.first_name || ' ' || ss.last_name,
         sup.rank, sup.supervisor_id, h.level + 1, h.root_id
  FROM supervision_hierarchy h
  JOIN doctor sup ON h.supervisor_id = sup.staff_id
  JOIN staff ss ON sup.staff_id = ss.staff_id
  WHERE h.supervisor_id IS NOT NULL
)
SELECT h.root_id AS doctor_id,
```

```

rs.first_name || ' ' || rs.last_name AS doctor_name,
rd.rank AS doctor_rank, h.level,
CASE h.level
    WHEN 0 THEN 'Ο ίδιος'
    WHEN 1 THEN 'Άμεσος Επόπτης'
    ELSE      'Επόπτης Επιπέδου ' || h.level
END AS level_description,
h.doctor_id AS supervisor_id, h.doctor_name AS supervisor_name,
h.doctor_rank AS supervisor_rank
FROM supervision_hierarchy h
JOIN doctor rd ON h.root_id = rd.staff_id
JOIN staff rs ON h.root_id = rs.staff_id
ORDER BY h.root_id, h.level;

```

Ανάλυση Επίλυσης

WITH RECURSIVE - Base case:

Κάθε ιατρός εισάγεται ως level=0, root_id=staff_id. Αποτελεί την αφετηρία της αναδρόμησης.

UNION ALL - Recursive step:

Για κάθε εγγραφή στο τρέχον αποτέλεσμα, αναζητά τον επόπτη (supervisor_id) στον πίνακα doctor. Αυξάνει το level κατά 1. Η αναδρόμηση σταματά όταν supervisor_id IS NULL (κορυφή ιεραρχίας).

root_id:

Διατηρεί αναφορά στον αρχικό ιατρό σε κάθε βήμα αναδρόμησης, ώστε το τελικό SELECT να μπορεί να επιστρέψει την πλήρη ιεραρχία ομαδοποιημένη ανά ιατρό.

CASE level:

Μετατρέπει τον αριθμητικό αριθμό επιπέδου σε εύληπτη ετικέτα για τον χρήστη.

Q14: Κατηγορίες ICD-10 με Ίδιο Αριθμό Εισαγωγών σε Συνεχόμενα Έτη

Σκοπός

Εντοπίζει κατηγορίες ICD-10 που παρουσίασαν τον ίδιο ακριβώς αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος. Αποτελεί εργαλείο επιδημιολογικής ανάλυσης για εντοπισμό σταθερών τάσεων.

Κώδικας SQL

```

WITH yearly_counts AS (
    SELECT
        i.category,
        EXTRACT(YEAR FROM h.admission_date)::INT AS year,
        COUNT(h.hospitalization_id) AS hosp_count
    FROM hospitalization h
    JOIN icd10_code i ON h.admission_diag_icd10 = i.icd10_code
    GROUP BY i.category, EXTRACT(YEAR FROM h.admission_date)
    HAVING COUNT(h.hospitalization_id) >= 5
)
SELECT y1.category, y1.year AS year1, y2.year AS year2,
       y1.hosp_count AS count_year1, y2.hosp_count AS count_year2
FROM yearly_counts y1
JOIN yearly_counts y2
    ON y1.category = y2.category
   AND y2.year = y1.year + 1
   AND y1.hosp_count = y2.hosp_count
ORDER BY y1.category, y1.year;

```

Ανάλυση Επίλυσης

CTE yearly_counts:

Υπολογίζει ανά κατηγορία ICD-10 ανά έτος τον αριθμό εισαγωγών, με HAVING >= 5 για φιλτράρισμα μικρών δειγμάτων.

Self-join με y2.year = y1.year + 1:

Συνδέει κάθε έτος με το αμέσως επόμενο. Η συνθήκη y1.hosp_count = y2.hosp_count εντοπίζει κατηγορίες με πανομοιότυπο αριθμό εισαγωγών σε συνεχόμενα έτη.

Αποτέλεσμα:

Κάθε γραμμή δείχνει μια κατηγορία ICD-10 με τα δύο έτη και τον αριθμό εισαγωγών (ίδιος και στα δύο).

Q15: Κατανομή Triage ανά Επίπεδο Επείγοντος

Σκοπός

Αναλύει τα περιστατικά triage ανά επίπεδο επείγοντος, υπολογίζοντας μέσο χρόνο αναμονής, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα. Αποτελεί βασικό εργαλείο ανάλυσης λειτουργίας τμήματος επειγόντων περιστατικών.

Κώδικας SQL

```

SELECT
    t.urgency_level,
    CASE t.urgency_level
        WHEN 1 THEN 'Άμεσο' WHEN 2 THEN 'Επείγον'
        WHEN 3 THEN 'Επιτακτικό' WHEN 4 THEN 'Λιγότερο επείγον' WHEN 5 THEN 'Μη
επείγον'
    END AS urgency_label,
    SUM(COUNT(*)) OVER (PARTITION BY t.urgency_level) AS total_cases,
    ROUND(AVG(EXTRACT(EPOCH FROM (h.admission_date::TIMESTAMP -
t.arrival_time)) / 60.0)
        ::NUMERIC, 2) AS avg_wait_minutes,
    SUM(COUNT(*) FILTER (WHERE t.outcome = 'παραπομπή για νοσηλεία'))
        OVER (PARTITION BY t.urgency_level) AS hospitalized_count,
    ROUND(100.0 * SUM(COUNT(*) FILTER (WHERE t.outcome = 'παραπομπή για
νοσηλεία'))
        OVER (PARTITION BY t.urgency_level) / SUM(COUNT(*)) OVER (PARTITION BY
t.urgency_level), 2) AS hospitalization_pct,
    t.referred_dept_name,
    COUNT(*) AS referrals_to_dept,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (PARTITION BY t.urgency_level),
2) AS pct_referrals_within_level
FROM triage t
LEFT JOIN hospitalization h ON t.triage_id = h.triage_id
GROUP BY t.urgency_level, t.referred_dept_name
ORDER BY t.urgency_level, referrals_to_dept DESC;

```

Ανάλυση Επίλυσης

Window functions με PARTITION BY urgency_level:

Το `SUM(COUNT(*)) OVER (PARTITION BY urgency_level)` υπολογίζει το σύνολο περιστατικών ανά επίπεδο επείγοντος, ενώ το `GROUP BY` γίνεται ανά τμήμα. Οι window functions επιτρέπουν ταυτόχρονη ανάλυση σε δύο επίπεδα κοκκοποίησης.

COUNT(*) FILTER (WHERE outcome = ...):

Μετρά μόνο τα περιστατικά που οδήγησαν σε νοσηλεία, χωρίς να απαιτείται `CASE WHEN`. Η σύνταξη `FILTER` είναι πιο εκφραστική και αποδοτική.

EXTRACT(EPOCH FROM ...):

Μετατρέπει τη διαφορά χρόνων (interval) σε δευτερόλεπτα, τα οποία διαιρούνται με 60 για μετατροπή σε λεπτά. Απαραίτητο γιατί το PostgreSQL δεν επιτρέπει άμεση αριθμητική σε intervals με `AVG`.

LEFT JOIN hospitalization:

Χρησιμοποιείται LEFT JOIN ώστε να εμφανίζονται και triage περιστατικά που δεν οδήγησαν σε νοσηλεία (h.triage_id IS NULL).

Κεφάλαιο Z: Διεπαφή Χρήστη (Hospital Management UI)

Το σύστημα διαχείρισης του Γενικού Νοσοκομείου «Υγειόπολης» συνοδεύεται από πλήρη web εφαρμογή διαχείρισης (Hospital Management System — HMS), η οποία παρέχει γραφικό περιβάλλον για όλες τις λειτουργίες της βάσης δεδομένων. Η εφαρμογή αναπτύχθηκε σε HTML5/CSS3/JavaScript (frontend) με REST API backend σε Python Flask, και επικοινωνεί αποκλειστικά με τη βάση PostgreSQL μέσω των endpoints του API.

Z.1 Αρχιτεκτονική Εφαρμογής

Frontend

Single-page application (SPA) γραμμένη σε vanilla HTML/CSS/JavaScript. Χρησιμοποιεί custom CSS variables για theming (navy, burgundy, muted tones) και Google Fonts (Instrument Serif για τίτλους, Inter για σώμα κειμένου). Δεν χρησιμοποιεί εξωτερικά frameworks (React, Vue κτλ.) — όλη η λογική navigation και state management είναι custom.

Backend

Flask REST API (Python) που τρέχει στο localhost:5000. Κάθε σελίδα της εφαρμογής επικοινωνεί με αντίστοιχα /api/ endpoints μέσω fetch() calls. Το API διαβάζει και γράφει αποκλειστικά στη βάση PostgreSQL, χωρίς caching ή intermediate layer.

Σχεδιαστική φιλοσοφία

Η διεπαφή σχεδιάστηκε με γνώμονα τη χρηστικότητα σε νοσοκομειακό περιβάλλον: υψηλή αναγνωσιμότητα, σαφής ιεράρχηση πληροφοριών, γρήγορη πρόσβαση στις συχνότερες ενέργειες (quick actions στο Dashboard) και αποφυγή περιττών βημάτων κατά την εισαγωγή δεδομένων.

Z.2 Dashboard — Κεντρική Οθόνη

Η αρχική οθόνη (Dashboard) παρέχει στο χρήστη άμεση εικόνα της τρέχουσας κατάστασης του νοσοκομείου μέσω 4 βασικών δεικτών (KPIs) και γρήγορες ενέργειες για τις πιο συχνές λειτουργίες.

Δείκτες KPI (πάνω μέρος οθόνης)

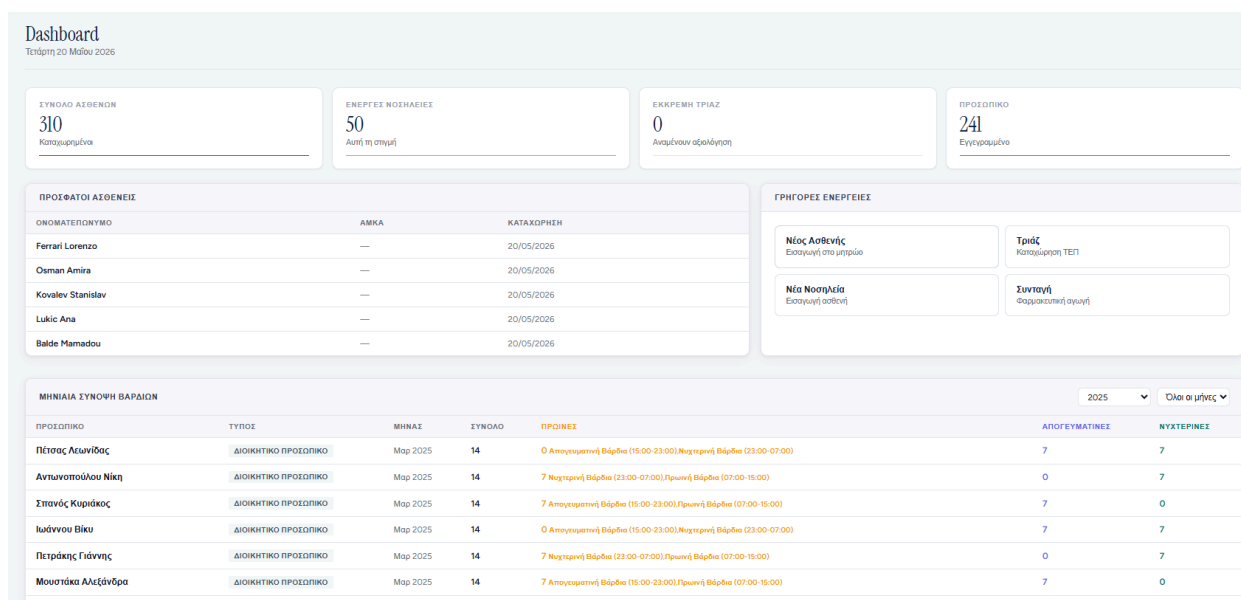
Σύνολο Ασθενών — Ενεργές Νοσηλείες — Εκκρεμή Τριάζ — Εγγεγραμμένο Προσωπικό. Οι τιμές ανανεώνονται σε πραγματικό χρόνο από το `/api/stats` endpoint κατά κάθε φόρτωση της σελίδας.

Γρήγορες Ενέργειες

Κουμπιά άμεσης πρόσβασης για: Νέος Ασθενής, Νέα Νοσηλεία, Νέο Τριάζ, Νέα Συνταγή — οι τέσσερις πιο συχνές εισαγωγές δεδομένων στην καθημερινή λειτουργία.

Πρόσφατοι Ασθενείς

Λίστα με τους τελευταίους εγγεγραμμένους ασθενείς για γρήγορη πρόσβαση στις καρτέλες τους.



Εικόνα Z.2: Κεντρική οθόνη Dashboard με δείκτες KPI και γρήγορες ενέργειες

Z.3 Μητρώο Ασθενών

Η ενότητα Μητρώου Ασθενών παρέχει πλήρη διαχείριση αρχείου ασθενών με δυνατότητες αναζήτησης, εισαγωγής, επεξεργασίας και προβολής λεπτομερούς καρτέλας.

Λίστα Ασθενών

Πίνακας με όλους τους καταχωρημένους ασθενείς. Εμφανίζει: ID, ονοματεπώνυμο, ΑΜΚΑ, ημερομηνία γέννησης, φύλο, υπηκοότητα, ασφαλιστικό φορέα. Υποστηρίζει αναζήτηση με ελεύθερο κείμενο (όνομα, ΑΜΚΑ) και φίλτρο ανά φύλο.

Εισαγωγή Νέου Ασθενή

Modal φόρμα με πλήρη δημογραφικά πεδία: ΑΜΚΑ, ονοματεπώνυμο, πατρώνυμο, ημερομηνία γέννησης, φύλο, διεύθυνση (οδός/πόλη/ΤΚ), τηλέφωνο, email, επάγγελμα, υπηκοότητα (dropdown ISO χωρών), ασφαλιστικός φορέας. Υποστηρίζει emergency εισαγωγή με ελάχιστα πεδία.

Καρτέλα Ασθενή

Αναλυτικό προφίλ με: δημογραφικά στοιχεία, ιστορικό νοσηλειών, ενεργές συνταγές, καταγεγραμμένες αλλεργίες σε δραστικές ουσίες. Επιτρέπει επεξεργασία στοιχείων απευθείας από την καρτέλα.

Μητρώο Ασθενών
Διαχείριση αρχείου ασθενών

Νέος Ασθενής

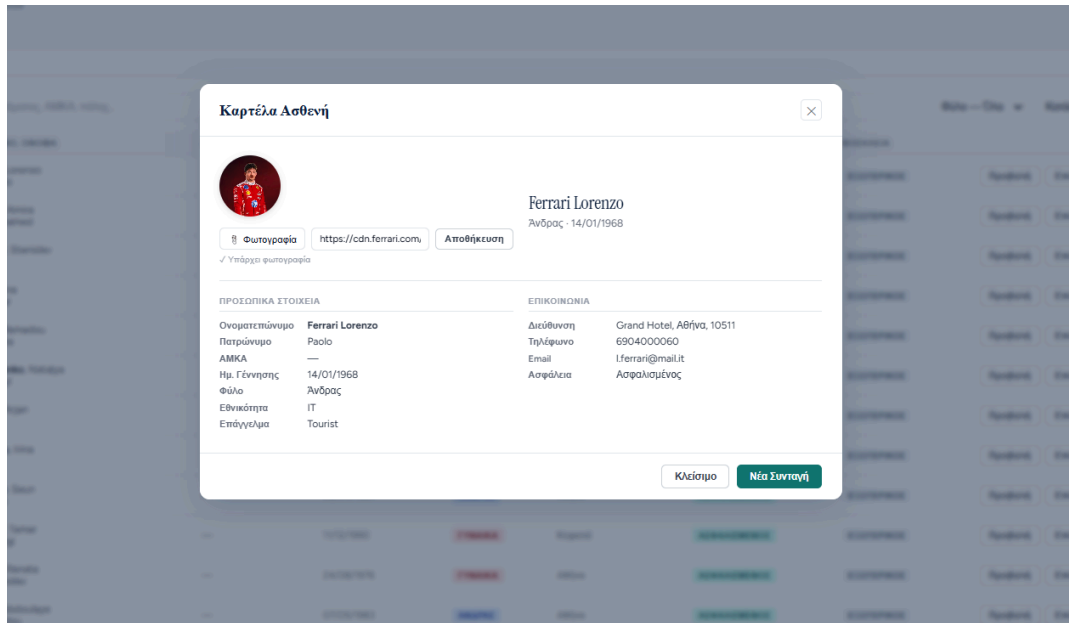
ΑΣΘΕΝΕΙΣ (310)

Αναζήτηση ανάμεσα ΑΜΚΑ, πόλη...

Φύλο — Όλοι Κατάσταση — Όλοι Ασφάλεια — Όλοι

ID	ΕΠΩΝΥΜΟ, ΟΝΟΜΑ	ΑΜΚΑ	ΗΜ. ΓΕΝΝΗΣΗΣ	ΦΥΛΟ	ΠΟΛΗ	ΑΣΦΑΛΕΙΑ	ΝΟΣΗΛΕΙΑ			
310	Ferrari, Lorenzo του Paolo	—	14/01/1968	ΑΝΔΡΑΣ	Αθήνα	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
309	Osman, Amira του Mohamed	—	27/09/1991	ΓΥΝΑΙΚΑ	Αθήνα	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
308	Kovalev, Stanislav του Igor	—	10/05/1983	ΑΝΔΡΑΣ	Καλλιθέα	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
307	Lukic, Ana του Petar	—	23/01/1977	ΓΥΝΑΙΚΑ	Νέα Ιωνία	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
306	Baldie, Mamadou του Alpha	—	06/09/1980	ΑΝΔΡΑΣ	Αθήνα	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
305	Kravchenko, Natalya του Vasily	—	19/06/1995	ΓΥΝΑΙΚΑ	Νέα Σμύρνη	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
304	Hozha, Arjan του Iir	—	02/01/1988	ΑΝΔΡΑΣ	Αθήνα	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
303	Popescu, Irina του Ion	—	15/08/1971	ΓΥΝΑΙΚΑ	Αθήνα	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
302	Adieleke, Seun του Bola	—	28/04/1984	ΑΝΔΡΑΣ	Αθήνα	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
301	Beridze, Tamar του Giorgi	—	11/02/1990	ΓΥΝΑΙΚΑ	Κηφισιά	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή
300	Horvat, Renata του Tomislav	—	24/08/1976	ΓΥΝΑΙΚΑ	Αθήνα	ΑΣΦΑΛΙΣΜΕΝΟΣ	ΕΣΩΤΕΡΙΚΟΣ	Προβολή	Επεξεργασία	Διαγραφή

Εικόνα Z.3.1: Λίστα ασθενών με φίλτρα αναζήτησης



Εικόνα Z.3.2: Αναλυτική καρτέλα ασθενή με ιστορικό νοσηλειών και αλλεργίες

Z.4 Τριάζ / Τμήμα Επειγόντων Περιστατικών

Η ενότητα Τριάζ αναπαριστά την ουρά αναμονής του ΤΕΠ και διαχειρίζεται τη διαδικασία εισαγωγής και αξιολόγησης ασθενών.

Ουρά Τριάζ

Λίστα ασθενών που αναμένουν αξιολόγηση (outcome IS NULL), ταξινομημένη κατά επίπεδο επείγοντος (1=άμεσο → 5=μη επείγον) και χρόνο άφιξης (FIFO εντός επιπέδου). Εμφανίζει: ώρα άφιξης, επίπεδο επείγοντος με χρωματική κωδικοποίηση, συμπτώματα, αρμόδιο νοσηλευτή.

Καταχώρηση Τριάζ

Modal φόρμα για νέα διαλογή: επιλογή ασθενή, νοσηλευτής διαλογής (από τους νοσηλευτές που είναι σε βάρδια αυτή τη στιγμή), συμπτώματα, επίπεδο επείγοντος, βάρος/ύψος, έκβαση (αποχώρηση με οδηγίες ή παραπομπή για νοσηλεία με επιλογή τμήματος).

Τριάζ / ΤΕΠ				
Τμήμα Επειγόντων Περιστατικών				
Νέο Περιστατικό				
ΟΥΡΑ ΤΡΙΑΖ				
Αναζήτηση ασθενή...				
Επίπεδο — Όλα Κατάσταση — Όλα				
ΑΣΘΕΝΗΣ	ΑΦΙΞΗ	ΕΠΕΙΓΟΝ	ΣΥΜΠΤΩΜΑΤΑ	ΚΑΤΑΣΤΑΣΗ
Οικονόμου Παναγιώτης	31/12/2024, 9:00:00 μ.μ.	Κρίσιμο	Τρίτη εισαγωγή — νεφρική ανεπάρκεια σοβαρή.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Σκούρης Νικόδημος	23/1/2025, 9:00:00 μ.μ.	Κρίσιμο	Λοίμωξη — σηψαιμία αρχόμενη.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Τσούκα Αγγελική	4/2/2025, 9:00:00 μ.μ.	Κρίσιμο	Παιδί 5 ετών, εισρόφηση αντικαμίνου, δύσπνοια, κούραση...	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Μαντζούφας Σταύρος	10/2/2025, 9:00:00 μ.μ.	Κρίσιμο	Παγκρεατίτιδα — οξεία.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Hoxha Agim	16/2/2025, 9:00:00 μ.μ.	Κρίσιμο	Acute abdomen — surgery.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Ahmed Zara	24/2/2025, 9:00:00 μ.μ.	Κρίσιμο	Σηψαμία — επανεισαγωγή.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Γκούμα Βαλέρα	7/3/2025, 9:00:00 μ.μ.	Κρίσιμο	Υπογλυκαιμία υποτροπή — απώλεια συνείδησης.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Sharma Arjun	31/3/2025, 9:00:00 μ.μ.	Κρίσιμο	Chest pain, sweating, shortness of breath. Possible MI.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Σαββίδου Κωνσταντίνα	1/4/2025, 9:00:00 μ.μ.	Κρίσιμο	Παιδί — υποτροπή εμπύρετων σπασμών.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Okafar Blessing	17/4/2025, 9:00:00 μ.μ.	Κρίσιμο	Infection — sepsis suspected.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Παπαδόπουλος Γεώργιος	12/5/2025, 9:00:00 μ.μ.	Κρίσιμο	Τρίτη εισαγωγή — καρδιακή ανακοπή.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Okonkwo Mohammed	16/5/2025, 9:00:00 μ.μ.	Κρίσιμο	Μηνιγγίτιδα — επανεισαγωγή.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Αναστασία Μαρία	23/5/2025, 9:00:00 μ.μ.	Κρίσιμο	Παρεγκεφαλικό ΑΕΕ — σπινιά.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Σεραφείμ Αλέξης	31/5/2025, 9:00:00 μ.μ.	Κρίσιμο	Τέταρτη εισαγωγή — σηψαμία.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Σαββίδου Κωνσταντίνα	13/6/2025, 9:00:00 μ.μ.	Κρίσιμο	Παιδί 8 ετών, υψηλός πυρετός 40°C, σπασμοί. Εμπύρετοι σ...	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Σίτανιδου Βασιλική	18/6/2025, 9:00:00 μ.μ.	Κρίσιμο	Ημipληγία — ΑΕΕ υποτροπή.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ
Χρυσόστομου Ευθύμιος	20/6/2025, 9:00:00 μ.μ.	Κρίσιμο	Τρίτη εισαγωγή — STEMI επανόληξη.	ΠΑΡΑΠΟΜΠΗ ΓΙΑ ΝΟΣΗΛΕΙΑ

Εικόνα Z.4: Ουρά αναμονής ΤΕΠ ταξινομημένη κατά επείγον και χρόνο άφιξης

Z.5 Νοσηλείες

Η ενότητα Νοσηλείων καταγράφει και διαχειρίζεται το σύνολο των νοσηλείων — ενεργών και ολοκληρωμένων.

Αρχείο Νοσηλείων

Πίνακας με όλες τις νοσηλείες. Εμφανίζει: ID νοσηλείας, ασθενή, τμήμα, κλίνη, ημερομηνία εισαγωγής, ημερομηνία εξόδου (κενό αν ενεργή), κωδικό KEN, συνολικό κόστος. Οι ενεργές νοσηλείες (χωρίς discharge_date) εμφανίζονται πρώτες με ειδική σήμανση.

Καταχώρηση Νοσηλείας

Modal φόρμα: επιλογή ασθενή, τμήμα, διαθέσιμη κλίνη (από το view v_available_beds), ημερομηνία εισαγωγής, διάγνωση εισαγωγής (ICD-10 dropdown), κωδικός KEN. Συνδέεται υποχρεωτικά με εγγραφή Τριάζ.

Ανάθεση Προσωπικού

Από την καρτέλα κάθε νοσηλείας, δυνατότητα ανάθεσης θεράποντα ιατρού, συμβούλου ιατρού και υπεύθυνου νοσηλευτή (hospitalization_staff).

Επεξεργασία / Έξοδος

Συμπλήρωση discharge_date, κωδικού ICD-10 εξόδου και KEN — το σύστημα υπολογίζει αυτόματα το κόστος μέσω trigger.

Νοσηλείες
Ενεργές και ιστορικές νοσηλείες

Νέα Νοσηλεία

ΑΡΧΕΙΟ ΝΟΣΗΛΕΙΩΝ

Αναζήτηση ασθενή, ICD-10...

Κατάσταση — Όλες

Νευρολογία

ID	ΑΣΘΕΝΗΣ	ΤΜΗΜΑ	ΚΡΕΒΑΤΙ	ΕΙΣΑΓΟΓΗ	ΕΞΑΓΟΓΗ	ICD-10	Επεξεργασία	Διαγραφή
982	Παπαθεοδώρου Αναστάσης	Νευρολογία	547	08/05/2026	ΕΝΕΡΓΗ	V01	Επεξεργασία	Διαγραφή
967	Γεωργαντά Βαββάρα	Νευρολογία	532	13/05/2026	ΕΝΕΡΓΗ	M05	Επεξεργασία	Διαγραφή
952	Νούσια Κλεοπάτρα	Νευρολογία	517	18/05/2026	ΕΝΕΡΓΗ	H25	Επεξεργασία	Διαγραφή
937	Κόκκινος Σπύρος	Νευρολογία	502	03/05/2026	ΕΝΕΡΓΗ	D52	Επεξεργασία	Διαγραφή
930	Μιχαλοπούλου Χριστίνα	Νευρολογία	409	08/04/2026	15/04/2026	S10	Επεξεργασία	Διαγραφή
923	Γαλάνης Βαγγέλης	Νευρολογία	402	08/04/2025	15/04/2025	S10	Επεξεργασία	Διαγραφή
501	Παπαγεωργίου Σάκης	Νευρολογία	111	01/01/2025	08/01/2025	A01	Επεξεργασία	Διαγραφή
449	Αντωνίου Δέσποινα	Νευρολογία	102	04/03/2025	09/03/2025	A15	Επεξεργασία	Διαγραφή
439	Χριστοφίδης Θανάσης	Νευρολογία	101	25/09/2025	29/09/2025	A54.8	Επεξεργασία	Διαγραφή
424	Γεωργακοπούλου Ανθή	Νευρολογία	101	27/12/2026	01/01/2027	C20	Επεξεργασία	Διαγραφή
415	Παπαγεωργίου Σάκης	Νευρολογία	111	22/07/2026	26/07/2026	B44.0	Επεξεργασία	Διαγραφή
407	Leika Gjergji	Νευρολογία	101	07/10/2025	09/10/2025	A56.2	Επεξεργασία	Διαγραφή

Εικόνα Z.5: Αρχείο νοσηλείων με ανάλυση ενεργών και ολοκληρωμένων

Z.6 Διαχείριση Προσωπικού

Η εφαρμογή διαθέτει τρεις ξεχωριστές ενότητες για τις τρεις κατηγορίες προσωπικού, αντανακλώντας την ISA ιεραρχία της βάσης.

Z.6.1 Ιατρικό Προσωπικό

Λίστα Ιατρών

Πίνακας με όλους τους ιατρούς: ονοματεπώνυμο, ειδικότητα, βαθμίδα (Ειδικευόμενος / Επιμελητής Β / Επιμελητής Α / Διευθυντής), αριθμός άδειας, τμήμα(τα).

Καταχώρηση / Επεξεργασία

Modal φόρμα με πλήρη στοιχεία: ΑΜΚΑ, ονοματεπώνυμο, ειδικότητα (dropdown από doctor_specialty), βαθμίδα, επόπτης (dropdown — εμφανίζει μόνο ιατρούς ανώτερης βαθμίδας), τμήμα(τα) ανάθεσης.

Διαθεσιμότητα Γιατρών

Ξεχωριστή σελίδα που εμφανίζει ποιοι ιατροί είναι on_shift / on_duty / available αυτή τη στιγμή, με αριθμό ενεργών ασθενών, ταξινομημένους ανά φόρτο εργασίας (από το view v_staff_availability).

Ιατρικό Προσωπικό					
Διαχείριση και προβολή γιατρών					
Αναζήτηση ανάματος, ειδικότητας... Επιμελητής Β					
ID	ΕΠΩΝΥΜΟ, ΟΝΟΜΑ	ΕΙΔΙΚΟΤΗΤΑ	ΒΑΘΜΟΣ	ΗΜ. ΠΡΟΣΕΛΗΨΗΣ	
291	 Αλεξανδροπούλου, Σοφία	Μαιευτική-Γυναικολογία	ΕΠΙΜΕΛΗΤΗΣ Β	01/10/2013	Προβολή Επεξεργασία Διαγραφή
290	 Σπηλιωτόπουλος, Λεωνίδας	Ουρολογία	ΕΠΙΜΕΛΗΤΗΣ Β	01/04/2015	Προβολή Επεξεργασία Διαγραφή
289	 Βασιλείου, Αικατερίνη	Ορθοπαιδική	ΕΠΙΜΕΛΗΤΗΣ Β	01/07/2014	Προβολή Επεξεργασία Διαγραφή
288	 Παπαχρήστου, Δημήτριος	Χειρουργική	ΕΠΙΜΕΛΗΤΗΣ Β	01/02/2016	Προβολή Επεξεργασία Διαγραφή
287	 Χατζηγιάννης, Νικόλαος	Ψυχιατρική	ΕΠΙΜΕΛΗΤΗΣ Β	01/11/2015	Προβολή Επεξεργασία Διαγραφή
286	 Μουστάκη, Ελισάβετ	Παιδιατρική	ΕΠΙΜΕΛΗΤΗΣ Β	01/09/2014	Προβολή Επεξεργασία Διαγραφή
285	 Ανδριόδης, Σταύρος	Νεφρολογία	ΕΠΙΜΕΛΗΤΗΣ Β	01/06/2016	Προβολή Επεξεργασία Διαγραφή
284	 Κολοκοτρώνη, Μαρία	Γαστρεντερολογία	ΕΠΙΜΕΛΗΤΗΣ Β	01/03/2013	Προβολή Επεξεργασία Διαγραφή
283	 Σταββίδης, Ανδρέας	Πνευμονολογία	ΕΠΙΜΕΛΗΤΗΣ Β	10/01/2015	Προβολή Επεξεργασία Διαγραφή
65	 Θεοδώρου, Ουρανία	Ογκολογία	ΕΠΙΜΕΛΗΤΗΣ Β	01/08/2007	Προβολή Επεξεργασία Διαγραφή
64	 Κανελλόπουλος, Σταύρος	Νευρολογία	ΕΠΙΜΕΛΗΤΗΣ Β	01/07/2007	Προβολή Επεξεργασία Διαγραφή

Εικόνα Z.6.1: Λίστα ιατρικού προσωπικού με ειδικότητα, βαθμίδα και διαθεσιμότητα

Z.6.2 Νοσηλευτικό & Διοικητικό Προσωπικό

Αντίστοιχες ενότητες για νοσηλευτές (με βαθμίδα και τμήμα) και διοικητικό προσωπικό (με ρόλο και γραφείο). Κάθε ενότητα παρέχει λίστα, φόρμα εισαγωγής/επεξεργασίας και προφίλ μέλους.

Νοσηλευτικό Προσωπικό						Νέος Νοσηλευτής
Αναζήτηση ανάματος...						Βοηθός Νοσηλεύη ΜΕΘ
ID	ΕΠΩΝΥΜΟ, ΟΝΟΜΑ	ΒΑΘΜΟΣ	ΤΜΗΜΑ			
232	 Μακρής Σπαύτης	ΒΟΗΘΟΣ ΝΟΣΗΛΕΥΤΗ	ΜΕΘ	Προβολή	Επεξεργασία	Διαγραφή
231	 Τσιμπούκη Ασπασία	ΒΟΗΘΟΣ ΝΟΣΗΛΕΥΤΗ	ΜΕΘ	Προβολή	Επεξεργασία	Διαγραφή
230	 Κανελλής Γρηγόρης	ΒΟΗΘΟΣ ΝΟΣΗΛΕΥΤΗ	ΜΕΘ	Προβολή	Επεξεργασία	Διαγραφή
229	 Δαμανού Ανθή	ΒΟΗΘΟΣ ΝΟΣΗΛΕΥΤΗ	ΜΕΘ	Προβολή	Επεξεργασία	Διαγραφή
228	 Πασάς Εμμανουήλ	ΒΟΗΘΟΣ ΝΟΣΗΛΕΥΤΗ	ΜΕΘ	Προβολή	Επεξεργασία	Διαγραφή
227	 Ντούρου Κρυσταλία	ΒΟΗΘΟΣ ΝΟΣΗΛΕΥΤΗ	ΜΕΘ	Προβολή	Επεξεργασία	Διαγραφή

Εικόνα Z.6.2: Λίστα νοσηλευτικού προσωπικού με βαθμίδα και τμήμα ανάθεσης

Z.7 Συνταγογράφηση Φαρμάκων

Η ενότητα Συνταγών διαχειρίζεται τη φαρμακευτική αγωγή ασθενών κατά τη νοσηλεία, με αυτόματο έλεγχο αλλεργιών.

Αρχείο Συνταγών

Πίνακας με όλες τις συνταγογραφήσεις: ασθενής, συνταγογράφος ιατρός, φάρμακο, δοσολογία, συχνότητα, ημερομηνία έναρξης/λήξης, νοσηλεία.

Νέα Συνταγή

Modal φόρμα: επιλογή ασθενή, νοσηλεία, ιατρός (πρέπει να ανήκει στο hospitalization_staff), φάρμακο (searchable dropdown από τον κατάλογο EMA), δοσολογία, συχνότητα, ημερομηνίες. Το backend απορρίπτει αυτόματα συνταγές που παραβιάζουν αλλεργίες (trigger trg_allergy_check) και εμφανίζει ενημερωτικό μήνυμα στον χρήστη.

Εικόνα Z.7: Αρχείο συνταγογραφήσεων με φίλτρα ανά ασθενή και νοσηλεία

Z.8 Εφημερίες & Προγραμματισμός Βαρδιών

Η ενότητα Εφημεριών παρέχει πλήρη εικόνα του προγραμματισμού βαρδιών με ημερήσια και εβδομαδιαία προβολή.

Ημερήσια Προβολή

Φίλτρα ανά ημερομηνία, τμήμα και τύπο βάρδιας (Πρωινή/Απογευματινή/Νυχτερινή). Εμφανίζει τα μέλη προσωπικού κάθε βάρδιας με ρόλο και κατηγορία, μαζί με περίληψη (πόσα άτομα ανά τύπο βάρδιας).

Εβδομαδιαία Εικόνα

Grid προβολή 7 ημερών × τμήματα, εμφανίζει τον αριθμό εφημερευόντων ανά βάρδια/τμήμα/ημέρα για γρήγορη επισκόπηση κάλυψης.

Διαθεσιμότητα βαρδιών

Ξεχωριστή σελίδα που εμφανίζει για κάθε τύπο βάρδια σήμερα πόσοι ιατροί είναι σε υπηρεσία, ανά τμήμα.

Εφημερίες
Οργάνωση βαρδιών και duty assignments ανά τμήμα

ΗΜΕΡΟΜΗΝΙΑ: 03/03/2025 ΤΜΗΜΑ: Όλα τα Τμήματα ΒΑΡΔΙΑ: Όλες Ανανέωση

ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΒΑΡΔΙΩΝ						Δευτέρα 3 Μαρτίου 2025
ΒΑΡΔΙΑ	ΟΡΕΣ	ΠΡΟΣΩΠΙΚΟ	ΤΜΗΜΑ	ΤΥΠΟΣ	SHIFT ID	
ΠΡΩΙΝΗ ΒΑΡΔΙΑ (07:00-15:00)	—	ΝΟΣΗΛΕΥΤΗΣ Αλεξάνδρης Φώτης ΝΟΣΗΛΕΥΤΗΣ Αρβανίτη Θεοδώρα — Προϊστάμενος ΝΟΣΗΛΕΥΤΗΣ Κατωπόδη Φωτεινή — Βοηθός Νοσηλεύτη ΝΟΣΗΛΕΥΤΗΣ Καραλής Ηλίας — Βοηθός Νοσηλεύτη ΝΟΣΗΛΕΥΤΗΣ Κουτσούρη Ντίνα ΓΙΑΤΡΟΣ Κωνσταντίνου Δημήτριος — Χειρουργική ΓΙΑΤΡΟΣ Λάμπρου Πέτρος — Χειρουργική ΝΟΣΗΛΕΥΤΗΣ Μανωλάς Γιώργος — Νοσηλεύτης ΓΙΑΤΡΟΣ Παπαχρήστου Δημήτριος — Χειρουργική ΝΟΣΗΛΕΥΤΗΣ Παρασκευάς Δαμιανός — Βοηθός Νοσηλεύτη ΝΟΣΗΛΕΥΤΗΣ Τζύμα Ανδρονίκη — Βοηθός Νοσηλεύτη	Χειρουργική	11 άτομα	#127	
ΑΠΟΓΕΥΜΑΤΙΝΗ ΒΑΡΔΙΑ (15:00-23:00)	—	ΝΟΣΗΛΕΥΤΗΣ Αλεξάνδρης Φώτης ΝΟΣΗΛΕΥΤΗΣ Βλάχος Αλέκος — Βοηθός Νοσηλεύτη ΝΟΣΗΛΕΥΤΗΣ Θεοδωράκης Πάνος — Νοσηλεύτης ΝΟΣΗΛΕΥΤΗΣ Κατωπόδη Φωτεινή — Βοηθός Νοσηλεύτη ΝΟΣΗΛΕΥΤΗΣ Κουτσούρη Ντίνα — Βοηθός Νοσηλεύτη ΓΙΑΤΡΟΣ Λάμπρου Πέτρος — Χειρουργική ΝΟΣΗΛΕΥΤΗΣ Μπαλτάς Νίκος ΝΟΣΗΛΕΥΤΗΣ Παρασκευάς Δαμιανός — Βοηθός Νοσηλεύτη ΓΙΑΤΡΟΣ Πολίτης Τάκης — Χειρουργική ΝΟΣΗΛΕΥΤΗΣ Σισόη Αγάπη — Νοσηλεύτης ΓΙΑΤΡΟΣ Χαλκίδης Νικόλαος — Χειρουργική	Χειρουργική	11 άτομα	#128	
		ΝΟΣΗΛΕΥΤΗΣ Αρβανίτη Θεοδώρα — Προϊστάμενος ΝΟΣΗΛΕΥΤΗΣ Θεοδωράκης Πάνος — Νοσηλεύτης ΝΟΣΗΛΕΥΤΗΣ Καραλής Ηλίας — Βοηθός Νοσηλεύτη ΝΟΣΗΛΕΥΤΗΣ Κουτσούρη Ντίνα				

Εικόνα Z.8.1: Ημερήσια προβολή εφημεριών με φίλτρα ανά τμήμα και βάρδια

[illegible]

Εικόνα Ζ.8.2: Εβδομαδιαία εικόνα βαρδιών ανά τμήμα

Z.9 Χειρουργεία & Επεμβάσεις

Η ενότητα Χειρουργείων παρέχει αρχείο εκτελεσμένων ιατρικών πράξεων και επεμβάσεων ανά νοσηλεία, με στοιχεία εκτελούντος ιατρού, ημερομηνίας και αίθουσας.

Ιατρικές Πράξεις & Εξετάσεις

Χειρουργικές, Θεραπευτικές & Διαγνωστικές πράξεις - Εργαστηριακές εξετάσεις

Ιατρικές Πράξεις

Εργαστηριακές Εξετάσεις

ΑΡΧΕΙΟ ΙΑΤΡΙΚΩΝ ΠΡΑΞΕΩΝ

Αναζήτηση κωδικού, ονομασίας...

Κατηγορία — Όλες

ημ/μμ/εεεε

ημ/μμ/εεεε

ID	ΑΙΤΩΝΗΣ	ΤΜΗΜΑ	ΠΡΑΞΗ	ΚΑΤΗΓΟΡΙΑ	ΕΠΙΧΕΦΑΛΗΣ ΙΑΤΡΟΣ	ΕΝΑΡΞΗ	ΚΟΣΤΟΣ ΚΕΝ
#334	Αθηνά Χαλκοπού	Νεφρολογία	ΑΝΑΤΑΣΗ ΚΑΤΑΓΜΑΤΟΣ ΤΗΣ ΔΙΑΦΥΣΗΣ ΤΗΣ ΚΝΗΜΗΣ (ΜΕ Ή ΧΩΡΙΣ ΚΑΤΑΓΜΑ ΠΕΡΟΝΗΣ), ΚΑΙ ΟΣΤΕΟΣΥΝΘΕΣΗ ΜΕ ΕΝΔΟΜΥΕΛΙΚΟ ΗΛΟ	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Ζωή Τούλη Νεφρολογία	7/8/25, 3:00 π.μ.	2.316,00 €
#325	Ευτύχης Μεταξάς	Χειρουργική	ΛΑΠΑΡΟΣΚΟΠΗΣΗ, ΧΕΙΡΟΥΡΓΙΚΗ ΡΙΖΙΚΗ ΝΕΦΡΕΚΤΟΜΗ (ΠΕΡΙΛΑΜΒΑΝΕΙ ΑΦΑΙΡΕΣΗ ΤΗΣ ΝΕΦΡΙΚΗΣ ΠΕΡΙΤΟΝΙΑΣ ΚΑΙ ΤΟΥ ΠΕΡΙΒΑΛΛΟΝΤΟΣ ΛΙΠΩΔΟΥΣ ΙΣΤΟΥ, ΑΦΑΙΡΕΣΗ ΤΩΝ ΤΟΠΙΚΩΝ ΛΕΜΦΑΔΕΝΩΝ, ΚΑΙ ΕΠΙΠΕΦΡΙΑΔΕΚΤΟΜΗ)	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Δημήτρης Κωνσταντίνου Χειρουργική	2/8/25, 3:00 π.μ.	992,00 €
#314	Ευτύχης Μεταξάς	Νεφρολογία	ΕΠΙΛΕΜΦΥΤΕΥΣΗ, ΑΝΤΙΧΕΙΡΑΣ (ΠΕΡΙΛΑΜΒΑΝΕΙ ΑΠΟ ΤΗΝ ΚΑΡΠΙΟΜΕΤΑΚΑΡΠΙΑ ΑΡΘΡΩΣΗ ΕΩΣ ΤΗΝ ΜΕΤΑΚΑΡΠΙΟΦΑΛΑΓΓΙΚΗ ΑΡΘΡΩΣΗ), ΟΛΙΚΟΣ ΑΚΡΟΤΗΡΙΑΣΜΟΣ	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Ζωή Τούλη Νεφρολογία	30/7/25, 3:00 π.μ.	992,00 €
#333	Νικόλαος Τζαντινάκης	Ουρολογία	ΧΕΙΛΙΚΟΣ ΚΑΘΑΡΙΣΜΟΣ ΑΙΜΑΤΟΜΑΤΟΣ Ή ΑΠΟΣΤΗΜΑΤΟΣ ΠΥΕΛΟΥ Ή ΚΑΤ'ΙΣΧΙΟΝ ΑΡΘΡΩΣΗΣ	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Κυριάκος Σταμάτιου Ανασθησιολογία	30/7/25, 3:00 π.μ.	992,00 €
#319	Φανή Αποστόλου	Γαστρεντερολογία	ΕΝΔΟΣΚΟΠΗΣΗ ΟΥΡΗΤΗΡΑ ΜΕΣΩ ΤΟΠΟΘΕΤΗΜΕΝΗΣ ΟΥΡΗΤΗΡΟΣΤΟΜΙΑΣ, ΜΕ Ή ΧΩΡΙΣ ΠΛΥΞΗ, ΕΝΣΤΑΛΑΣΗ, Ή ΟΥΡΗΤΗΡΟΠΥΕΛΟΓΡΑΦΙΑ, ΑΝΕΥ ΑΚΤΙΝΟΛΟΓΙΚΗΣ ΥΠΗΡΕΣΙΑΣ, ΜΕ ΑΦΑΙΡΕΣΗ ΒΕΝΟΥ ΣΥΜΜΑΤΟΣ Ή ΛΙΘΟΥ	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Αλεξάνδρα Χριστοπούλου Γαστρεντερολογία	29/7/25, 3:00 π.μ.	992,00 €
#315	Ευτύχης Μεταξάς	Νεφρολογία	ΜΕΡΙΚΗ ΑΙΔΟΕΚΤΟΜΗ, ΑΠΛΗ	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Πόπη Κορράς Νεφρολογία	29/7/25, 3:00 π.μ.	992,00 €
#327	Σοφία Τσαούση	Ουρολογία	ΧΕΙΡΟΥΡΓΙΚΟΣ ΚΑΘΑΡΙΣΜΟΣ: ΔΕΡΜΑ, ΥΠΟΔΟΡΙΣ ΙΣΤΟΣ, ΜΥΣ ΚΑΙ ΟΣΤΟ	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Πηνελόπη Ρούσσου Ουρολογία	29/7/25, 3:00 π.μ.	2.316,00 €
#329	Φανή Αποστόλου	Ορθοπαιδική	ΑΠΟΚΑΤΑΣΤΑΣΗ - ΣΥΡΡΑΦΗ ΚΕΡΚΙΑΚΟΥ ΝΕΥΡΟΥ ΜΕΤΑ ΑΠΟ ΑΦΑΙΡΕΣΗ ΝΕΥΡΩΜΑΤΟΣ ΜΕ ΠΑΡΕΜΒΟΛΗ ΝΕΥΡΙΚΩΝ ΜΟΣΧΕΥΜΑΤΩΝ	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Θωμάς Κυριακίδης Ορθοπαιδική	28/7/25, 3:00 π.μ.	992,00 €
#331	Νικόδημος Παπαχρήστου	Χειρουργική	ΑΦΑΙΡΕΣΗ ΠΡΟΞΗΛΙΑΣ ΕΠΟΝΔΥΛΟΔΕΣΙΑΣ	ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ	Κλεόπας Λαμπρόπουλος Ανασθησιολογία	28/7/25, 3:00 π.μ.	2.316,00 €

Εικόνα Z.9: Αρχείο χειρουργικών επεμβάσεων με εκτελούντα ιατρό και στοιχεία νοσηλείας

Z.10 Τεχνικά Χαρακτηριστικά

Ο παρακάτω πίνακας συνοψίζει τα τεχνολογικά εργαλεία που χρησιμοποιήθηκαν για την ανάπτυξη της εφαρμογής:

Στρώμα	Τεχνολογία / Εργαλείο
Frontend	HTML5, CSS3, JavaScript (Vanilla) — Single Page Application
Γραμματοσειρές	Google Fonts: Instrument Serif (τίτλοι), Inter (σώμα)
Backend	Python 3 — Flask framework — REST API
Βάση Δεδομένων	PostgreSQL 15+
ORM / Queries	psycopg2 (raw SQL) — χωρίς ORM για πλήρη έλεγχο
Επικοινωνία	fetch() API (async/await) — JSON responses
Deployment	localhost:5000 — Flask development server

Κεφάλαιο Η: Χρήση AI για το Project

Χρησιμοποιήσαμε την TN σε **κάθε στάδιο** του project από την αρχική σύλληψη μέχρι τη λεπτομέρεια. Ξεχωρίζουν ιδιαίτερα η χρήση για **debugging λογικών σφαλμάτων** σε δεδομένα (κάτι που χρειάζεται βαθιά κατανόηση του domain).

H.1 Φάση 1 - Σχεδιασμός & Αρχιτεκτονική

Ξεκινήσαμε από μηδέν, ζητώντας βοήθεια για το στήσιμο του επαγγελματικού περιβάλλοντος ανάπτυξης (PostgreSQL, Docker, DBeaver, VS Code με GitHub).

Χρησιμοποιήσαμε την TN για:

- Δημιουργία **DBML schema** για το dbdiagram.io
- Δημιουργία **E-R diagram σε draw.io XML** (Chen notation) με entities, relationships, και σωστή cardinality
- Ονοματοδοσία σχέσεων (relations) για κάθε FK του σχήματος

H.1 Φάση 2 - Ανάπτυξη & Δεδομένα

Με το σχήμα έτοιμο, χρησιμοποιήσαμε την TN για να χτίσουμε τα δεδομένα και να διορθώσουμε σφάλματα:

- **Εντοπισμός bugs** στο SQL script (τυπογραφικά σφάλματα σε ICD-10 κωδικούς, foreign key violations, duplicate entries, λάθος σε triggers)
- **Διόρθωση λογικών σφαλμάτων** στα δεδομένα νοσηλειών (admission > discharge dates, άντρες σε Μαιευτική, παιδιά σε λάθος τμήμα, εξασφάλιση 50 ενεργών νοσηλειών)
- **Δημιουργία 500 εγγγραφών** hospitalization με σωστή δομή και ακεραιότητα
- **Διόρθωση view bugs** (shift_type values που δεν ταίριαζαν με CHECK constraints)

H.1 Φάση 3 - Εφαρμογή & UI

- Στήσιμο **Flask backend** με Docker και σύνδεση στην PostgreSQL
- Δημιουργία **AI-powered hospital database assistant** (HTML artifact) που επιτρέπει ερωτήματα φυσικής γλώσσας στη βάση
- Βοήθεια για σύνδεση pgAdmin με Docker container
- Σύνταξη **SQL queries με CTEs** για ανάλυση triage δεδομένων

H.1 Φάση 4 - Documentation

Προσπαθήσαμε με την βοήθεια της TN να φτιάξουμε τον πιο επαγγελματικό τρόπο τεκμηρίωσης μιας βάσης (README, schema docs, ERD, data dictionary, migration files).